

Sam's Teach Yourself C++17 Notes

Contents

1	Getting Started	6
1.1	History of C++	6
1.2	Programming a C++ Application	7
1.2.1	Steps to Generating an Executable	7
1.2.2	Programming Your First C++ Application	8
1.3	QA	10
1.4	Quiz	11
1.5	Exercises	12
2	The Anatomy of a C++ Program	13
2.1	Parts of the Hello World Program	13
2.2	Preprocessor Directive <code>#include</code>	14
2.3	The Body of Your Program <code>main()</code>	15
2.4	Returning a Value	16
2.5	Namespaces	17
2.6	Functions in C++	19
2.7	Basic Input Using <code>std::cin</code> and Output using <code>std::cout</code>	21
2.8	QA	22
2.9	Quiz	23
2.10	Exercises	24
3	Using Variables, Declaring Constants	25
3.1	What Is a Variable?	25
3.1.1	Memory and Addressing	25
3.1.2	Declaring Variables to Access and Use Memory	26
3.1.3	Declaring and Initializing Multiple Variables of a Type	27
3.1.4	Scope of a Variable	28
3.1.5	Global Variables	29
3.2	Common Compiler-Supported C++ Variable Types	30
3.2.1	Signed vs Unsigned integers	31
3.2.2	Signed integer types: <code>short</code> , <code>int</code> , <code>long</code> , and <code>long long</code>	33
3.2.3	Unsigned integer types: <code>unsigned short</code> , <code>unsigned int</code> , <code>unsigned long</code> , <code>unsigned long long</code>	34
3.2.4	Integer Overflow	35
3.2.5	Floating-Point types <code>float</code> and <code>double</code>	36
3.3	Size of a Variable using <code>sizeof</code>	38
3.4	Automatic Type Inference Using <code>auto</code>	39
3.5	<code>typedef</code> and/or using to Substitute a Variable's Type	40
3.6	What is a Constant?	41
3.6.1	Literal constants	41
3.6.2	Declaring Variables as Constants Using <code>const</code>	43
3.6.3	Constant Expression Using <code>constexpr</code>	44
3.6.4	Enumerations	46
3.6.5	Defining Constants Using <code>#define</code>	48
3.7	Keywords that Cannot be Used as Variable or Constant Names	49
3.8	QA	50
3.9	Quiz	51

4	Managing Arrays and Strings	53
4.1	What is an Array?	53
4.1.1	The Need for Arrays	54
4.1.2	How Data Is Stored in an Array	55
4.1.3	Accessing Data Stored in an Array	56
4.1.4	Modifying Data Stored in an Array	57
4.2	Multidimensional Arrays	58
4.3	Dynamic Arrays	59
4.4	C-style Character Strings	61
4.4.1	Difference between '\n' and <code>std::endl</code>	62
4.5	C++ Strings: <code>std::string</code>	63
4.6	QA	64
4.7	Quiz	65
5	Working with Expressions, Statements, and Operators	67
5.1	Statements	67
5.2	Compound Statements or Blocks	68
5.3	Using Operators	69
5.3.1	Assignment Operator (=)	69
5.3.2	L-values and R-values	69
5.3.3	Operators to Add (+), Subtract(-), Multiply(* or <<), Divide(/ or >>), and Modulo(%)	70
5.3.4	Operators to Increment(++) and Decrement(-)	71
5.3.5	Equality Operators (==) and (!=)	72
5.3.6	Relational Operators	74
5.3.7	Logical Operations NOT, AND, OR, and XOR	75
5.3.8	Bitwise Operators: NOT (~), AND (&), OR (), XOR (^)	76
5.3.9	Bitwise right shift >> and left shift <<	77
5.3.10	Compound Assignment Operators	79
5.3.11	Using sizeof Operator	80
5.3.12	Operator Precedence	81
5.4	QA	82
5.5	Quiz	83
6	Controlling Program Flow	85
7	Organizing Code with Functions	86
8	Pointers and References	88
8.1	What is a Pointer?	88
8.1.1	Declaring a Pointer	89
8.1.2	Determining the Address of a Variable Using the Reference Operator (&)	90
8.1.3	Access Pointer Data Using the Dereference Operator (*)	91
8.1.4	What is sizeof() a Pointer?	92
8.2	Dynamic Memory Allocation	93
8.2.1	Using new and delete or delete[]	93
8.2.2	Using malloc, realloc, etc and free	94
8.2.3	Operators vs Functions vs constexpr	98
8.2.4	Operator Overloading Examples (Unary, Binary, and Ternary-like)	101
8.2.5	Incrementing (++) and Decrementing (— on Pointers)	104
8.2.6	Using the const keyword on Pointers	106
8.2.7	Passing by ref C vs C++	108
8.2.8	Clarification for pointer declarations	109
8.2.9	Passing Pointers to Functions	112
8.2.10	Pointers in Static vs Dynamic Memory	113
8.3	Common Programming Mistakes when using Pointers	115
8.4	References	116
8.4.1	References as Function Parameters	117
8.4.2	References vs Pointers as Function Arguments	119

8.4.3	Using const on References	121
8.4.4	Extended: Using const with References	122
8.4.5	Definition of Reference in C++ and why it's not valid in C	123
8.4.6	QA	124
8.4.7	Quiz	126
9	Classes and Objects	128
9.1	Classes	128
9.1.1	An Object as an Instance of a Class	128
9.1.2	Accessing Members Using the Dot Operator (.)	129
9.1.3	Accessing Members Using the Pointer Operator (->)	130
9.1.4	Class vs Struct	132
9.1.5	Different Ways to Declare a struct	133
9.1.6	Public, Private, Protected, Friend in C++	135
9.1.7	This Keyword	137
9.1.8	Static Variables in Classes	138
9.2	Constructors	139
9.2.1	Declaring and Implementing a Constructor	139
9.2.2	When and How to Use Constructors	140
9.2.3	Overloading Constructors	140
9.2.4	Class Without a Default Constructor	140
9.2.5	Constructor Parameters with Default Values	141
9.2.6	Constructors with Initialization Lists	142
9.3	Destructor	144
9.3.1	Declaring and Implementing a Destructor	144
9.3.2	When and How to Use a Destructor	145
9.4	Copy Constructors	146
9.4.1	Shallow Copying and Associated Problems	146
9.4.2	Ensuring Deep Copy Using a Copy Constructor	148
9.4.3	Explicit keyword	152
9.4.4	Move Constructors Help Improve Performance	153
9.5	Different Uses of Constructors and Destructors	156
9.5.1	Class That Does Not Permit Copying	156
9.5.2	Singleton Class That Permits a Single Instance	157
9.6	Class That Prohibits Instantiation on the Stack	158
9.6.1	Using Constructors to Convert Types	159
9.7	sizeof() a Class	162
9.8	How struct differs from class	166
9.9	Declaring a 'friend' of a class	168
9.10	union: A Special Data Storage Mechanism	169
9.11	Using Aggregate Initialization on Classes and Structs	170
9.12	constexpr with Classes and Objects	172
9.13	QA	173
9.14	Quiz	174
10	Implementing Inheritance	176
10.1	Basics of Inheritance	176
10.1.1	Access specific keyword protected	178
10.1.2	Base Class Initiation — Passing Parameters to the Base Class	179
10.1.3	Derived Class Overriding Base Class's Methods	180
10.1.4	Invoking Overridden Methods of a Base Class	181
10.1.5	Derived Class Hiding Base Class's Methods	182
10.2	Order of Construction	183
10.3	Order of Destruction	184
10.4	Private Inheritance	186
10.5	Composition (side note)	188
10.6	Protected Inheritance	189
10.7	Multiple Inheritance	191

10.8	Avoid Inheritance via <code>final</code>	193
11	Polymorphism	196
11.1	Basics of Polymorphism	196
11.1.1	Need for Polymorphic Behavior	196
11.1.2	Polymorphic Behavior Implemented Using Virtual Functions	197
11.1.3	Need For Virtual Destructors	198
11.1.4	How do Virtual Functions Work? — The Virtual Function Table (vtable)	200
11.2	Abstract Base Classes and Pure Virtual Functions	203
11.3	Using 'virtual' Inheritance to Solve the Diamond Problem	204
11.4	Specifier 'override' to Indicate Intention to Override	207
11.5	Using 'final' to Prevent Function Overriding	209
11.6	Virtual Copy Constructors?	210
11.7	QA	211
11.8	Quiz	212
12	Operator Types and Operator Overloading	214
12.1	What are Operators in C++?	214
12.2	Unary Operators	216
12.2.1	Types of Unary Operators	217
12.2.2	Programming a Unary Increment/Decrement Operator	218
12.2.3	Programming Conversion Operators	219
12.2.4	Programming Dereference Operator (*) and Member Selection Operator (— >)	221
12.3	Binary Operators	222
12.3.1	Type of Binary Operators	223
12.3.2	Programming Binary Addition (a + b) and Subtraction (a - b) Operators	224
12.3.3	Implementing Addition Assignment (+=) and Subtraction Assignment (-=) Operators	225
12.3.4	Overloading Equality (==) and Inequality (!=) Operators	227
12.3.5	Overloading <, >, <=, and >= Operators	228
12.3.6	Overloading Copy Assignment Operator (=)	229
12.3.7	Subscript Operator ([])	231
12.4	Function Operator ()	234
12.5	Move Constructor and Move Assignment Operator for High Performance Programming	235
12.5.1	The Problem of Unwanted Copy Steps	235
12.5.2	Declaring a Move Constructor and Move Assignment Operator	238
12.6	User Defined Literals	244
12.7	Operators That Cannot Be Overloaded	245
13	Casting Operators	246
13.1	The Need for Casting	246
13.2	Examples of C++ Casts	247
13.3	The C++ Casting Operators	252
13.3.1	Using <code>static_cast</code>	253
13.3.2	Using <code>dynamic_cast</code> and Runtime Type Identification	256
13.3.3	Using <code>reinterpret_cast</code>	257
13.3.4	Using <code>const_cast</code>	258
13.4	Problems with the C++ Casting Operators	259
14	An Introduction to Macros and Templates	262
14.1	The Preprocessor and the Compiler	262
14.2	Using Macro <code>#define</code> to Define Constants	263
14.2.1	Using Macros to Protection against Multiple Inclusion	264
14.3	Using <code>#define</code> to Write Macro Functions	266
14.3.1	Why All the Parentheses?	267
14.3.2	Using Macro 'assert' to Validate Expressions	268
14.3.3	Adv and Disadv of Using Macro Functions	269
14.4	An Introduction to Templates	271
14.4.1	The Different Types of Template Declarations	272

14.4.2	Template Functions	273
14.5	Templates and Type Safety	274
14.5.1	Template Classes	275
14.5.2	Declaring Templates with Mutliple Parameters	277
14.5.3	Declaring Templates with Default Parameters	278
14.5.4	Sample Template 'class<>' HoldsPair	279
14.5.5	Template Instantiation and Specilization	280
14.5.6	Template Classes and 'static' Members	281
14.5.7	Variable Templates, Also Called Variadic Templates	283
14.5.8	Using 'static_assert' to Perform Compile-Time Checks	285
15	Lambda Expressions	288
15.1	What is a Lambda Expression?	288
15.2	How to Define a Lambda Expression	290
15.3	Lambda Expressions for a Unary Function	291
15.4	Lambda Expression for a Unary Predicate	292
15.5	Lambda Expression with State via Capture Lists [...]	293
15.6	The Generic Syntax of Lambda Expressions	294
15.7	Lambda Expression for a Binary Function	296
15.8	Lambda Expression for a Binary Predicate	297
16	Understanding Smart Pointers	299
16.1	What are Smart Pointers?	299
16.1.1	The Problem with Using Conventional (Raw) Pointers	299
16.1.2	How Do Smart Pointers Help?	300
16.2	How Are Smart Pointers Implemented?	301
16.3	Types of Smart Pointers	302
16.3.1	Deep Copy	303
16.3.2	Copy on Write Mechanism	305
16.3.3	Reference-Counted Smart Pointers	306
16.3.4	Reference-Linked Smart Pointers	307
16.3.5	Destructive Copy	308
16.3.6	Using std::unique_ptr	309
16.4	Smart pointers std::	311
16.4.1	std::unique_ptr	311
16.4.2	std::shared_ptr	313
16.4.3	std::weak_ptr	315

1 Getting Started

1.1 History of C++

- Initially developed by Bjarne Stroustrup at Bell Labs in 1979.
- It was meant to be a successor to C.
- C++ was designed to be an **object-oriented** language with concepts like
 - inheritance
 - abstraction
 - polymorphism
 - encapsulation
- In 1998, the first standard of C++ was ratified by the ISO Committee.
(This is when we get C++14 for example for the year it is ratified (2014) again and so on)
- C++ can be for operating systems, web services, databases, and much more.

1.2 Programming a C++ Application

When you start Notepad on Windows or a terminal on Linux, you are running an `executable` of that program. The **executale** is the finished product that can be run.

1.2.1 Steps to Generating an Executable

1. **Writing** (or programming) C++ code using a text editor/IDE.
2. **Compiling** code using a C++ `compiler` that converts it to a **machine language** version contained in "`object files`"
(gcc (gnu compiler collection) or clang (llvm) or msvc (microsoft visual c++), etc)

gcc example: `g++ main.cpp -o main`
clang example: `clang++ main.cpp -o main`
msvc example: `cl main.cpp`
3. **Linking** the output of the compiler using a `linker` to get an `executable` (.exe for Windows, for example)

```
main.cpp
|
v
Compiler (g++, clang++, MSVC)
|
v
main.o (Linux/macOS) or main.obj (Windows)
|
v
Linker
|
v
program.exe (Windows)
program      (Linux/macOS)
```

Compilation is the step where code in C++ (typically text files with .cpp extension) is converted into `byte code` that the processor can **execute**.

The `Compiler` converts one code file at a time, generating an **object file** with a `.o` (linux) or `.obj` (Windows) extension, **ignoring dependencies** that this cpp file may have on code in another file.

The `Linker` joins the dots and resolves these **dependencies**.

In the event of successful **linkage**, it creates an **executable** for the programmer to execute and distribute.

This whole process is called *building an exectuable*.

Recall `.c` refers to C files and not C++ files.

Stick to vim little boy.

1.2.2 Programming Your First C++ Application

Note all code will be in ../code dir.

Listing 1.1 Hello.cpp, the Hello World Program

Using linux, you can create the executable via

```
g++ -o hello Hello.cpp
```

This command instructs g++ to create an **executable** named **hello**

clang equivalent command:

```
clang++ -o hello Hello.cpp
```

Both g++ and clang++ support many of the same options or **flags**, for example:

```
g++ -std=c++20 -Wall -Wextra -O2 -o hello Hello.cpp
```

```
clang++ -std=c++20 -Wall -Wextra -O2 -o hello Hello.cpp
```

- **-std=c++20**: Compiles the program using the C++20 language standard. Other common standards include **c++11**, **c++14**, **c++17**, and **c++23**.
- **-Wall**: Enables a broad set of commonly useful compiler warnings. While it does not enable *all* warnings, it catches many common programming mistakes.
- **-Wextra**: Enables additional warnings beyond those provided by **-Wall**. Using both flags together is considered good practice during development.
- **-O2**: Enables a moderate level of compiler optimizations to improve the performance of the generated executable while keeping compilation times reasonable. Other optimization levels include:
 - **-O0**: No optimization (default for debugging).
 - **-O1**: Basic optimizations.
 - **-O2**: Recommended general-purpose optimization.
 - **-O3**: More aggressive optimizations that may increase executable size.
 - **-Ofast**: Enables aggressive optimizations that may sacrifice strict standards compliance.
- **-g**: Includes debugging information in the executable so that a debugger (such as **gdb** or **lldb**) can display source code lines, variables, stack frames, and other useful debugging information. This flag does *not* make the program slower by itself; it simply embeds additional symbolic information.
- **-o hello**: Specifies the name of the output executable. Without this flag, the default executable name is typically **a.out** on Linux/macOS or **a.exe** on Windows (depending on the compiler).

Some more feature of compilers include **Preprocessor Definitions**

The **-D** compiler flag defines a preprocessor macro at compile time. For example,

```
g++ -DDEBUG -o hello Hello.cpp
```

is equivalent to having

```
#define DEBUG
```

before preprocessing begins. This allows for *conditional compilation*:


```
#ifdef DEBUG
    std::cout << "Debug mode enabled\n";
#endif
```

The code inside the `#ifdef DEBUG` block is only compiled when the `DEBUG` macro is defined. This technique is commonly used to enable debugging, logging, assertions, or other development-only features.

Remember to actually execute the executable by calling it like `./hello` on Linux or `hello.exe` on Windows.

A thing to remember is through the evolution of C++, the same program can be compiled using different compilers on different operating systems.

1.3 QA

- **Q: Can I ignore warning messages from my compiler?**

A: These warnings show to the programmer a better/safer way to go about things.

It is up to them to decide to ignore or not. Generally, it's best to address them for production level code.

- **Q: How does an interpreted language differ from a compiled language?**

A: Languages such as Python or Javascript are **interpreted** (for the most part). There is **no compilation step**.

An **interpreted language** use an **interpreter** that directly reads the script text file (code) and performs the desired actions.

Consequently, you need to have the interpreter installed on a machine where the script needs to be executed; consequently, **performance usually takes a hit** as the interpreter works as a **runtime translator** between the microprocessor and the code written.

- **Q: What are runtime errors, and how are they different from compile-time errors?**

A: Errors that happen when you execute your application are called **runtime errors**.

Compile-time errors do not reach the end user and are typically caused by **syntax** (e.g., missing semicolons, unmatched braces) or **semantic** (e.g., undeclared variables, type mismatches) errors. They prevent the compiler from successfully generating an executable.

1.4 Quiz

1. What is the difference between an interpreter and compiler?

A: An **interpreter** is a tool that interprets what you code (or an **intermediate byte code**) and performs certain actions.

A **compiler** takes your code as an input and generates an **object file**. In the case of C++, after **compiling** and **linking** have have an **executable** that can run directly by the processor without need for any further interpretation.

2. What does the linker do?

A: A compiler takes a C++ file as input and generates an **object file** in machine language.

Often your code has **dependencies** on libraries and functions in other code files. Creating these **links** and generating an executable that integrates all dependencies is the job of the **linker**.

3. What are the stages in the normal development cycle?

A: Code. Compile to create object file. Link to create executable. Execute. Debug. Fix any errors. Repeat.

In many/most cases compiling and linking are one step.

1.5 Exercies

1. E1
2. E3

2 The Anatomy of a C++ Program

C++ programs are comprised into `classes` comprising `member functions` and `member variables/fields`.

2.1 Parts of the Hello World Program

Analyze ts:

Listing 2.1 HellowWorldAnalysis.cpp: Analyze a Simple C++ Program.

The C++ program can be classified into two parts:

1. the `preprocessor directives` that start with a `#` and
2. the `main body` of the program that starts with `int main()`.

2.2 Preprocessor Directive `#include`

The `preprocessor` is a tool that runs before the actual **compilation** starts.

Preprocessor directives are commands to the **preprocessor** and always start with a `#` symbol.

```
#include <iostream>
#define PI 3.14159
#ifdef DEBUG
#endif
#ifndef HEADER_H
#endif
```

In our code Listing 2.1, the `#include <filename>` tells the preprocessor to take the contents of the file (`iostream` in our example) and include them at the line where the directive is made.

`iostream` is a *standard header file* that enables the usage of `std::cout` to display "Hello World" on the screen.

The compiler was able to compile the code that contains `std::cout` b/c we instructed the preprocessor to include the definition of `std::cout`

Note: We can include our own header files using `#include "relative/path/to/FileB.h"`. Use **double quotes** (`"`) for project headers and **angle brackets** (`<>`) for standard library headers (e.g., `<iostream>`, `<vector>`).

The underlying rule is:

`#include "..."` → searches the current/project directories first, then system include paths.

`#include <...>` → searches only the system include paths.

The syntax (`"`) vs (`<>`) does not define "own vs standard".

The include search path (set with `-I`) is what actually lets the compiler find your own headers in different folders.

2.3 The Body of Your Program `main()`

Following the **preprocessor directive(s)** is the *body* of the program characterized by the function **`main()`**.

The execution of a C++ program always starts here. It is a standardized convention that **`main()`** is of type **`int`**, which stands for **integer**.

`int main()` can take many forms in C/C++:

```
int main() {  
    return 0;  
}
```

Standard C++ entry point with no command-line arguments (also valid in C, though `int main(void)` is preferred).

```
int main()  
{  
    return 0;  
}
```

Same as above; only the brace style differs.

```
int main(void)  
{  
    return 0;  
}
```

Standard C entry point explicitly specifying that no arguments are accepted.

```
int main(int argc, char *argv[])  
{  
    return 0;  
}
```

Standard entry point that receives command-line arguments.

```
int main(int argc, char **argv)  
{  
    return 0;  
}
```

Equivalent to the previous form since array parameters decay to pointers.

```
int main(int argc, char *argv[], char *envp[])  
{  
    return 0;  
}
```

Implementation-defined form that also receives environment variables.

```
int wmain(int argc, wchar_t *argv[])  
{  
    return 0;  
}
```

Microsoft-specific Unicode entry point using wide-character command-line arguments.

```
void main()  
{  
    /* Non-standard */  
}
```

Accepted by some compilers but not part of the C or C++ standards.

2.4 Returning a Value

Functions must have a **return type**. If a function's return type is **void**, it does not return a value, so you do not need to write a **return** statement (although you may write **return;** to exit the function early).

main() is a function of type **int**, so it would always return an **int** or integer.

main() having a return type of type **int** can be useful for OSes to determine the state of which the program had exited. For example, in many cases one application is launched by another and the *parent application* (that launches it) wants to know if the *child application* (that was launched) has completed its task successfully.

The programmer can use the **return value** of **main()** to convey a success or error state to the **parent application**.

Conventionally, **main()** returns:

- **0** for **success**
- **-1** for **error**

However, the return type is an **integer** and you can still have flexibility to convey many different states of success or failure using the available range of integer return values.

Side note: C/C++ is case-sensitive, so **Int** and **int** aren't the same. **Int** would have to be a class/struct/typedef/etc and wouldn't be the same as the standard **int** type.

2.5 Namespaces

The reason why you used `std::cout` and not just `cout` is that the **artifact** (`cout`) is apart of the **standard (std) namespace**.

So, what are **namespaces**?

First, assume that you didn't use the **namespace qualifier** in invoking `cout` and assume `cout` existed in **two locations** known to the **compiler** — which one should the compiler invoke?

This trivially causes a conflict and the **compilation fails**.

Namespaces to the rescue. Namespaces are **names** given to parts of code that reduce the potential for **naming conflicts**.

By invoking `std::cout`, you are telling the compiler to use that one unique `cout` that is available in the **std namespace**.

To use the namespace you simply do:

```
using namespace <namespace_name>;

// like

using namespace std;
```

To create a **namespace**, you do:

```
namespace MyNamespace
{
    int x = 10;

    void hello()
    {
        std::cout << "Hello!\n";
    }
}
```

You access members using the scope resolution operator `::`:

```
int main()
{
    std::cout << MyNamespace::x << '\n';
    MyNamespace::hello();
}
```

Instead of writing `<namespace_name>::whatever` every time, you can write:

```
using namespace MyNamespace;
```

to make all names in `MyNamespace` directly accessible.

Alternatively, if you only want to bring a single name into scope, use:

```
using MyNamespace::whatever;
```

std stands for the **standard** namespace and you invoke functions, etc there that have been **ratified** by the ISO Standards Committee.

Listing 2.2 The using namespace Declaration

Listing 2.3 Another Demo of the *using* keyword

However, with all this said:

Avoid using namespace ...; in global scope, especially in headers and large programs. this limits damage/errors to a smaller scope.

- The difference btw `using namespace std` and `using std::cout` or `std::endl` is that the **former** allows **all artifacts** in the **std namespace** (`cout`, `cin`, etc) to be used without explicit inclusion of the namespace.
- With the **latter**, the namespace would explicitly be restricted to only `std::cout` and `std::endl`.

2.6 Functions in C++

- We've already written a function before: **int main()**.
- Functions have a **return type** like **int** for **int main()**.
- Other return types include:
 - **void** — the function returns no value.
 - **float** — returns a single-precision floating-point number.
 - **double** — returns a double-precision floating-point number.
 - **char** — returns a single character.
 - **bool** — returns a boolean value (true/false).
 - **std::string** — returns a string object (C++).
 - **int*** / **double*** / **etc.** — returns a pointer to a value.
 - **custom types (struct/class)** — returns user-defined data types.
- We also have **lambda functions**, which are anonymous functions (they do not have an explicit name).
- **Listing 2.4, Declaring, Defining, and Calling a Function**

- **Function declarations** are, for example:

```
int DemoConsoleOutput();
```

This is a function declaration without a body (i.e. no definition). It informs the compiler that a function with this name, return type, and parameter list exists somewhere in the program.

This allows the function to be called before its actual definition appears in the file. The **compiler** assumes the **definition** will be provided later (possibly in the same file or in another translation unit).

- **Function definitions** provide the actual implementation of the function. For example:

```
int DemoConsoleOutput()
{
    std::cout << "Hello\n";
    return 0;
}
```

A function definition includes the function body, where the actual logic is written and executed when the function is called.

Function definitions allocate the actual code that runs at runtime, whereas declarations only describe the function's interface.

Using separate declarations and definitions helps with:

- better code organization (especially in large projects),
- separation of interface (.h files) and implementation (.cpp files),
- allowing functions to be used across multiple files.

- **Listing 2.5** Using the Return Value of a Function
- Functions
 - can take **parameters** (values defined in the function signature that receive input from the caller as **arguments**),
 - can be **recursive** (a function that calls itself and requires a base case to stop),
 - can contain multiple return statements (though if not inside a condition, the compiler may issue a warning),
 - can be **overloaded** (multiple functions with the same name but different parameter lists),
 - can be **overwritten** (more correctly: overridden in inheritance, where a derived class replaces a base class function),
 - can be **expanded in-line** by the compiler (the compiler replaces the function call with the function body to reduce call overhead),
 - and much more.
- This is discussed in **Lesson 7**.

2.7 Basic Input Using `std::cin` and Output using `std::cout`

- We use the **console** to write and read information.
- use **`std::cout`** (standard "see-out") to **write** text to the console.

`std::cout` is accompanied by `<<`

```
std::cout << "Hello World" << std::endl;
```

- use **`std::cin`** (standard "see-in") to **read** text from the console.

`std::cin` is accompanied by `>>`

```
std::cin >> Variable;
```

```
std::cin >> Variable1 >> Variable2;
```

- | |
|-------------------------------------|
| Listing 2.6 Use cin and cout |
|-------------------------------------|

2.8 QA

- **Q: What does `#include` do?**

A: This is a **directive** to the **preprocessor** when you call your compiler. This specific directive causes the contents of the file named in `<>` after `#include` to be **inserted** at that line as if it were typed at that location in your source code.

- **Q: When do you need to program command-line arguments?**

A: To supply **options** that allow the user to alter the behavior of a program.

For example, `ls` in Linux or `dir` in Windows enables you to see the contents within the current directory or folder. To view files in another directory, you specify the path of the same using command-line arguments, as in `ls /` or `dir \`.

2.9 Quiz

- **Q:** What is the problem in declaring `Int main()`?

A: `int main()` is what it must be.

- **Q:** Can comments be longer than one line?

A: Yes via `/* */`

2.10 Exercises

1. e1.cpp

3 Using Variables, Declaring Constants

- **Variables** are points in memory that store data in a finite amount of time.
- **Constants** are points in memory that define **artifacts** there are not allowed to **change**.

3.1 What Is a Variable?

3.1.1 Memory and Addressing

- All computers contain a **microprocessor** and a certain amount of **memory** for **temporary storage** called **Random Access Memory (RAM)**.
- In addition, many devices also allow for data to be **persisted** on a **storage device** such as the **hard disk**.
- The **microprocessor** executes your application, and in doing so works with the **RAM** to fetch the application **binary code** to be executed as well as the **data** associated with it.
- **RAM** can be thought as a **storage** akin to a row of lockers in a hallway, each locker having a **number** — that is, an **address**.
- To access a location in **memory**, say location 578, the processor needs to be asked via an **instruction** to **read** a value from there or **write** a value to it.
- There exist **virtual addresses**, which allow each process to have its **own address space** independent of **physical memory**. These **virtual addresses** are translated into **physical addresses** by the operating system and hardware.
- The main purpose of **virtual addresses** is to provide **memory isolation**, **memory protection**, and the illusion that each process has its own large, contiguous block of memory, even though **physical memory is shared** among many processes.

3.1.2 Declaring Variables to Access and Use Memory

- For variables, it's best to initialize it no matter if you use it, so that you know what kind of value it holds.
- If you don't, you wouldn't know the value, which may cause problems ahead.
- | |
|--------------------|
| Listing 3.1 |
|--------------------|
- The compiler does its job of mapping the variable to a location **in memory** and takes care of the associated **memory-address** bookkeeping for you.
- The programmer works with the human-friendly names, while the **compiler** manages **memory-addressing** and creates the instructions for the microprocessor to execute in working with the **RAM**.

3.1.3 Declaring and Initializing Multiple Variables of a Type

- You can condense the declaration of variables of the same type to one line of code:

```
int firstNum = 0, secondNum = 0, thirdNum = 0; // and so on
```

- Note: data stored in variables is data stored in RAM. It is not persistent and is lost upon application termination. To persist it, as of today, you must write it to a file on disk.

3.1.4 Scope of a Variable

- When outside the `scope` of a variable, the **variable names** will not be recognized by the compiler and your program won't compile.
- Beyond its scope a variable is an **unidentified entity** that the compiler knows nothing of.
- `Listing 3.2 Scope`

3.1.5 Global Variables

- Not best practice for production projects in source files.
- All functions within the source file can access global variables declared in that file (provided they are in scope). Functions in other source files cannot access them unless the variables have external linkage (e.g., are declared with `extern` in a header and defined in exactly one source file).

Example:

```
// globals.h
extern int g_value;

// globals.cpp
int g_value = 42;
```

- | |
|-------------------------------------|
| Listing 3.3 Global variables |
|-------------------------------------|

3.2 Common Compiler-Supported C++ Variable Types

- `bool` – stores `true` or `false`.
- `char` – typically 1 byte, range: -128 to 127 (or 0 to 255 if unsigned).
- `signed char` – 1 byte, range: -128 to 127 .
- `unsigned char` – 1 byte, range: 0 to 255 .
- `short` (or `short int`) – typically 2 bytes, range: $-32,768$ to $32,767$.
- `unsigned short` – typically 2 bytes, range: 0 to $65,535$.
- `int` – typically 4 bytes, range: $-2,147,483,648$ to $2,147,483,647$.
- `unsigned int` – typically 4 bytes, range: 0 to $4,294,967,295$.
- `long` – typically 8 bytes on Linux/macOS and 4 bytes on Windows.
- `unsigned long` – unsigned version of `long`.
- `long long` – typically 8 bytes, range: -9.22×10^{18} to 9.22×10^{18} .
- `unsigned long long` – typically 8 bytes, range: 0 to 1.84×10^{19} .
- `float` – typically 4 bytes, approximately $\pm 3.4 \times 10^{38}$ (about 7 decimal digits of precision).
- `double` – typically 8 bytes, approximately $\pm 1.7 \times 10^{308}$ (about 15–16 decimal digits of precision).
- `long double` – extended precision floating-point type (size and precision are implementation-defined).
- `void` – represents the absence of a value (e.g., functions that return nothing).

Fixed-Width Integer Types (<stdint>)

- `int8_t` – exactly 8 bits, range: -128 to 127 .
- `uint8_t` – exactly 8 bits, range: 0 to 255 .
- `int16_t` – exactly 16 bits, range: $-32,768$ to $32,767$.
- `uint16_t` – exactly 16 bits, range: 0 to $65,535$.
- `int32_t` – exactly 32 bits, range: $-2,147,483,648$ to $2,147,483,647$.
- `uint32_t` – exactly 32 bits, range: 0 to $4,294,967,295$.
- `int64_t` – exactly 64 bits, range: $-9,223,372,036,854,775,808$ to $9,223,372,036,854,775,807$.
- `uint64_t` – exactly 64 bits, range: 0 to $18,446,744,073,709,551,615$.

3.2.1 Signed vs Unsigned integers

- **Sign** implies **positive** or **negative**.
- All numbers you work with using a computer are stored in the **memory** in the form of **bits** and **bytes**.
- A **memory location** that is **1 byte** long contains **8 bits**.
- Each **bit** can either be **0** or **1**.
- Thus, a **memory location** that is **1 byte** long can contain a maximum of **2** to the power of **8 values** — that is **256** unique values.
- Similarly, a **memory location** that is **16 bits** large (or **2 bytes**) can contain **2** to the power of **16 values** — that is, **65, 536** unique values.
- Think of it as just

$$2^{\text{number of bits}} = \text{quantity of how many numbers we can represent}$$

- If the values are **unsigned** (i.e., only non-negative), then an **n -bit** integer can represent values from **0 to $2^n - 1$** .
- For example:
 - **1 byte** (8 bits): 0 to $2^8 - 1 = 255$.
 - **2 bytes** (16 bits): 0 to $2^{16} - 1 = 65,535$.
- For an 8-bit unsigned integer, the place values are:

$$2^7 \quad 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0$$

If every bit is set to 1,

$$11111111_2 = 2^7 + 2^6 + \cdots + 2^1 + 2^0 = 2^8 - 1 = 255.$$

- To represent **negative numbers**, signed integers are typically stored using **two's complement**. Rather than sacrificing the leftmost bit, it is assigned a weight of -2^{n-1} , while all remaining bits retain their usual positive weights.
- For an **n -bit signed integer**, the representable range is

$$-2^{n-1} \text{ to } 2^{n-1} - 1.$$

- Thus, an **8-bit signed integer** has the range

$$-2^7 \text{ to } 2^7 - 1 = -128 \text{ to } 127.$$

- Since there are $2^8 = 256$ unique bit patterns, exactly half represent negative values and half represent non-negative values.

- The place values for an 8-bit signed integer are

$$-2^7 \quad 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0.$$

Therefore,

$$10000000 = -2^7 = -128,$$

and

$$11111111 = -128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = -1.$$

- Recognize the smallest/minimum is when only the **MSB** is 1 and the rest are 0, which gives our msb weight of -2^{n-1} .

3.2.2 Signed integer types: `short`, `int`, `long`, and `long long`

- These are the fundamental **signed integer types** in C++, used to store whole numbers (both positive and negative).
- They primarily differ in the amount of memory they occupy and, consequently, the range of values they can represent.
- In general, `short` stores smaller integers than `int`, while `long` and `long long` can store progressively larger values.
- The exact size of each type is **implementation-defined** (depends on the compiler and platform), although modern systems typically use:
 - `short`: 2 bytes
 - `int`: 4 bytes
 - `long`: 4 bytes (Windows) or 8 bytes (Linux/macOS)
 - `long long`: 8 bytes
- When an exact integer size is required (e.g., exactly 32 or 64 bits), prefer the fixed-width integer types from `<cstdint>` such as `int32_t` and `int64_t`.

3.2.3 Unsigned integer types: unsigned short, unsigned int, unsigned long, unsigned long long

- These are the fundamental **unsigned integer types** in C++, used to store only **non-negative** whole numbers.
- Since no bits are reserved for representing negative values, unsigned integers can represent approximately twice the positive range of their signed counterparts of the same size.
- They have the same typical sizes as their signed versions:
 - `unsigned short`: 2 bytes
 - `unsigned int`: 4 bytes
 - `unsigned long`: 4 bytes (Windows) or 8 bytes (Linux/macOS)
 - `unsigned long long`: 8 bytes

- An unsigned *n*-**bit** integer has the range

$$0 \text{ to } 2^n - 1.$$

- Unsigned integers are useful when negative values are impossible (e.g., sizes, counts, and bit manipulation), but care should be taken when mixing signed and unsigned types, as implicit conversions can produce unexpected results.

3.2.4 Integer Overflow

- **Integer overflow** occurs when the result of an arithmetic operation is outside the range that can be represented by the integer's type.
- For example, an 8-bit signed integer can only store values from -128 to 127 . Attempting to compute

$$127 + 1$$

produces a value that cannot be represented.

- For **unsigned integers**, overflow is **well-defined** in C++. Arithmetic wraps around modulo 2^n , where n is the number of bits. For example, an 8-bit unsigned integer satisfies

$$255 + 1 = 0.$$

- For **signed integers**, overflow results in **undefined behavior**. This means the C++ standard does not specify what should happen—the program may appear to wrap around, produce an unexpected value, or behave unpredictably.
- To avoid overflow, choose an integer type with a sufficiently large range (e.g., `int64_t` instead of `int32_t`) or check whether an operation would exceed the type's limits before performing it.

- **Listing 3.4** Signed and Unsigned overflow

- The output shows the wrap around effect of incrementing and overflowing for both types.

3.2.5 Floating-Point types float and double

- **Floating-point types** are used to store **real numbers** (numbers with a fractional or decimal component), such as 3.14 or -0.001.
- The two most commonly used floating-point types are:
 - **float** – typically occupies **4 bytes** and provides about **7 decimal digits** of precision.
 - **double** – typically occupies **8 bytes** and provides about **15–16 decimal digits** of precision.
- Floating-point numbers are generally represented according to the **IEEE 754** standard, which stores a number using three components:
 - a **sign bit**,
 - an **exponent**,
 - and a **significand (mantissa)**.
- Since only a finite number of bits are available, many decimal values (such as 0.1) cannot be represented exactly. As a result, floating-point arithmetic may introduce small rounding errors.
- Floating-point types should not be used when exact decimal precision is required (e.g., financial calculations). Instead, use an appropriate decimal or fixed-point representation.
- In general, prefer **double** over **float** unless reduced memory usage or compatibility with specific hardware is important.

- Under the hood, **float** and **double** are stored using the **IEEE 754 floating-point format**. A number is represented in **scientific notation in base 2**:

$$(-1)^{\text{sign}} \times 1.\text{fraction} \times 2^{\text{exponent}}$$

- A **float** (32 bits) is split into:
 - 1 bit for the **sign**
 - 8 bits for the **exponent**
 - 23 bits for the **fraction (mantissa)**
- A **double** (64 bits) is split into:
 - 1 bit for the **sign**
 - 11 bits for the **exponent**
 - 52 bits for the **fraction (mantissa)**
- For example, the value 5.75 is stored as:

$$5.75_{10} = 101.11_2 = 1.0111 \times 2^2$$

So the stored components are:

- sign = 0 (positive)
- exponent = 2 (stored with a bias)
- fraction = 0111 (remaining bits after the leading 1)

For the above example: 5.75

- Whole part: 5

$$5 = 101_2$$

- Fractional part: 0.75

$$- 0.75 \times 2 = 1.5 \rightarrow 1$$

$$- 0.5 \times 2 = 1.0 \rightarrow 1$$

$$0.75 = 0.11_2$$

- Combine whole and fraction:

$$5.75 = 101.11_2$$

- Normalize (move binary point left 2 places):

$$101.11_2 = 1.0111 \times 2^2$$

- **Why we multiply by 2 repeatedly for the 2nd step?**

We are converting a decimal fraction (base 10) into a binary fraction (base 2).

In binary, each fractional bit represents a negative power of two:

$$\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \dots$$

We therefore need a method to determine which of these powers of two are present in the number.

Multiplying by 2 repeatedly allows us to extract these bits one at a time by shifting the fraction:

- the integer part after multiplication gives the next binary digit,
- the remaining fractional part is used for the next step.

Converting back to base 10:

We start from the binary result:

$$5.75 = 101.11_2$$

Split into integer and fractional parts:

$$101.11_2 = 101_2 + 0.11_2$$

Convert each part separately:

- Integer part:

$$101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 4 + 0 + 1 = 5$$

- Fractional part:

$$0.11_2 = 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = \frac{1}{2} + \frac{1}{4} = 0.75$$

Therefore:

$$101.11_2 = 5.75_{10}$$

3.3 Size of a Variable using sizeof

- **Size** is the amount of **memory** that the compiler reserves when a variable is declared.
- The size of a variable depends on its **type**.
- We can see the size via `sizeof(<type>)` and this outputs the **size of type in bytes**.
- **Listing 3.5** Finding the Size of Standard C++ Variable Types
- Note these sizes are **dependent** on the 32 or 64-bit **compiler** (not necessarily an OS).

3.4 Automatic Type Inference Using `auto`

- Compilers supporting C++11 and beyond give the option of **not** having to **explicitly** specify the variable type when using the keyword `auto`

```
auto coinFlippedHeads = true;
```

- We leave the task of defining an exact type for variable `coinFlippedHeads` to the **compiler**.
- The **compiler** checks the nature of the value the variable is initialized to and then decides the best possible type that suits this variable.
- In the above example, the compiler determines `bool` as the type that suits variable `coinFlippedHeads` and **internally** treats `coinFlippedHeads` as a `bool`.

- **Listing 3.6** Using `auto`

- When you **don't initialize** a variable of type `auto`, you get a **compile error** like:

```
error: declaration of 'auto willGetError' has no initializer
9 |         auto willGetError;
```

3.5 typedef and/or using to Substitute a Variable's Type

- You can substitute a variable's type to something that you might find convenient.
- You would use the keyword `typedef` for that.
- Here is an example where a programmer wants to call an **unsigned int** a descriptive `STRICTLY_POSITIVE_INTEGER`.

```
typedef unsigned int STRICTLY_POSITIVE_INTEGER
STRICTLY_POSITIVE_INTEGER numEggsInBasket = 4532;
```

- Note, an equivalent keyword now in practice in modern C++ (introduced in C++11) is the keyword `using`:

```
// Modern C++ equivalent
using STRICTLY_POSITIVE_INTEGER = unsigned int;
STRICTLY_POSITIVE_INTEGER numEggsInBasket = 4532;
```

- When compiled, the first line tells the compiler that a `STRICTLY_POSITIVE_INTEGER` is an **unsigned int**. At later stages when the compiler encounters the already defined type `STRICTLY_POSITIVE_INTEGER`, it substitutes it for **unsigned int** and continues compilation.
- **typedef** or **type substitution** is useful for when dealing with complex types, for example types that use **templates**.

3.6 What is a Constant?

- C++ enables you to define a variable as **constant** that means it **cannot change (its value)** after **declaration**.
- Assignment to a constant cause compilation errors.
- Constants can be:
 - Literal constants
 - Declared constants using the **const** keyword
 - Constant expressions using the **constexpr** keyword (new since C++11)
 - Enumerated constants using the **enum** keyword
 - Defined constants that are not recommended and deprecated.

3.6.1 Literal constants

- **Literal constants** can be of many types — **integer, string, and so on**.
- In our Hello World program, here "Hello World" is a *string literal constant* (or just **string constant**) meaning they cannot be modified.

String literals are **immutable** in C++. Attempting to modify a string literal results in **undefined behavior**. For example:

```
const char* str = "Hello";
str[0] = 'J';    // Error: cannot modify a string literal

char* ptr = "Hello";
ptr[0] = 'J';    // Undefined behavior
```

If you need a modifiable sequence of characters, initialize a character array instead:

```
char str[] = "Hello";
str[0] = 'J';    // OK

std::cout << str;    // Prints: Jello
```

Here, the compiler creates a writable character array by copying the contents of the string literal.

Note: for `const char* str = "Hello";` the pointer itself is *not* placed in read-only memory—the **string literal** is.

For example:

```
const char* str = "Hello";
```

In this declaration:

- "Hello" is a *string literal*, and its characters are typically stored in read-only memory.
- **str** is a pointer variable (usually stored on the stack if it is a local variable). It stores the address of the first character of the string literal.

- The `const` qualifier prevents the characters from being modified through `str`.

In contrast,

```
char str[] = "Hello";
```

creates a writable character array by copying the contents of the string literal into the array. Since the array is a separate copy, modifying it is valid:

```
str[0] = 'J';    // OK
```

- For ***int literals***, we have also been using them all along in our programs. An integer literal such as 42 or 0 is simply a fixed value written directly in the source code.

Unlike variables, an integer literal is not a named object in memory that we can modify. Instead, it is used by the compiler as a constant value when evaluating expressions or initializing variables.

For example:

```
int x = 42;
x = x + 1;
```

Here, 42 is an integer literal that is copied into the variable `x`. We are not modifying the literal itself; we are modifying the variable `x` that stores a copy of that value.

- This is why the following declaration makes sense:

```
const char* p = "Hello";
```

The string literal "Hello" corresponds to a real array of characters stored in memory (typically in a read-only section of the program). Therefore, `p` can hold the address of its first character.

In contrast, the following is invalid:

```
int* p = &42;    // invalid
```

This does not work because 42 is an integer literal, not a memory object. It is a value used directly by the compiler, and it does not have a valid, addressable memory location. Therefore, taking its address using `&` is not allowed.

- ***Binary literals*** were introduced in **C++14**. They allow writing numbers in base 2 using the prefix `0b` or `0B`. For example:

```
int x = 0b1010;    // 10 in decimal
```

- ***Octal literals*** have existed since early C++. They use a leading zero 0. For example:

```
int x = 075;    // 61 in decimal
```

Here, 075 is interpreted as an octal (base-8) number rather than a decimal.

- ***User-defined literals*** were introduced in **C++11**. They allow programmers to define custom suffixes for literals, enabling more readable and expressive code.

For example, we can define a literal to represent distances in meters:

```
long double operator"" _m(long double x) {
    return x;
}
```

```
long double distance = 5.0_m;
```

Here, 5.0_m is a user-defined literal, where the suffix `_m` converts the number into a meaningful unit.

User-defined literals can be used with integers, floating-point numbers, characters, and strings, and they are commonly used in libraries to improve readability (e.g., time durations, units, or complex numbers).

3.6.2 Declaring Variables as Constants Using const

- const is meant to be used before the variable type is declared.

```
const type-name constant-name = value;
```

- Listing 3.7 Declaring a Constant Called pi
- Here is the compile-error if you try to modify a const

```
const double pi = 22.0 / 7;  
pi = 345;
```

```
error: assignment of read-only variable 'pi'  
11 |         pi = 345;
```

- It's good programming practice to define variables that are **not supposed to change** as const since it shows the programmer has thought about ensuring the correctness and constantness of data (acts as protection).

3.6.3 Constant Expression Using constexpr

- Keyword `constexpr` allows function-like declarations of **constants**:

```
constexpr double GetPi() {return 22.0 / 7;}
```

- One **constexpr** can use another:

```
constexpr double TwicePi() {return 2 * GetPi();}
```

- **constexpr** make look like a **function**, however, allows for optimization possibilities from the compiler's view. So long as a compiler is capable of evaluating a **constant expression** to a **constant**, it can be used in statements and expressions at places where a constant is expected.
- In the preceding example, `TwicePi()` is a **constexpr** that uses a **constant expression** `GetPi()`. This will possibly trigger a **compile-time optimization** wherein every usage of `TwicePi()` is simply **replaced** by `6.28571` by the compiler, and not the code that would calculate $2 \times 22/7$ when executed.
- We can also combine `constexpr` with `const` when we want to emphasize that a value is both **compile-time computable** and **read-only after initialization**. While **constexpr** already implies constness for the expression, adding `const` can still be useful for clarity in variable declarations. For example:

```
constexpr double GetPi() { return 22.0 / 7; }
```

```
const double pi1 = GetPi();           // runtime or compile-time constant
constexpr double pi2 = GetPi();       // guaranteed compile-time constant
```

Here, `pi2` must be evaluated at compile time, while `pi1` is simply a read-only value initialized from a constant expression. In practice, **constexpr** is preferred when compile-time evaluation is desired, since it already enforces immutability and constant-expression rules.

- We can also place `const` at the end of a member function declaration to indicate that the function does not modify the state of the object. This is different from `const` variables or **constexpr** functions.

For example:

```
class Circle {
public:
    double GetRadius() const {
        return radius;
    }

    void SetRadius(double r) {
        radius = r;
    }

private:
    double radius;
};
```

Here, `GetRadius()` `const` guarantees that no data members of the object will be modified inside the function. If we attempt to modify any member variable inside a `const` function, the compiler will produce an error.

This `const` qualifier at the end of a function is commonly used for **getter functions** to ensure they are safe and do not change object state.

- **Listing 3.8** using `constexpr`

- The two `constexpr`'s may look like functions, but they are not exactly.

Functions are invoked at program **execution time**.

Constant expressions are already substituted to their hard values by the compiler at **compile time**.

- So here in the listing, the `constexpr` saves **time and computation** to the same calculation being contained in a function.
- Note if the block/body/definition of the `constexpr` relies on user input/numerics not known at compile time, then the compiler treats the `constexpr` as a **regular function**, i.e. no compile-time optimization.

3.6.4 Enumerations

- Sometimes you want a **variable** whose values are **restricted** to a certain **set** of values defined by you.
- `Enumerations` are the tool and are characterized by the *keyword* `enum`.
- Example:

```
enum RainbowColor
{
    Violet = 0,
    Indigo,
    Blue,
    Green,
    Yellow,
    Orange,
    Red
};
```

- A cardinal direction example:

```
enum CardinalDirections
{
    North,
    South,
    East,
    West
};
```

- **Enumerations** are **user-defined** types.
- Variables of this **type** can be assigned to a **range of values** restricted to the **enumerators** contained in the **enumeration**.
- So, if defined a variable that contains the colors of a rainbow, you declare the variable like this:

```
RainbowColors MyFavColor = Blue;
```

- This works for a traditional (unscoped) enumeration because the enumerator names (such as `Blue`) are placed directly into the surrounding scope.

If the enumeration were declared as a **enum class** (a scoped enumeration, introduced in C++11), the enumerators would remain inside the scope of the enumeration. In that case, we would need to qualify the name:

```
RainbowColors MyFavColor = RainbowColors::Blue;
```

Scoped enumerations help prevent name collisions by requiring the enumeration name when referring to its values.

- C++ provides two kinds of enumerations: the traditional `enum` and the newer `enum class` (introduced in **C++11**). The main difference is that an **enum class** is **scoped**, meaning its enumerator names remain inside the enumeration and do not pollute the surrounding scope.

For example, with a traditional enumeration:

```
enum RainbowColors { Red, Orange, Yellow, Green, Blue };

RainbowColors color = Blue;    // OK
```

The name `Blue` is placed directly into the surrounding scope, so it can be used without qualification. However, this can lead to name conflicts if another enumeration also contains an enumerator named `Blue`.

With a scoped enumeration:

```
enum class RainbowColors { Red, Orange, Yellow, Green, Blue };
```

```
RainbowColors color = RainbowColors::Blue;    // OK
```

Here, the enumerator must be qualified with the enumeration name (`RainbowColors::Blue`). This avoids name collisions and provides better type safety, making `enum class` the preferred choice in modern C++.

- Generally, the compiler converts the **enumerator** such as *Violet* and so on into integers. Each **enumerated value** specified is **one more than the previous value** if not explicitly initialized. You have the choice of specifying a **starting value**, and if this is not specified, the compiler takes it as **0**.
- You can still specify the explicit value against each of the enumerated constants by initializing them.

```
enum ErrorCode
{
    Success = 0,
    Warning = 100,
    Error = 200
};
```

- | |
|--------------------------------------|
| Listing 3.9 Enumerated Values |
|--------------------------------------|

3.6.5 Defining Constants Using `#define`

- For C++ this kind of defining is not recommended.
- This is legacy support (C defines $\rightarrow$ OK in C but we're in C++).
- We can define pi like before as

```
#define pi 3.14286
```

- `#define` is a *preprocessor macro*.
- What is done here is that all mentions of '`pi`' henceforth are replaced by 3.14286 for the **compiler** to process.
- Note: the compiler neither knows nor cares about the **actual type** of the **constant** in question (it's just a text replacement (read: non-intelligent replacement) done by the **preprocessor**).

3.7 Keywords that Cannot be Used as Variable or Constant Names

The following are examples of **reserved keywords** in C++.

Since these words have special meanings to the compiler, they cannot be used as identifiers (such as variable, constant, or function names).

- | | | |
|-------------|-------------|------------|
| • asm | • false | • signed |
| • auto | • float | • sizeof |
| • bool | • for | • static |
| • break | • friend | • struct |
| • case | • goto | • switch |
| • catch | • if | • template |
| • char | • inline | • this |
| • class | • int | • throw |
| • const | • long | • true |
| • constexpr | • mutable | • try |
| • continue | • namespace | • typedef |
| • default | • new | • typeid |
| • delete | • nullptr | • typename |
| • do | • operator | • union |
| • double | • private | • unsigned |
| • else | • protected | • using |
| • enum | • public | • virtual |
| • explicit | • register | • void |
| • export | • return | • volatile |
| • extern | • short | • while |

3.8 QA

- **Q: Why define constants at all if you can use regular variables instead of them?**

A: **Constants**, especially those declared using the keyword `const`, are your way of telling the **compiler** that the **value** of a particular variable be **fixed** and **not allowed to change**.

It ensures/enforces correctness and constantness and quality of your code, so that another programmer reading/writing your program would be aware that this `const` variable is to not be changed.

- **Q: Why should I initialize a variable?**

A: If you don't, you would not know for sure what the variable **contains** for a starting value.

It would contain junk data at that memory location of the variable, and you cannot check against non-zero values of it cause you wouldn't know that it's not initialized with this check.

- **Q: Why shouldn't I use global variables frequently?**

A: **Global variables** can be **read** and **assigned** globally. The **latter** is the problem as they can be changed globally.

Trivially, other programmers can mistakenly overwrite your global variables and that would effect your code—even in another `.cpp` file.

- **Q: What happens if I decrement from zero contained in an unsigned int?**

A: You see a **wrapping effect**. Decrementing an **unsigned integer** that contains 0 by 1 means that it wraps to the **highest** value it can hold!

3.9 Quiz

- What is the difference btw a **signed** and an **unsigned integer**?

A: A **signed** integer can store both positive and negative values, whereas an **unsigned** integer can store only non-negative values (0 and positive numbers). Since unsigned integers do not need to represent negative values, they can represent a larger maximum positive value.

- Why should you **not use** `#define` to declare constant?

A: Because `#define` performs a simple text substitution and is not type-safe. Modern C++ prefers `const` or `constexpr` because they are checked by the compiler, have a type, and are easier to debug.

- Why would you initialize a variable?

A: Initializing a variable gives it a known value before it is used, helping to prevent bugs caused by uninitialized (garbage) values.

- Consider the enum below. What is the value of `Queen`?

```
enum YourCards { Ace, Jack, Queen, King };
```

A: `Queen` has the value 2. By default, enumeration values begin at 0 and increase by 1.

- What is wrong with this variable name?

```
int Integer = 0;
```

A: Nothing is technically wrong with the name. `Integer` is not a reserved C++ keyword, so it is a valid identifier. However, it is not a descriptive variable name and could be confused with the type `int`, so a more meaningful name is recommended.

4 Managing Arrays and Strings

4.1 What is an Array?

- An `array` is a group of elements forming a **complete** unit. Example: an array of solar panels.
 - is a collection of elements.
 - all elements contained in an array are of the **same kind**.
 - this collection forms a **complete set**.
- In C++, **arrays** enable you to **store** data elements of a type in **memory**, in a **sequential** and ordered fashion (cache friendly).

4.1.1 The Need for Arrays

- Imagine you want to store values for 500 integers.
- The best way to do this is with an array:

```
int arr[500];
```

- There are several common ways to initialize all integers in an array to 0:

1. Brace initialization (recommended)

```
int arr[500] = {0};
```

2. Empty brace initialization (C++11 and later)

```
int arr[500] = {};
```

3. Using `memset`

```
int arr[500];  
memset(arr, 0, sizeof(arr));
```

4. Using `std::fill`

```
int arr[500];  
std::fill(arr, arr + 500, 0);
```

5. Using a range-based for loop

```
int arr[500];  
for (int &x : arr)  
    x = 0;
```

6. Using a traditional for loop

```
int arr[500];  
for (int i = 0; i < 500; i++)  
    arr[i] = 0;
```

7. Partially initialize the first two elements

```
int arr[5] = {10, 20};  
// arr = {10, 20, 0, 0, 0}
```

- **Note:** `memset` should only be used to initialize an integer array to 0. It should *not* be used to initialize an array of integers to arbitrary values (e.g., 1, -1, or 100), since it operates on bytes rather than integer values.

`memset` = sets raw memory bytes

`int` initialization = sets typed values

If you do `memset(arr, -1, sizeof(arr))` — this only “works as expected” in some cases because it produces all bits set:

`0xFF 0xFF 0xFF 0xFF` = -1 (two’s complement)

- Such **arrays** are called *static arrays* because the **number of elements** they contain as well as the **memory** the array consumes is **fixed** at **compile-time**.

4.1.2 How Data Is Stored in an Array

- Indices in C++ start with **0**.
- Memory of the elements in the array are equally allocated.
- The amount of **memory** reserved by the compiler for the array of 5 ints is:

`sizeof(int) * 5`

- In general, the amount of memory reserved by the compiler for an array in **bytes** is

`Bytes consumed by an array = sizeof(element-type) * Number of Elements`

4.1.3 Accessing Data Stored in an Array

- When you access an element at index N , the compiler does not "search" for it. Instead, it computes the exact memory address using **arithmetic**.
- Arrays in C/C++ are stored in **contiguous memory**, meaning each element is placed directly next to the previous one.
- The compiler treats the **array name** as a pointer to the first element. That is, the array name holds the memory address of index **0**.
- To access element $A[N]$, the compiler:
 - Starts from the base address $A[0]$
 - Adds an offset of $N \times \text{sizeof}(\text{element})$

`address of A[N] = base_address + N * sizeof(element)`

- This is why arrays and pointers are closely related: **A** is essentially the address of **A[0]**, so:

`A[i] == *(A + i)`

- In other words, pointer arithmetic is what makes array indexing possible.
- The C++ compiler does not check if the index is within the actual defined bounds of the array.
- Accessing an array beyond its bounds results in unpredictable behavior and raises security risks to your program like crashing and should be avoided at all costs.
- **Listing 4.1** Arrays

4.1.4 Modifying Data Stored in an Array

- **Listing 4.2** Assigning Values to Elements in Array

4.2 Multidimensional Arrays

- A multidimensional array is an array of arrays to comprise in total an *n-dimensional array*.
- The most common type is a **2D array**, which can be thought of as a table with rows and columns.
- Declaration example:

```
int grid[3][4];
```

This creates a 2D array with 3 rows and 4 columns.

- Initialization example:

```
int grid[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

- Accessing elements uses two indices:

```
grid[row][col]
```

- Example:

```
grid[0][1] = 2;  
grid[1][2] = 6;
```

- Memory layout: Multidimensional arrays are stored in **row-major order**, meaning rows are stored contiguously in memory.
- Example layout:

```
{1, 2, 3, 4, 5, 6}
```

- Pointer relationship:

```
grid[i][j] == (*(grid + i) + j)
```

- Common use cases:

- Matrices in math
- Grids (games, maps)
- Tables of data

- **Listing 4.3** Multidimensional Array

4.3 Dynamic Arrays

- A dynamic array is an array that can change size during runtime, unlike a normal C-style array which has a fixed size.
- In C++, the most common way to use dynamic arrays is through **vectors**.
- A vector is part of the Standard Template Library (STL) and automatically manages memory for you.
- Declaration example:

```
#include <vector>

vector<int> nums;
```

- Adding elements dynamically:

```
nums.push_back(10);
nums.push_back(20);
nums.push_back(30);
```

- Accessing elements (same as arrays):

```
cout << nums[0];    // 10
cout << nums[1];    // 20
```

- Size of the vector:

```
cout << nums.size();
```

- Example full usage:

```
vector<int> nums = {1, 2, 3};

for (int i = 0; i < nums.size(); i++)
{
    cout << nums[i] << " ";
}
```

- Key advantages:
 - Automatically resizes when elements are added
 - No need to manually manage memory
 - Safer than raw pointers and arrays
- Internally, vectors use dynamic memory allocation and may reallocate to a larger block when capacity is exceeded.

Side Notes:

Fixed Arrays using `std::array`

- `std::array` is a container from the C++ Standard Library that represents a fixed-size array.
- It is defined in the header:

```
#include <array>
```

- Unlike `std::vector`, the size of a `std::array` is fixed at compile time.
- Declaration example:

```
std::array<int, 5> nums = {1, 2, 3, 4, 5};
```

- Accessing elements works like normal arrays:

```
cout << nums[0];    // 1
cout << nums[3];    // 4
```

- Bounds-checked access can be done using `at()`:

```
cout << nums.at(0);
```

- Getting the size:

```
cout << nums.size();
```

- Iterating through a `std::array`:

```
for (int i = 0; i < nums.size(); i++)
{
    cout << nums[i] << " ";
}
```

- Range-based loop (modern C++):

```
for (int x : nums)
{
    cout << x << " ";
}
```

- Key advantages over raw arrays:
 - Knows its own size (`.size()`)
 - Safer access with `.at()`
 - Works well with STL algorithms
 - No pointer decay unless explicitly used
- Key difference from `std::vector`:
 - `std::array`: fixed size
 - `std::vector`: dynamic size

4.4 C-style Character Strings

- **C-style strings** are a special case of an array of characters.
- We have seen examples of C-style strings in the form of **string literals**:

```
std::cout << "Hello World";
```

- This is equivalent to using the **array declaration**:

```
char sayHello[] = {'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '\0'};  
std::cout << sayHello << std::endl;
```

- Equivalent way using a string literal (recommended):

```
char sayHello[] = "Hello World";  
std::cout << sayHello << std::endl;
```

- Equivalent using a pointer to a string literal:

```
const char* sayHello = "Hello World";  
std::cout << sayHello << std::endl;
```

- Equivalent using `std::string` (modern C++ strings):

```
#include <string>  
  
std::string sayHello = "Hello World";  
std::cout << sayHello << std::endl;
```

- **Listing 4.6** Risky app of C-style strings

- If you forget the **null terminator** (`\0`) or it gets overwritten, then certain functions that rely on this **null-terminator** will go beyond the intended data and can cause data security issues and potentially a crash.
- All strings must end in `\0`.

- Example: `strlen()` computes the length of a string by walking the character buffer and counts the number of characters crossed until it reaches a **null-terminator** that indicates the **end of the string**.
- This behavior of `strlen()` makes it **dangerous** as it can easily walk past the bounds of the character array if this null-terminator isn't present.
- C programs use these kind of functions (`strlen()` (length of a string), `strcpy()` (copy a string), `strcat()` (concatenate a string), etc) that take in as input strings and can be dangerous as they seek the **null-terminator** and can exceed the boundaries potentially.

Side note:

4.4.1 Difference between `'\n'` and `std::endl`

- Both `'\n'` and `std::endl` are used to move output to a new line, but they are not the same.

`'\n'` (newline character)

- Inserts only a newline character into the output stream.
- Does **not** flush the output buffer.
- More efficient and faster.

```
cout << "Hello\n";
```

`std::endl`

- Inserts a newline character.
- Forces a **flush** of the output buffer.
- Slower because flushing is an expensive operation.

```
cout << "Hello" << endl;
```

Key difference

- `'\n'` only moves to the next line.
- `std::endl` moves to the next line **and flushes output immediately**.

Comparison table

Feature	<code>'\n'</code>	<code>std::endl</code>
New line	Yes	Yes
Flush buffer	No	Yes
Performance	Faster	Slower

When to use

- Use `'\n'` for normal output (recommended most of the time).
- Use `std::endl` when you need immediate output (e.g., debugging).

4.5 C++ Strings: `std::string`

- C++ standard strings are an efficient and **safer** way to deal with text input.
- `std::string` is not static in size like a **char array** implementation of a **C-style string** is.
- `std::string` can **dynamically resize** when more data is added, unlike fixed-size C-style character arrays. It is only slightly slower in theory due to overhead, but in practice and with compiler optimizations, it's minute (small difference in speed).
- Listing 4.7 using `std::string`

4.6 QA

- **Q: Why take the trouble to initialize a static array's elements?**

A: Unless initialized, the array, unlike a variable of any other type, contains **junk** and **unpredictable values** as the **memory** at that location was left untouched after the last operations.

Initializing ensures a predictable initial state.

- **Q: Would you need to initialize the elements in a dynamic array for the same principles?**

A: No, not necessarily. A **dynamic array** in C++ is smart. Elements are typically defaulted to a value of 0 if left uninitialized.

- **Q: Does the length of the string include the null-terminator?**

A: No, it doesn't. The length of the string "Hello World" is 11, including the space character in btw the words and excluding the **null-terminator**.

- **Q: If I want to still use C-style arrays, what should be its size?**

A: The size should be the length of the string + the null-terminator. So, basically account for a + 1 to the size when allocating the array.

"Hello World" should be allocated in an array of size $11 + 1 = 12$.

4.7 Quiz

1. If you need to allow the user to input strings, would you use C-style strings?

A: No. As shown, the input can be greater than your bounds of the statically allocated C-style string.

This will lead to undefined behavior if not checked.

2. How many characters are in `'\0'` as seen by the compiler?

A: One character: the null-terminator.

5 Working with Expressions, Statements, and Operators

At its heart, a program is just a sequence of steps whether it be sequentially or in parallel.

These steps are programmed into expressions, statements, and use operators to perform calculations/actions.

5.1 Statements

- Languages — spoken or programmed — are made up of `statements` that are executed one after another.
- Example:

```
std::cout << "Hello World" << std::endl;
```

- All statements in C++ end in a ;
- Extra spaces between statements **do not** matter (some languages it does matter like Python)
- In the front end of a compiler, the **lexical analyzer (lexer)** typically ignores whitespace that separates tokens. For example, the following declarations are treated identically:

```
char c;  
char    c;  
char  
c;  
char
```

```
c;
```

- The lexer produces the same sequence of tokens for each:

```
KEYWORD(char) IDENTIFIER(c) SEMICOLON(;;)
```

- Whitespace is only significant in places where it separates tokens. Outside of string and character literals, extra spaces, tabs, and newlines generally do not change the meaning of a C++ program.
- Writing whitespaces within a statement can effect certain things like for example string literals:

```
cout << "Hello  
World" << endl; // is invalid
```

- To solve the above we can either do

```
cout << "Hello \  
World" << endl; // split ot two lines is OK
```

- Or this but entails a concatenation by the compiler

```
cout << "Hello "  
"World" << endl; // two string literals is also OK^{*}  
// (compiler does extra work --- catenation)
```

- The above one with \ is best to split up a long statement into multiple lines.

5.2 Compound Statements or Blocks

- When you group **statements** together within **braces** `{ . . . }`, you create a *compound statement* or *block*.

```
{  
    int daysInYear = 365;  
    cout << "Block contains an int and a cout statement" << endl;  
}
```

- A *block* typically groups many statements to indicate that they belong together.
- We've seen it already through functions, `constexpr`'s, arithmetic operations, and so on with conditionals, loops, etc

5.3 Using Operators

- **Operators** are tools for you to be able to work with data, transform it, process it, etc.

5.3.1 Assignment Operator (=)

- We've been using this

```
int daysInYear = 365;
```

- The **assignment operator** replaces the value contained by the **operand** on the left (called **l-value**) by that on the right (called **r-value**).

5.3.2 L-values and R-values

- **L-values** often refer to **locations in memory**.
 - A **variable** such as `daysInYear` from the preceding example is actually a **handle** to a **memory location** and is an **l-value**.
 - **R-values**, on the other hand, can be the very **content** of a **memory location**.
-
- an l-value = a reference/handle to a memory location (something you can assign to, like a variable)
 - an r-value = the value contained in that location (the data itself)

- So, all **l-values** can be used as **r-values**, but not all **r-values** can be used as **l-values**.

Example:

```
daysInYear = 365;    // valid (l-value = r-value)
```

```
365 = daysInYear;    // invalid (r-value cannot be assigned to)
```

- So, strictly speaking: an r-value **cannot** be an l-value.

5.3.3 Operators to Add (+), Subtract(-), Multiply(* or <<), Divide(/ or >>), and Modulo(%)

- These are basic arithmetic operators used to perform mathematical operations on numeric values.
 - **Addition (+)**: Computes the sum of two operands.
 - **Subtraction (-)**: Computes the difference between two operands.
 - **Multiplication (* or <<)**: Multiplies two values; in some contexts, << is used for bitwise left shift, which effectively multiplies by powers of 2.
 - **Division (/ or >>)**: Divides one value by another; >> is a bitwise right shift, often corresponding to integer division by powers of 2.
 - **Modulo (%)**: Returns the remainder after division of two integers.
-
- **Division (/)**: Performs division between two operands. If both operands are integers, the result is also an integer and the fractional part is truncated (i.e., decimal portion is discarded, effectively rounding toward zero). If at least one operand is a `float` or `double`, floating-point division is performed and the decimal part is preserved.

Key clarification (division behavior in C++)

<code>7 / 2 = 3</code>	(integer / integer → truncates toward zero)
<code>7 / 2.0 = 3.5</code>	(integer / double → floating-point division)
<code>7.0 / 2 = 3.5</code>	(double / integer → floating-point division)
<code>7.0 / 2.0 = 3.5</code>	(double / double → floating-point division)
<code>-7 / 2 = -3</code>	(truncates toward zero, NOT -4)
<code>-7 / 2.0 = -3.5</code>	(floating-point division)
<code>-7.0 / 2 = -3.5</code>	(floating-point division)
<code>int x = 7 / 2;</code>	<code>// x = 3 (result truncated before assignment)</code>
<code>double y = 7 / 2;</code>	<code>// y = 3.0 (division happens first, still integer division, IMPORTANT)</code>
<code>double z = 7 / 2.0;</code>	<code>// z = 3.5 (floating-point division)</code>

5.3.4 Operators to Increment(++) and Decrement(--)

- The increment operator (++) increases a variable's value by 1, and the decrement operator (--) decreases it by 1. These operators are shorthand for `x = x + 1` and `x = x - 1`.
- **Prefix form** (++x, --x): The variable is modified first, and the updated value is then used in the expression.
- **Postfix form** (x++, x--): The current value is used first, and then the variable is modified.
- Example:

```
int x = 5;
int a = ++x; // x becomes 6, a = 6
int b = x++; // b = 6, then x becomes 7
```

- **Pointers and increment/decrement:** When used on pointers, ++ and -- move the pointer to the next or previous memory location of its type (not just +1 byte).
- Example:

```
int arr[] = {10, 20, 30};
int* p = arr;

p++; // moves to arr[1]
p--; // moves back to arr[0]
```

- **Iterators in loops:** In C++ iterators behave like pointers, so ++it moves to the next element. This is commonly used in for loops to traverse containers.
- Example:

```
for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
{
    cout << *it << endl;
}
```

- **Prefix** is preferred over **postfix** due to, at least theoretically, with the **postfix operators**, the compiler needs to store the initial value temporarily in the event of it needing to be assigned. The effect on performance in these cases is negligible with respect to integers, but in the case of certain classes there might be a point in this argument.

Smart pointers may optimize away the differences.

5.3.5 Equality Operators (==) and (!=)

- (==) checks if two **operands** are equal.
- (!=) checks if two **operands** are not equal.
- The behavior of relational and equality operators can depend on the **types being compared**. For example, comparing integers vs floating-point numbers may involve implicit type conversion, and comparing pointers checks addresses rather than values.
- Example: integer vs floating-point comparison

```
int a = 5;
double b = 5.0;

cout << (a == b) << endl;  // true (a is promoted to double)
```

- Example: floating-point precision issues

```
double x = 0.1 + 0.2;
double y = 0.3;

cout << (x == y) << endl;  // false (precision error in floating-point math)
```

Why (0.1 + 0.2) != 0.3 in C++

Even though mathematically:
0.1 + 0.2 = 0.3

In floating-point (binary) representation:
0.1 is stored as an approximation
0.2 is stored as an approximation
0.3 is also stored as an approximation

So:

(0.1 + 0.2) produces a slightly different value than 0.3

Example result:

```
x = 0.1 + 0.2  -> 0.3000000000000000444
y = 0.3        -> 0.2999999999999999889
```

Therefore:

(x == y) is false due to tiny precision error

- Example: pointer comparison (compares addresses, not values)

```
int a = 10, b = 10;

int* p1 = &a;
int* p2 = &b;

cout << (p1 == p2) << endl;  // false (different memory locations)
cout << (a == b) << endl;    // true (values are equal)
```

- Example: string comparison (C-style strings compare addresses, not content)


```
const char* s1 = "hello";  
const char* s2 = "hello";  
  
cout << (s1 == s2) << endl; // may be true or false (implementation-dependent)
```

5.3.6 Relational Operators

- Relational operators are used to compare two operands and determine the relationship between them. The result of any relational expression is a **boolean value** (true or false).
- **Less than (<)**: Checks if the left operand is smaller than the right operand.
- **Greater than (>)**: Checks if the left operand is larger than the right operand.
- **Less than or equal to (<=)**: Checks if the left operand is smaller than or equal to the right operand.
- **Greater than or equal to (>=)**: Checks if the left operand is larger than or equal to the right operand.
- Relational operators are commonly used in conditions such as **if** statements and loops.
- Example:

```
int a = 5, b = 10;

cout << (a < b) << endl;    // true
cout << (a > b) << endl;    // false
cout << (a <= 5) << endl;   // true
cout << (b >= 20) << endl;  // false
```

- Relational comparisons can behave differently depending on types. For example, comparing integers is exact, while floating-point comparisons may be affected by precision errors as demonstrated in the previous section.
- In C++, relational and logical expressions evaluate to a **boolean value** (true or false). When printed using **cout**, **false** is displayed as 0 and **true** is displayed as 1.

Any non-zero value is treated as **true** in conditional expressions.

The compiler checks the condition against zero. So, if it's not zero, then the condition is marked true.

5.3.7 Logical Operations NOT, AND, OR, and XOR

- Logical operators are used to combine or modify boolean expressions. In C++, any expression evaluates to `true` (1) or `false` (0) when used in a logical context.

- **NOT (!) Operator:** Reverses the boolean value.

```
Operand: false
Result : !false = true (1)
```

```
Operand: true
Result : !true = false (0)
```

- **AND (&&) Operator:** Returns true only if both operands are true.

```
Operand 1: false   Operand 2: true
Result    : false && true = false (0)
```

```
Operand 1: true     Operand 2: true
Result    : true && true = true (1)
```

- **OR (||) Operator:** Returns true if at least one operand is true.

```
Operand 1: false   Operand 2: true
Result    : false || true = true (1)
```

```
Operand 1: false   Operand 2: false
Result    : false || false = false (0)
```

- **XOR concept (not a built-in logical operator in C++):** Exclusive OR is true only when operands are different. In C++, it is implemented using bitwise XOR `^` for integers.

```
Operand 1: false (0)   Operand 2: true (1)
Result    : 0 ^ 1 = 1 (true)
```

```
Operand 1: true (1)    Operand 2: true (1)
Result    : 1 ^ 1 = 0 (false)
```

- **Key idea:** In C++, `false` is represented as 0 and `true` as 1 when printed, but internally they are boolean values, not integers.
- **Short-circuit behavior:**
 - `&&` stops evaluating if the first operand is false.
 - `||` stops evaluating if the first operand is true.

5.3.8 Bitwise Operators: NOT (~), AND (&), OR (|), XOR (^)

- Bitwise operators manipulate the individual **bits** of integral data types (such as `char`, `int`, and `long`). They operate on the binary representation of values rather than their decimal values.
- **Bitwise NOT (~)**: Flips every bit in the operand (0 becomes 1, and 1 becomes 0).
- Example:

```
unsigned char x = 10;      // 00001010
unsigned char y = ~x;      // 11110101
```

- **Bitwise AND (&)**: Produces a 1 in each bit position only if the corresponding bits of both operands are 1.
- Example:

```
12 & 10

1100
1010
----
1000    // 8
```

- **Bitwise OR (|)**: Produces a 1 in each bit position if either corresponding bit is 1.
- Example:

```
12 | 10

1100
1010
----
1110    // 14
```

- **Bitwise XOR (^)**: Produces a 1 in each bit position only when the corresponding bits are different.
- Example:

```
12 ^ 10

1100
1010
----
0110    // 6
```

- Bitwise operators are commonly used for bit masking, setting or clearing flags, toggling bits, low-level hardware programming, and performance-sensitive code.

5.3.9 Bitwise right shift >> and left shift <<

- Bitwise shift operators move the bits of an integral value to the left or right by a specified number of positions.
- **Left Shift (<<):** Shifts all bits to the left. Each left shift by one position is approximately equivalent to multiplying the value by 2, provided no significant bits are discarded (overflow).

- Example:

```
5 << 1

00000101   (5)
<< 1
-----
00001010   (10)
```

```
5 << 2

00000101   (5)
<< 2
-----
00010100   (20)
```

- **Right Shift (>>):** Shifts all bits to the right. Each right shift by one position is approximately equivalent to integer division by 2, discarding any remainder.

- Example:

```
20 >> 1

00010100   (20)
>> 1
-----
00001010   (10)
```

```
20 >> 2

00010100   (20)
>> 2
-----
00000101   (5)
```

- Bits shifted out of the value are discarded. For a left shift, zeros are shifted into the least significant bits. For unsigned integers, a right shift shifts zeros into the most significant bits.

- Example:

```
1 << 5

00000001
<< 5
-----
00100000   (32)
```

- Bitwise shift operators are commonly used for efficient multiplication or division by powers of two, bit masking, manipulating binary data, and low-level systems programming.

- **Note:** Although left and right shifts can often be thought of as multiplication or division by powers of two, this relationship is only reliable when no overflow occurs (for left shifts) and when shifting unsigned integers.

5.3.10 Compound Assignment Operators

- Compound assignment operators combine an arithmetic or bitwise operation with assignment. They provide a shorthand for updating the value of a variable.
- **Addition Assignment (+=)**: Adds the right operand to the left operand, then stores the result in the left operand.

```
x += 5;    // Equivalent to: x = x + 5;
```

- **Subtraction Assignment (-=)**: Subtracts the right operand from the left operand, then stores the result.

```
x -= 5;    // Equivalent to: x = x - 5;
```

- **Multiplication Assignment (*=)**: Multiplies the left operand by the right operand, then stores the result.

```
x *= 5;    // Equivalent to: x = x * 5;
```

- **Division Assignment (/=)**: Divides the left operand by the right operand, then stores the result.

```
x /= 5;    // Equivalent to: x = x / 5;
```

- **Modulo Assignment (%=)**: Computes the remainder after division and stores the result.

```
x %= 5;    // Equivalent to: x = x % 5;
```

- **Bitwise AND Assignment (&=)**: Performs a bitwise AND with the right operand and stores the result.

```
x &= y;    // Equivalent to: x = x & y;
```

- **Bitwise OR Assignment (|=)**: Performs a bitwise OR with the right operand and stores the result.

```
x |= y;    // Equivalent to: x = x | y;
```

- **Bitwise XOR Assignment (^=)**: Performs a bitwise XOR with the right operand and stores the result.

```
x ^= y;    // Equivalent to: x = x ^ y;
```

- **Left Shift Assignment (<<=)**: Shifts the bits of the left operand to the left and stores the result.

```
x <<= 2;    // Equivalent to: x = x << 2;
```

- **Right Shift Assignment (>>=)**: Shifts the bits of the left operand to the right and stores the result.

```
x >>= 2;    // Equivalent to: x = x >> 2;
```

- Compound assignment operators are commonly used to write shorter, more readable code while avoiding repetition of the left-hand operand.

5.3.11 Using sizeof Operator

- sizeof operator tells you the **amount of memory** in bytes consumed by a particular type or a variable.

```
sizeof(variable);  
    or  
sizeof(type);
```

- sizeof() may look like a function, but it is **not a function**; it's an **operator**.
- Interestingly, this operator **cannot be overloaded**, i.e. **cannot** be defined by the programmer.

5.3.12 Operator Precedence

1. Scope resolution
2. Member selection, subscripting, increment, decrement
3. sizeof, prefix increment and decrement, complement, and, not, unary minus and plus, address-of and dereference, new, new[], delete, delete[], ccasting, sizeof()
4. Member selection for pointer
5. Multiply, divide, modulo
6. Add, subtract
7. Shift (shift left, shift right)
8. Inequality relational
9. Equality, inequality
10. Bitwise AND
11. Bitwise exclusive OR
12. Bitwise OR
13. Logical AND
14. Logical OR
15. Conditional
16. Assignment operators
17. Comma

5.4 QA

- Trivial questions

5.5 Quiz

1. I am writing an app to divide numbers. What's a better suited data type: int or float?

A: I would say float personally, cause it wouldn't truncate the decimals.

6 Controlling Program Flow

- **Switch statements** instead of if conditions
- The `switch` (variable) must match the case `variable_type`:
- If you don't do a `break`; in the cases, you **fall through** to the next below case.

```
#include <iostream>

using namespace std;

int main() {
    int choice;

    cout << "Enter a number (1-3): ";
    cin >> choice;

    switch (choice) {
        case 1:
            cout << "You selected option 1" << endl;
            break;

        case 2:
            cout << "You selected option 2" << endl;
            break;

        case 3:
            cout << "You selected option 3" << endl;
            break;

        default:
            cout << "Invalid option" << endl;
            break;
    }

    return 0;
}
```

7 Organizing Code with Functions

Don't need this personally.

8 Pointers and References

8.1 What is a Pointer?

- A `pointer` is also a **variable** — one that **stores** an address in **memory**.
 - Just the same way as a **variable** of type **int** is used to contain an **integer value**, a `pointer variable` is used to contain a **memory address**.
 - Thus, a **pointer** is a **variable**, and like all variables a **pointer** occupies **space in memory**.
 - The **value** contained in a **pointer** is interpreted as a memory address.
 - Again, a **pointer** is a special variable that *points* to a **location in memory**.
-
- Note: **memory addresses** are in `hexadecimal (base 16)` and start with an 0x

8.1.1 Declaring a Pointer

- Just like a variable, a pointer needs to be declared.
- You normally declare a pointer to point to a specific **type** (ex: `int*`)
- This would mean that the **address** contained in the **pointer** points to a **location in the memory** that holds an integer.
- A `void pointer` (`void*`) is a pointer that can store the address of any data type. It is often used to represent a generic pointer to raw memory. However, since it has no type information, it cannot be dereferenced directly and must first be cast to a specific pointer type.

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;
    double y = 3.14;

    void* ptr;

    ptr = &x;    // void pointer holds address of int
    cout << *(static_cast<int*>(ptr)) << endl;

    ptr = &y;    // now holds address of double
    cout << *(static_cast<double*>(ptr)) << endl;

    return 0;
}
```

- As in the case with most variables, unless you initialize a pointer it will contain a random value.
- To avoid accessing a **random (uninitialized) memory address**, pointers should be initialized to **NULL** (C style) or **nullptr** (C++11 and later). This ensures the pointer is explicitly set to point to nothing until it is assigned a valid address.
- **nullptr** is preferred in modern C++ because it is type-safe, whereas **NULL** is typically just defined as 0 and may cause ambiguity in overloaded function calls.
- Thus, declaring a pointer to an integer would be

```
int *pointsToInt = NULL; // NULL lol
```

8.1.2 Determining the Address of a Variable Using the Reference Operator (&)

- if `varName` is a variable, `&varName` gives the **address in memory** where its value is placed.
- So, if you have declared an integer `int age = 30;`, `&age` would be the **address in memory** where the value 30 is placed.
- | |
|------------------------------|
| Listing 8.1 Addresses |
|------------------------------|

 VERY IMPORTANT

8.1.3 Access Pointer Data Using the Dereference Operator (*)

- The dereference operator (*) is used to access the value stored at the memory address held by a pointer.
- Instead of working with the address itself, dereferencing allows you to work with the **actual data stored at that address**.
- Example:

```
int x = 10;
int *p = &x;

cout << p << endl;    // prints address of x
cout << *p << endl;    // prints value of x (10)
```

- The expression *p means “go to the memory location stored in p and access its value.”
- If the value at that address is changed through the pointer, the original variable is also modified.
- Example:

```
int x = 10;
int *p = &x;

*p = 20;    // modifies x through the pointer

cout << x << endl;    // outputs 20
```

- Dereferencing only works on valid (non-null) pointers. Using * on an uninitialized or null pointer leads to undefined behavior.

- For example, just look at Listing 8.1

- When the *dereference operator (*)* is used, the program uses the **address stored in the pointer** to access the **object located at that memory address**. The amount of memory accessed depends on the **data type of the pointer** (e.g., int, double), as determined by `sizeof(type)` on the given system.
- The **dereference operator** is also called the *indirection operator* (never heard that in my life)
- Basically, the **memory address** stored in a **pointer** is very important and can cause issues if the address is invalid.

8.1.4 What is sizeof() a Pointer?

- The **pointer** is just another **variable** that contains a **memory address**.
- Hence, irrespective of the **type** that is being pointed to, the content of a **pointer** is an **address** — a number.
- The **length of an address**, that is the **number of bytes** required to store it, is a **constant** for a given system.
- The **sizeof()** a **pointer** is hence dependent on the **compiler** and the **operating system** the program has been compiled for and is not dependent on the nature of the **data** being pointed to.
- | |
|--------------------|
| Listing 8.6 |
|--------------------|
- The output of the **sizeof(int*)** would be
 - 4 bytes on a **32-bit compiler** and
 - 8 bytes on a **64-bit compiler**.

8.2 Dynamic Memory Allocation

Dynamic memory allocation allows a program to **request memory at runtime** from the **heap** instead of the **stack**.

This is useful when the **required memory size** is **not known at compile time**.

The `sizeof()` operator is very useful here.

8.2.1 Using `new` and `delete` or `delete[]`

- In C++, dynamic memory is allocated using the **new** operator and released using **delete**.
- For arrays, **new[]** and **delete[]** must be used.
- Example:

```
int *p = new int;    // allocate single integer
*p = 10;

cout << *p << endl;

delete p;            // free memory
```

- Example (arrays):

```
int *arr = new int[5];

for (int i = 0; i < 5; i++)
{
    arr[i] = i * 10;
}

for (int i = 0; i < 5; i++)
{
    cout << arr[i] << " ";
}

delete[] arr;        // free array memory
```

- **Important:** Using **delete** on memory allocated with **new[]** (or vice versa) leads to undefined behavior.

8.2.2 Using malloc, realloc, etc and free

- In C (also available in C++), dynamic memory is managed using **malloc**, **calloc**, **realloc**, and **free** from the `<stdlib.h>` library.

- **malloc**: Allocates a block of memory (uninitialized).

```
int *p = (int*) malloc(5 * sizeof(int));

for (int i = 0; i < 5; i++)
{
    p[i] = i + 1;
}

free(p);
```

- **calloc**: Allocates memory and initializes it to zero.

```
int *p = (int*) calloc(5, sizeof(int));
```

- **realloc**: Resizes previously allocated memory block.

```
p = (int*) realloc(p, 10 * sizeof(int));
```

- **free**: Releases memory allocated by malloc/calloc/realloc.

```
free(p);
```

- **Important difference**: Unlike **new/delete**, these functions do not call constructors or destructors in C++ and should be used carefully in modern C++ code.

Important side note:

- In C-style dynamic memory allocation, functions like `malloc`, `calloc`, and `realloc` return a `void pointer (void*)`, which is a generic pointer type that can hold the address of any data type.
- A `void*` is used because these functions do not know in advance what type of data the allocated memory will store. Instead, they simply return a raw memory address from the heap.
- Example:

```
int *p = (int*) malloc(5 * sizeof(int));
```

- The cast `(int*)` is required in C++ because a `void*` cannot be automatically converted into a typed pointer without an explicit conversion.
- **Why malloc returns void*:**
 - It allows a single allocation function to be **generic** and work with any data type.
 - It represents raw, untyped memory returned from the heap.
 - The programmer decides how to interpret that memory by casting it to the appropriate pointer type.

- Example showing interpretation:

```
void* ptr = malloc(4 * sizeof(int));

int* iptr = (int*) ptr;
iptr[0] = 10;
```

- **Important:** Unlike `new`, these functions do not perform type construction or initialization—they only allocate raw memory.

- In C++, it is recommended to use `static_cast` instead of C-style casts when converting a `void*` to a typed pointer.

- Example:

```
void* ptr = malloc(4 * sizeof(int));

int* iptr = static_cast<int*>(ptr);

// or really just

int* cppiptr = new int;

iptr[0] = 10;
```

- The expression `(int*) ptr` is a C-style cast, while `static_cast<int*>(ptr)` is the safer and more explicit C++-style cast.
- **Note:** The cast is required in C++ because a `void*` cannot be implicitly converted to another pointer type.

Additional important side notes:

- The **new operator** in C++ can also be **overloaded** for custom memory management in user-defined classes. This allows a class to define how memory is allocated when **new** is used.
- Example:

```
class A {
public:
    void* operator new(size_t size);
};

void* A::operator new(size_t size)
{
    cout << "Custom new called" << endl;
    return malloc(size);
}
```

- In this example, the class **A** defines its own version of the **new** operator. The parameter **size_t size** represents the number of bytes required to allocate an object of type **A**.
- **Note:** Although the operator returns **void*** internally in the declaration above, in practice it is typically expected to return a pointer of type **A*** after allocation.

- Listing 8.7 new and delete with operator *
- **Operator *delete*** cannot be invoked on any address containing a pointer, rather only those that have been returned by the **operator *new*** and **only** those that have not already been released by a *delete*.
- Listing 8.8

Side note:

8.2.3 Operators vs Functions vs constexpr

- In C++, computation can be expressed using **operators**, **functions**, and **constexpr expressions**. Each serves a different purpose in how code is written and evaluated.
- **Operators**: Special symbols that perform built-in operations on operands. They are concise and have fixed precedence and associativity rules.

```
int x = 5 + 3;    // + is an operator
```

- **Functions**: Named blocks of reusable code that perform a task. They are called explicitly and can contain complex logic.

```
int add(int a, int b)
{
    return a + b;
}
```

```
int x = add(5, 3);
```

- **constexpr**: A specifier that tells the compiler that a value or function can be evaluated at **compile time** if given constant inputs. This improves performance and enables compile-time computation.
- Example of constexpr variable:

```
constexpr int x = 5 + 3;    // evaluated at compile time
```

- Example of constexpr function:

```
constexpr int square(int x)
{
    return x * x;
}
```

```
int y = square(4);    // may be computed at compile time
```

- **Key differences**:
 - Operators: built-in symbolic operations with fixed rules
 - Functions: runtime or compile-time logic (depending on usage)
 - constexpr: guarantees compile-time evaluation when possible
- **Summary**:
 - Operators → concise symbolic computation
 - Functions → reusable procedural logic
 - constexpr → compile-time computable values/functions

- Operators and functions are handled differently by the compiler. Function calls use an explicit argument list in parentheses, while operators are part of the C++ language grammar and are parsed as expressions rather than function calls.

- For example, a function call is parsed as:

`add(5, 3);`

- In contrast, an operator expression is parsed using syntax rules defined by the language:

`5 + 3`

- This difference comes from the compiler's parser: function calls follow a "function-call grammar rule", while operators follow "expression grammar rules" with precedence and associativity.
- This is why operators such as `+`, `-`, and `new` do not use function-style parentheses for arguments, even though they conceptually operate on operands.
- In summary, functions are explicitly invoked using argument lists, while operators are built into the language syntax and are interpreted as part of expressions during parsing.

- When the compiler parses an expression, it first performs lexical and syntactic analysis. If it encounters an **identifier**, it determines whether it refers to a variable, function, or other named entity. If it encounters an **operator**, it applies the language grammar rules for expressions, using operator precedence and associativity to determine how operands are grouped and evaluated.

- For example, in a function call like:

`add(5, 3);`

the compiler treats `add` as an identifier referring to a function and processes the parentheses as a function-call argument list.

- In contrast, in an expression like:

`5 + 3 * 2`

the compiler recognizes `+` and `*` as operators and constructs an expression tree based on precedence rules, evaluating multiplication before addition.

- When the compiler encounters a custom operator, it does not introduce new syntax rules. The expression is first parsed using the same grammar rules as built-in operators. For example, an expression like `a + b` is always parsed as a binary operator expression.
- During semantic analysis, the compiler then performs **operator overload resolution**. It checks whether a valid function named `operator+` exists for the given operand types, either as a member function or a non-member function.
- For example, the expression:

`a + b`

may be resolved internally as:

`a.operator+(b)`

- If a matching overload is found, the compiler replaces the operator expression with the corresponding function call. If no valid overload exists, a compilation error occurs.

- In summary, custom operators are not new language constructs; they are existing operators that are bound to user-defined functions during the overload resolution phase of compilation.

- In C++, operators have a fixed number of operands (called **arity**) determined by the language grammar. Operators cannot accept an arbitrary number of arguments like functions.

- **Unary operators** operate on one operand:

```
!a  
++a  
-a
```

- **Binary operators** operate on two operands:

```
a + b  
a < b  
a == b
```

- **Ternary operator** is the only operator with three operands:

```
condition ? expr1 : expr2
```

- The compiler does not allow changing the arity of an operator. For example, `a + b + c` is not a three-operand addition operator, but is instead parsed as:

```
(a + b) + c
```

- During compilation, the expression is first parsed using fixed grammar rules (unary, binary, or ternary forms), and then overload resolution determines whether a matching **operator+**, **operator!**, etc., function exists for the given operand types.
- In summary, operator argument count is fixed by the language, and the compiler handles operators by first parsing expression structure and then binding them to appropriate overloaded functions if available.

8.2.4 Operator Overloading Examples (Unary, Binary, and Ternary-like)

- Unary operator example (prefix ++):

```
#include <iostream>
using namespace std;

class A {
    int x;

public:
    A(int val) : x(val) {}

    // Unary prefix ++
    A operator++()
    {
        ++x;
        return *this;
    }

    void show()
    {
        cout << x << endl;
    }
};

int main()
{
    A obj(5);
    ++obj;
    obj.show();    // 6
}
```

- Binary operator example (+):

```
#include <iostream>
using namespace std;

class A {
    int x;

public:
    A(int val) : x(val) {}

    // Binary +
    A operator+(const A& other)
    {
        return A(x + other.x);
    }

    void show()
    {
        cout << x << endl;
    }
};

int main()
{
    A a(5), b(10);
```

```

    A c = a + b;

    c.show();    // 15
}

```

- **Ternary operator (?:) note:**

- The ternary operator `?:` cannot be overloaded in C++.
- It is built into the language grammar and cannot be redefined.

- **Ternary-like behavior using a function:**

```

#include <iostream>
using namespace std;

class A {
    int x;

public:
    A(int val) : x(val) {}

    void show()
    {
        cout << x << endl;
    }
};

A choose(bool cond, A a, A b)
{
    return cond ? a : b;
}

int main()
{
    A a(5), b(10);

    A c = choose(true, a, b);
    c.show();    // 5
}

```

- In an expression like `a + b`, the operator `+` is not a standalone operation. If `operator+` is defined as a member function inside class `A`, the expression is interpreted as a function call on the left-hand object.

- For example:

```
A c = a + b;
```

- The compiler rewrites this as:

```
A c = a.operator+(b);
```

- This means the operator "belongs" to the left operand (`a`), and the right operand (`b`) is passed as an argument.

- In general:

– $a + b \rightarrow a.operator+(b)$

– `b + a` $\rightarrow$ `b.operator+(a)`

- This is why operator order matters when using member operator overloading. If symmetry is required, a non-member (friend) operator overload is often used instead.
- **To create an overloaded operator function, you write *operator* followed by the operator symbol.**

8.2.5 Incrementing (++) and Decrementing (– on Pointers)

- A **pointer** contains a **memory address**, ex: 0x002EFB34 — the **address** where, say, an integer is placed.
- The **integer** is **4 bytes** long on our system, so it **occupies 4 places in memory** from:
 - 0x002EFB34 to 0x002EFB37 inclusive
- **Incrementing** the **pointer** using the **operator** (++) would **not** result in the **pointer** pointing to 0x002EFB35 (notice the 4 to the 5), for pointing to the middle of the integer would be pointless.
- An **increment** or **decrement** operation on a **pointer** is interpreted by the **compiler** as your need to point to the **next value** in the **block of memory**, assuming it would be of the same **type**, and **not** to the **next byte** (unless the value type is **1 byte** large, like a **char** for instance).

Example:

Suppose we have the following code:

```
int arr[] = {10, 20, 30, 40};
int *p = arr;
```

Assume an **int** occupies **4 bytes** in memory. The array could be stored as follows:

Address	Value
1000	10
1004	20
1008	30
1012	40

Initially, **p** points to address 1000, so:

```
*p    // 10
```

After incrementing the pointer:

```
p++;
```

the compiler knows that **p** is an **int ***, so it advances the pointer by **sizeof(int)** (4 bytes), not by 1 byte. The pointer now points to address 1004:

Address	Value
1000	10
1004 ← p	20
1008	30
1012	40

Thus,

```
*p    // 20
```


Incrementing the pointer again moves it to address 1008, where:

```
*p    // 30
```

In general, for a pointer of type **T ***, the operation **p++** advances the pointer by **sizeof(T)** bytes, making it point to the **next object** of type **T** in memory rather than simply the next byte.

- So, **incrementing a pointer** such as

```
int age = 30;
int* pointsToInt = &age;
```

results it being **incremented** by **4 bytes**, which is the **sizeof(int)**.

- | |
|---|
| It increments based on the <u>size of the type</u> it points to, <u>not</u> the size of the pointer itself. |
|---|
- Using **++** on this **pointer** is telling the compiler that you want it to point to the **next consecutive integer**.
Hence, after incrementing, the pointer would then point to 0x002EFB38 (from 4 to 8 in hex)
- Similarly, adding 2 to this pointer would result in it moving 2 **integers** ahead, that is **8 bytes** (in this case).
- **Decrementing** pointers using **--** gives the same effect but you reduce by the **sizeof** what the pointer is pointing to.

8.2.6 Using the const keyword on Pointers

- Lesson 3 we learned that declaring a **variable** as a **const** ensures that value of the variable is **fixed** as the initialization value for the **life** of the variable.
- The value of a **const-variable** cannot be changed, and therefore it cannot be used as an **l-value**.

- **Pointers** are **variable** too, and hence the **const** keyword can be used here.
- **However**, **pointers** are a **special kind of variable** as they contain a **memory address** and are used to modify **memory** at that address.
- Thus, when it comes to **pointers** and **constants**, you have the following combinations:

1.

The **address** contained in the **pointer** is constant and cannot be changed, yet the **data** at that address can be changed.

```
int daysInMonth = 30;
int* const pDaysInMonth = &daysInMonth;
*pDaysInMonth = 31; // OK! Data pointed to can be changed
int daysInLunarMonth = 28;
pDaysInMonth = &daysInLunarMonth; // NOT OK! Cannot change address!
```

Note here also, the **const** keyword is after the int*!!!

```
const int *p;
~~~~~
data is const (the int being pointed to is const)

int *const p;
~~~~~
pointer is const (the pointer variable p cannot be changed)
```

Also

Valid:

```
const int x = 5;
int const x = 5;
```

Invalid:

```
int x const = 5;    // not allowed in C/C++
```

```
const int x -> x is a const int
int const x -> same thing (just different order)
int x const -> not legal grammar in C/C++
```

2.

Data pointed to is **constant** and **cannot be changed**, yet the **address** contained in the pointer **can be changed** — that is, the **pointer** can also point elsewhere:

```

int hoursInDay = 24;
const int* pointsToInt = &hoursInDay; // OK! address can be changed but not the data.
int monthsInYear = 12;
pointsToInt = &monthsInYear; // OK again for same reason
*pointsToInt = 13; // NOT OK! Cannot change data being pointed to

int* newPointer = pointsToInt; // NOT OK! Cannot assign const to non-const
// cause this would entail you being able to change data.
// const int *p means:
// "I promise I will not modify the int through this pointer"
// int* newPointer = pointsToInt; violates that contract.

```

3.

Both the **address** contained in the pointer and the **value** being pointed to are **constant** and cannot be changed (most restrictive variant):

```

int hoursInDay = 24;
const int* const pHoursInDay = &hoursInDay;

*pHoursInDay = 25; // NOT OK! Cannot change data being pointed to.

int daysInMonth = 30;
pHoursInDay = &daysInMonth; // NOT OK! Cannot change address.

```

- These different forms of **const** is particularly useful when passing **pointers** to **functions**.
- Function parameters need to be declared to support the highest possible (restrictive) level of **constness**, to ensure that the function does **not** modify the pointed value when it is not supposed to.

8.2.7 Passing by ref C vs C++

```
// C++ only (reference)
void foo(int& v) {
    v = 5;
}

int main() {
    int v = 0;
    foo(v);
}

// C/C++ compatible (pointer to int)
void foo(int* v) {
    *v = 5;
}

int main() {
    int v = 0;
    foo(&v);
}
```

8.2.8 Clarification for pointer declarations

The confusing part is that the `*` is **part of the declaration**, not part of the type that is being made `const`.

Step 1: A Regular Variable

A normal integer declaration is:

```
int x;
```

If we make the variable constant:

```
const int x;  
// or equivalently  
int const x;
```

the `int` itself becomes **constant**.

Step 2: Introduce a Pointer

Consider the declaration:

```
int *p;
```

This is read as:

“`p` is a pointer to an `int`.”

Notice that there are now **two different things** (or both to be the most restrictive!) that could be made constant:

1. the **integer** being pointed to,
2. the **pointer** itself.

Case 1: Make the Integer Constant

Starting from:

```
int *p;
```

we can make the integer constant:

```
const int *p;
```

The `const` applies to the `int`, not to the pointer.

You can think of it conceptually as:

```
(const int) *p
```

This means:

- the integer cannot be modified through `p`,
- the pointer itself may still point somewhere else.

Conceptually:

```
p  
|  
v  
+-----+  
| 42 |    <- const int  
+-----+
```

Therefore,

```
*p = 10;          // NOT OK  
p = &otherInt;    // OK
```

Case 2: Make the Pointer Constant

Again start with:

```
int *p;
```

This time make the pointer constant:

```
int *const p;
```

Here, the `const` appears *after* the `*`, so it applies to the pointer itself.
Conceptually:

```
(int *) const p
```

This means:

- the pointer cannot be changed,
- the integer being pointed to may still be modified.

Conceptually:

```
const p
|
v
+-----+
| 42   |
+-----+
```

Therefore,

```
*p = 10;          // OK
p = &otherInt;    // NOT OK
```

Why Doesn't `const int *` Mean “Constant Pointer”?

This is because, in C/C++, the `*` belongs to the **declarator** (the variable being declared), not to the base type.

For example,

```
int *p, q;
```

does **not** declare two pointers.

Instead, it declares:

```
int *p;    // pointer
int q;     // ordinary int
```

If the `*` were part of the type, then both `p` and `q` would be pointers, but they are not.
Therefore, the compiler interprets

```
const int *p;
```

as

```
(const int) *p
```

rather than

```
const (int *)
```

Summary

There are two separate objects involved in a pointer declaration:

1. the object being pointed to,
2. the pointer itself.

The position of `const` determines which one is constant:

Declaration	Meaning
<code>int *p</code>	pointer to an int
<code>const int *p</code>	pointer to a constant int
<code>int *const p</code>	constant pointer to an int
<code>const int *const p</code>	constant pointer to a constant int

A useful way to read declarations is:

- `const int *p` → “p is a pointer to a **constant int**.”
- `int *const p` → “p is a **constant pointer** to an int.”

Once you recognize that there are **two different things** that can be made constant (the pointee and the pointer), the placement of `const` becomes much more intuitive.

- **Add const (always allowed):**

```
int *p
const int *p      (OK: add const to data)
int *const p      (OK: add const to pointer)
const int *const p (OK: add const to both)
```

- **Remove const (NOT allowed implicitly):**

```
const int *p -> int *p      (NOT OK)
int *const p -> int *p      (NOT OK)
const int x  -> int x       (NOT OK)
```

- **Safe conversions (const-preserving):**

```
int *p      -> const int *p  (OK)
int x       -> const int x   (OK)
int *p      -> int *const p  (OK only when initializing)
```

- **Pointer rule intuition:**

You can always promise "I won't modify",
but you cannot take back that promise implicitly.

8.2.9 Passing Pointers to Functions

- Pointers are an effective way to **pass memory space** that contains relevant data for functions to work on.
- The **memory space** shared can also return the result of an **operation**.
- Given the previous section, you should modify pointers as parameters to be `const` or not if you want to restrict the callee modifying the pointer or the value pointed to or both.
- | |
|---|
| Listing 8.10 Use <code>const</code> in pointer function parameters |
|---|

8.2.10 Pointers in Static vs Dynamic Memory

- **Pointers are not tied only to dynamic memory.** Pointers can store the address of variables in **stack (static)** memory or **heap (dynamic)** memory.
- For example, static arrays are just a single pointer underneath and we go to each element via adding the `sizeof(element_type)` to the current pointer and increment or decrement the address by that size in bytes (cause `sizeof` returns bytes)

- **Static (stack) memory example:**

```
int x = 10;
int *p = &x;
```

- x is stored in stack memory
- p simply stores its address
- no dynamic allocation is involved

- **Array example (contiguous static memory):**

```
int arr[] = {1, 2, 3};
int *p = arr;
```

- arrays decay into pointers in expressions
- pointer arithmetic allows traversal of elements
- `p++` moves to the next array element, not next byte

- **Dynamic (heap) memory example:**

```
int *p = malloc(sizeof(int));
*p = 42;
```

// or

```
int *p = new int;
*p = 42;
```

- memory is allocated on the heap
- pointer is required to access and manage that memory
- must be manually freed in C using `free(p)` or `delete ptr` or `delete[] ptr` in C++

- **Function parameter example (pass by address):**

```
void foo(int *p) {
    *p = 100;
}
```

```
int main() {
    int x = 5;
    foo(&x);
}
```

- allows modification of original variable
- avoids copying large data structures

- **Key insight:** Pointers are a **general mechanism for storing memory addresses**, not a feature exclusive to dynamic memory.
- **Exam takeaway:**

- stack variables can have pointers
- heap variables require pointers
- arrays behave like pointers in most expressions
- pointer arithmetic works based on data type size, not byte-by-byte movement

8.3 Common Programming Mistakes when using Pointers

- **Uninitialized pointers (wild pointers)**

```
int *p;  
*p = 10;    // undefined behavior
```

- Pointer is not initialized to valid memory.
- Must always assign a valid address before dereferencing.

- **Dangling pointers (use-after-free)**

```
int *p = malloc(sizeof(int));  
free(p);  
*p = 5;    // undefined behavior
```

- Memory has been freed but pointer still holds the old address.
- Accessing it leads to undefined behavior.

- **Memory leaks**

```
int *p = malloc(sizeof(int));  
p = NULL;    // lost reference to allocated memory
```

- Original allocated memory is no longer reachable.
- This causes memory that cannot be freed (leak).

- **Null pointer dereference**

```
int *p = NULL;  
*p = 10;    // undefined behavior
```

- NULL does not point to valid memory.
- Dereferencing NULL leads to crashes or undefined behavior.

- **Incorrect pointer types**

```
int x = 10;  
float *p = &x;    // type mismatch
```

- Pointer type must match the type of the object.
- Otherwise memory is interpreted incorrectly.

- **Confusing pointer assignment vs dereference**

```
int x = 5;  
int *p = &x;  
  
p = 10;    // incorrect: changes pointer, not value  
*p = 10;   // correct: changes value at address
```

- p stores an address.
- *p accesses the value at that address.

- **Out-of-bounds pointer access**

```
int arr[3] = {1, 2, 3};  
int *p = arr;  
  
p[5] = 10;    // undefined behavior
```

- Pointer arithmetic must stay within allocated memory.
- Accessing outside bounds is undefined behavior.

8.4 References

- A `reference` is an **alias** for an existing variable.
- Once a reference is created, it **must be initialized immediately**.
- A reference does not store a separate copy of the data; it refers to the **same memory location** as the original variable.
- Any operation on the reference is performed on the **original variable**.

- References are declared using the **reference operator (&)**:

```
VarType original = value;  
VarType& ref = original;
```

- After binding, the reference becomes an alternative name for the same variable:

```
ref = 10;    // changes original as well
```

- **Important properties of references:**

- Must be initialized at declaration
- Cannot be null (unlike pointers)
- Cannot be reseated to refer to another variable
- Acts as an automatic dereference (no need for *)

- **Example:**

```
int x = 5;  
int& r = x;  
  
r = 10;    // x is now 10
```

- **Key intuition:** A reference behaves like the original variable, just under a different name.

8.4.1 References as Function Parameters

- References are commonly used as **function parameters** in C++ to allow functions to modify the original argument.
- When a parameter is declared as a reference, the function receives an **alias** to the original variable, not a copy.

- **Pass-by-value vs Pass-by-reference:**

```
// Pass-by-value (copy)
void foo(int x) {
    x = 10;    // does NOT affect original
}
```

```
// Pass-by-reference (no copy)
void bar(int& x) {
    x = 10;    // modifies original
}
```

- In pass-by-value:
 - A copy of the argument is created
 - Changes inside the function do not affect the original variable
- In pass-by-reference:
 - No copy is made
 - The parameter becomes an alias for the original variable
 - Changes inside the function affect the caller's variable

- **Example:**

```
void increment(int& x) {
    x++;
}

int main() {
    int a = 5;
    increment(a);
    // a is now 6
}
```

- **Why use references in parameters?**
 - Avoid copying large objects (performance improvement)
 - Allow functions to modify input arguments
 - Cleaner syntax compared to pointers (no * or &)

- **Important rule:**

- A reference parameter behaves exactly like the original variable inside the function.
- It is just another name for the same memory location.

8.4.2 References vs Pointers as Function Arguments

- In C++, there are two common ways to allow a function to modify an argument:
 - **References** (C++)
 - **Pointers** (C/C++)

- **Pointer-based arguments (C/C++ compatible):**

```
void foo(int* p) {  
    *p = 10;  
}
```

```
int main() {  
    int x = 5;  
    foo(&x);  
}
```

- The function receives the **address** of the variable.
- You must explicitly pass **&x** (address-of operator).
- Inside the function, you must dereference using ***p**.

- **Reference-based arguments (C++ only):**

```
void foo(int& x) {  
    x = 10;  
}
```

```
int main() {  
    int x = 5;  
    foo(x);  
}
```

- The function receives an **alias** to the original variable.
- No need to use **&** or ***** at the call site.
- The syntax is cleaner and behaves like the original variable.

- **Key difference in calling style:**

```
Pointer version:    foo(&x);  
Reference version:  foo(x);
```

- **Conceptual difference:**

- Pointer: function receives an **address** and manually dereferences it.
- Reference: function receives an **alias** automatically (implicit dereference).

- **Important note:**

- You do NOT write `foo(&arg)` when the function expects a reference.
- `&` is only used when the function expects a pointer.

- Listing 8.17
- Listing 8.18
- Listing 8.19

8.4.3 Using `const` on References

- You might need to have **references** that are not allowed to change the **value** of the original variable being **aliased**.
- Using **`const`** when declaring such references is the way to achieve this:

```
int original = 30;
const int& constRef = original; // think of (const belongs to const (int) &, so can't change re
constRef = 40; // NOT OK! constRef can't change value in original
int& ref2 = constRef; // NOT OK! ref2 is not const
const int& constRef2 = constRef; // OK
```

8.4.4 Extended: Using const with References

- A **const reference** prevents modification of the value being referenced.

```
int original = 30;
const int& ref = original;

ref = 40;    // NOT OK: cannot modify through const reference
```

- A const reference still refers to the **same object**, but only allows read-only access.
- You can bind const references to const or non-const variables:

```
int x = 10;
const int& r1 = x;    // OK
const int& r2 = r1;   // OK
```

- You cannot modify the original through a const reference:

```
r1 = 20;    // NOT OK
```

- **Important rule:**

- References in C++ are already fixed (cannot be reseated)
- **const** only controls whether the value can be modified
- **int& const** is meaningless in practice

```
int& -> fixed alias (cannot change what it refers to)
const int& -> fixed alias + cannot modify value
int& const -> conceptually meaningless / not used
```

8.4.5 Definition of Reference in C++ and why it's not valid in C

In C++, a *reference* is a language-level feature that creates an alias for an existing variable. Unlike pointers, references are not separate objects that store memory addresses; instead, they directly refer to the original variable.

- A reference must be initialized when it is declared.
- A reference cannot be re-seated to refer to another variable after initialization.
- A reference behaves as an alias of the original variable.
- C does not support this language feature; it only provides pointers.

In C++, references are declared using the `&` symbol:

```
int a = 10;
int& ref = a;    // ref is a reference to a
```

This allows `ref` to be used exactly like `a`:

```
ref = 20;    // modifies a directly
```

In contrast, C does not have references as a built-in type system feature. Instead, it uses pointers to achieve similar indirect access to variables:

```
int a = 10;
int *p = &a;    // pointer to a
```

However, pointers differ fundamentally from C++ references:

- Pointers can be reassigned to point to different variables.
- Pointers require explicit dereferencing using `*`.
- Pointers store memory addresses explicitly, whereas references are aliases.

For example:

```
int a = 10;
int b = 20;

int* p = &a;
p = &b;    // valid: pointer reassignment

// In C++, a reference cannot be "re-pointed":
int& r = a;
// r = b;    // this assigns the value of b to a, not rebinding r
```

Thus, the key difference is that C++ provides true reference semantics as part of the language, while C only provides pointer-based indirect access, which is more flexible but less abstract.

8.4.6 QA

- **Q: Why dynamically allocated when you can do with static arrays where you don't need to worry about deallocation?**

A: Static arrays have **fixed size** and will neither scale upward if your application needs more **memory** nor will they optimize your application needs less. Dynamic memory fixes that.

- **Q: I have 2 pointers:**

```
int* pointToAnInt = new int;
int* pCopy = pointToAnInt;
```

Am I not better off calling 'delete' using both to ensure memory is gone?

A: No. You are allowed to invoke **delete** only once on the **address** returned by **new**.

Also, you would ideally **avoid** having two pointers pointing to the **same address** b/c performing **delete** on any one would **invalidate** the other.

Your program should also not be written in a way that you have any uncertainty about the validity of **pointers** used.

- **Q: When should i use 'new(nothrow)'?**

A: If you don't want to handle the **exception** `std::bad_alloc`, you use the **nothrow** version of **operator new** that returns the **NULL** if the requested allocation **fails**.

- **Q: I can call a function to calculate the area using the following two methods:**

```
void CalculateArea(const double* const ptrRadius, double* const ptrArea);
void CalculateArea(const double& radius, double& area);
```

Which variant should I prefer?

A: In general, prefer the **reference version** (C++):

```
void CalculateArea(const double& radius, double& area);
```

Reason:

- References provide **cleaner syntax** (no , no & at call site).
- They guarantee a **valid object** must be passed (cannot be null).
- They make the function easier to read and use.

Important clarification:

- It is not that pointers are always "invalid".
- Pointers can be valid and safe if used correctly.
- However, pointers can also be **null or uninitialized**, which introduces extra risk.

Comparison:

- Pointer version:
 - * Can be null or invalid if misused

- * Requires explicit dereferencing (*)
- * More verbose at call site (&x)
- Reference version:
 - * Must always refer to a valid object
 - * Cleaner syntax (acts like a normal variable)
 - * Safer by default in most use cases

C vs C++ note:

- C does not support references.
- Therefore, C must use pointers.
- C++ supports both pointers and references.

• Q: I have a pointer

```
int number = 30;
const int* pointToAnInt = &number;
```

I understand that I cannot change the value of 'number' using the pointer 'pointToAnInt' due to the 'const' declaration. Can I assign 'pointToAnInt' to a non-const pointer and then use it to manipulate the value contained in integer number?

A: No, you cannot change the **const-correctness** of the pointer:

```
int* pAnother = pointToAnInt; // cannot assign pointer to const to a non-const
```

• Q: What is the difference btw these 2 declarations?

```
int myNums[100];
int* myArrays[100];
```

A: myNums is an array of integers — that is, myNums is a **pointer** to a **memory location** that holds 100 integers, points to the **first index** at index 0.

It is the static alternative to the following:

```
int* mNums = new int[100]; // dynamically allocated array
delete[] myNums;
```

myArrays, on the other hand, is an array of 100 **pointers**, each pointer being capable of pointing to an integer or an array of integers.

8.4.7 Quiz

1. Why can't you assign a **const** reference to a **non-const** reference?

A: You cannot bind a **const** reference to a **non-const** reference because it would allow the possibility of modifying a value that was originally declared as constant. This would violate C++'s type safety rules by potentially stripping away const-correctness. In general, C++ prevents dropping **const** qualifiers through reference assignment to ensure that immutable objects remain protected.

2. Are **new** and **delete** functions?

A: No, **new** and **delete** are not functions. They are C++ language operators. Unlike functions, they are built into the language and can be overloaded, but they do not behave like normal function calls.

3. What is the nature of value contained in a **pointer variable**?

A: A pointer variable stores a memory address as its value. This address refers to the location of another variable or object in memory.

4. What **operator** would you use to access the data pointed by a pointer?

A: The dereference operator ***** is used to access the value stored at the memory address pointed to by a pointer.

9 Classes and Objects

- Thus far, we've learned `procedural programming` and haven't dived into `object-oriented programming`. We now will.

9.1 Classes

- You declare a `class` by using the keyword `class` followed by the **name** of the class, followed by a **statement block** `{...}`
- A **declaration** of a `class` tells the **compiler** about the class and its properties.
- A class consists of its **fields** (data members) and `methods` (member functions).
- `Encapsulation` is the ability to logically **group data** and **functions** and we do that in our class in object-oriented programming.

9.1.1 An Object as an Instance of a Class

- A `class` is the **blueprint** and declaring a class alone has no interesting effect.
The **real-world** avator of a class at program execution time is an `object`.
- To use this blueprint you create an **instance** of the class, i.e. an `object`.
- You use this **object** to access its **attributes** and **member methods**. (or just attributes/fields and methods/functions)
- Creating an **object** of type `class Human` is similar to creating an instance of another type, say `double`:

```
double pi = 3.14;  
Human firstMan; // statically allocated object
```

- Alternatively, you can **dynamically** create an **instance** of `class Human` using `new` as you would for another type say `int`:

```
int* pointToAnInt = new int; // an integer allocated dynamically  
delete pointsToAnInt;
```

```
Human* firstHuman = new Human(); // dynamically allocated Human  
delte firstHuman;
```


9.1.2 Accessing Members Using the Dot Operator (.)

The dot operator (.) is used to access the data members and member functions of an object. It is applicable when working directly with an instance of a class rather than a pointer to that instance.

- Used with objects (instances) of a class.
- Accesses both data members and member functions.
- The object itself owns the members being accessed.

For example:

```
class Student {
public:
    std::string name;
    int age;

    void display() {
        std::cout << name << " is "
                  << age << " years old.\n";
    }
};

Student s;

s.name = "Alice";
s.age = 20;
s.display();
```

In this example, the object `s` accesses its data members and member function using the dot operator.

9.1.3 Accessing Members Using the Pointer Operator (->)

The pointer operator (->) is used to access the members of an object through a pointer. It combines pointer dereferencing and member access into a single operator.

- Used with pointers to objects.
- Accesses both data members and member functions.
- Equivalent to dereferencing the pointer and then using the dot operator:

`(*pointer).member`

For example:

```
Student* p = new Student;
```

```
p->name = "Bob";
```

```
p->age = 22;
```

```
p->display();
```

```
delete p;
```

The above is equivalent to:

```
(*p).name = "Bob";
```

```
(*p).age = 22;
```

```
(*p).display();
```

The pointer operator is generally preferred because it is shorter, easier to read, and less prone to errors than explicitly dereferencing the pointer before using the dot operator.

- **Listing 9.1** A compile-worthy class Human

9.1.4 Class vs Struct

In C++, both `class` and `struct` can contain data members, member functions, constructors, destructors, and other class features. The primary difference between them is their default access level.

- Both `class` and `struct` can contain data members and member functions.
- Both support constructors, destructors, inheritance, and polymorphism.
- The default access specifier for a `class` is `private`.
- The default access specifier for a `struct` is `public`.
- The default inheritance mode for a `class` is `private` inheritance.
- The default inheritance mode for a `struct` is `public` inheritance.
- By convention, `struct` is typically used for simple data structures, while `class` is used for objects that encapsulate both data and behavior.

For example, the following class has private members by default:

```
class Student {  
    int age;    // private by default  
};
```

To make members accessible, the `public` access specifier must be used:

```
class Student {  
public:  
    std::string name;  
    int age;  
};
```

In contrast, a structure has public members by default:

```
struct Student {  
    std::string name;    // public by default  
    int age;  
};
```

Thus, although `class` and `struct` are nearly identical in functionality, they differ primarily in their default access and inheritance rules, as well as their typical usage conventions.

9.1.5 Different Ways to Declare a struct

1. Define the structure and declare an object immediately:

```
struct Student {  
    std::string name;  
    int age;  
} student1;
```

Here, the structure type `Student` is defined, and an object named `student1` is created immediately.

2. Define the structure first, then declare objects later (most common):

```
struct Student {  
    std::string name;  
    int age;  
};  
  
Student student1;  
Student student2;
```

This is the preferred style because the type can be reused to create multiple objects.

3. Declare multiple objects when defining the structure:

```
struct Student {  
    std::string name;  
    int age;  
} student1, student2, student3;
```

Multiple objects can be declared immediately after the structure definition.

4. Declare a pointer to a structure:

```
struct Student {  
    std::string name;  
    int age;  
};  
  
Student* ptr = new Student;
```

The pointer `ptr` stores the address of a dynamically allocated `Student` object.

5. Anonymous structure (rarely used):

```
struct {  
    std::string name;  
    int age;  
} student1;
```

This creates an object named `student1` whose type has no name. Since the structure is anonymous, additional objects of the same type cannot be declared later.

Note on typedef struct In the C programming language, structures are commonly declared using `typedef` so that the `struct` keyword does not need to be written when declaring variables. For example:

```
typedef struct Student {  
    char name[50];  
    int age;  
} Student;
```

```
Student s1;
```

It is also common to define an anonymous structure using `typedef`:

```
typedef struct {  
    char name[50];  
    int age;  
} Student;
```

```
Student s1;
```

In C++, however, this is unnecessary because the structure name is automatically introduced as a type name:

```
struct Student {  
    std::string name;  
    int age;  
};
```

```
Student s1;
```

Although `typedef struct` is valid C++, it is considered redundant and is rarely used in modern C++ code.

9.1.6 Public, Private, Protected, Friend in C++

Access specifiers in C++ control the visibility and accessibility of class members. They are essential for encapsulation and data hiding.

- **public**
 - Members declared as **public** are accessible from anywhere in the program.
 - They form the interface of the class.
- **private**
 - Members declared as **private** are only accessible within the same class.
 - They are used to hide implementation details.
- **protected**
 - Members declared as **protected** are accessible within the class and its derived (child) classes.
 - They are not accessible from outside the class hierarchy.
- **friend**
 - The **friend** keyword allows a non-member function or another class to access private and protected members of a class.
 - Friendship is granted explicitly and is not inherited or transitive.

Example:

```
#include <iostream>
using namespace std;

class Base {
private:
    int privateValue = 10;

protected:
    int protectedValue = 20;

public:
    int publicValue = 30;

    void show() {
        cout << privateValue << " "
              << protectedValue << " "
              << publicValue << endl;
    }

    friend void friendFunction(Base obj);
};

void friendFunction(Base obj) {
    // Can access private and protected members
    cout << obj.privateValue << endl;
    cout << obj.protectedValue << endl;
    cout << obj.publicValue << endl;
}

class Derived : public Base {
public:
    void access() {
```

```

        // cout << privateValue;    // not accessible
        cout << protectedValue << endl; // accessible
        cout << publicValue << endl;    // accessible
    }
};

int main() {
    Base b;
    b.publicValue = 100;

    b.show();
    friendFunction(b);

    Derived d;
    d.access();

    return 0;
}

```

Key Summary:

- **public** → accessible everywhere
- **private** → accessible only inside the class
- **protected** → accessible in class and derived classes
- **friend** → grants special access to external functions/classes

9.1.7 This Keyword

The `this` keyword in C++ is an implicit pointer available inside all non-static member functions. It points to the object that invoked the member function.

- `this` is a pointer to the current object.
- It is automatically passed to all non-static member functions.
- It is used to access members of the current object explicitly.
- It helps resolve ambiguity when parameter names shadow member variables.
- It cannot be used in static member functions.

Example:

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;

public:
    void setAge(int age) {
        // 'this->age' refers to the member variable
        // 'age' refers to the parameter
        this->age = age;
    }

    void display() {
        cout << "Age: " << this->age << endl;
    }
};

int main() {
    Student s;
    s.setAge(20);
    s.display();

    return 0;
}
```

Key Points:

- `this` is an implicit pointer of type `ClassName* const`.
- It always points to the current object instance.
- It is commonly used to disambiguate member variables from parameters.
- It improves readability in method chaining scenarios.

9.1.8 Static Variables in Classes

A **static** data member in a class is shared among all objects of that class rather than each object having its own separate copy.

- A **static** member variable belongs to the class, not individual objects.
- Only one copy of the variable exists, regardless of how many objects are created.
- It is initialized outside the class definition.
- It can be accessed using the class name or through an object.
- It retains its value throughout the lifetime of the program.

Example:

```
#include <iostream>
using namespace std;

class Counter {
public:
    static int count;

    Counter() {
        count++;
    }

    void display() {
        cout << "Count: " << count << endl;
    }
};

// Definition and initialization of static member
int Counter::count = 0;

int main() {
    Counter c1;
    Counter c2;
    Counter c3;

    c1.display();
    c2.display();
    c3.display();

    cout << "Final count: " << Counter::count << endl;

    return 0;
}
```

Key Points:

- Static members are shared across all instances of a class.
- They must be defined outside the class using scope resolution operator `::`.
- They are useful for tracking shared state (e.g., object count, configuration values).

9.2 Constructors

- A `constructor` is a **special function** (or method) invoked during the **instantiation** of a **class** to construct an **object**.

9.2.1 Declaring and Implementing a Constructor

- A **constructor** is a **special function** that takes the name of the class and returns **no value**.
- So, class `Human` would have a constructor that is declared like this:

```
class Human
{
public:
    Human(); // declaration of a constructor
}
```

- This constructor can be implemented either **inline** within the class or **externally** outside the class declaration.
- An `implementation` (also called `definition`) inside the class looks like this:

```
class Human
{
public:
    Human()
    {
        // constructor code here
    }
};
```

- A variant enabling you to define the constructor **outside** the class's declaration looks like this:

```
class Human
{
public:
    Human(); // constructor declaration
};

// constructor implementation (definition)
Human::Human()
{
    // constructor code here
}
```

- the `::` is called the `scope resolution operator`.

For example, `Human::dateOfBirth` is referring to the **variable** `dateOfBirth` declared within the scope of class `Human`.

- `::dateOfBirth` would refer to the `dateOfBirth` in the global scope.

9.2.2 When and How to Use Constructors

- An **constructor** is always invoked during **object creation**, when an **instance** of a class is constructed.
- This is a perfect time to initialize the class members of your instance.
- **Listing 9.3** Using Constructors

9.2.3 Overloading Constructors

- Constructors can be **overloaded** (mean multiple constructors with different parameters) just like functions.
- Example:

```
class Human
{
public:
    Human() {...} // default constructor code

    Human(string humansName) {...} // overloaded constructor code
};
```

- **Listing 9.4** Multiple Constructors
- You can choose to **not** implement the ***default constructor*** to enforce **object instantiation** with certain minimal parameters as explained in the next section.

9.2.4 Class Without a Default Constructor

- **Listing 9.4** Class with overload constructor and No Default Constructor
- You also need not to declare any constructor since the default one is always provided implicitly

9.2.5 Constructor Parameters with Default Values

- Constructors can have **default values** like functions, allowing objects to be created with fewer arguments while still initializing members safely.

```
#include <iostream>
#include <string>

using namespace std;

class Student {
private:
    string name;
    int age;
    double gpa;

public:
    // Constructor with default parameter values
    Student(string n = "Unknown", int a = 0, double g = 0.0) {
        name = n;
        age = a;
        gpa = g;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    Student s1;                // uses all defaults
    Student s2("Alice");       // only name provided
    Student s3("Bob", 22);      // name + age
    Student s4("Charlie", 21, 3.9); // full initialization

    s1.display();
    cout << "-----" << endl;

    s2.display();
    cout << "-----" << endl;

    s3.display();
    cout << "-----" << endl;

    s4.display();

    return 0;
}
```

9.2.6 Constructors with Initialization Lists

An initialization list initializes a class's data members before the constructor body is executed. This is the preferred method of initializing member variables in modern C++.

- An initialization list appears after the constructor parameter list and before the constructor body.
- Data members are initialized directly rather than first being default-constructed and then assigned values.
- Initialization lists are generally more efficient than assigning values inside the constructor body.
- They are **required** when initializing:
 - `const` data members.
 - Reference (`&`) data members.
 - Class members that do not have a default constructor.
- Data members are initialized in the order they are declared in the class, **not** in the order they appear in the initialization list.

Syntax:

```
ClassName(type parameter)
    : member(parameter)
{
    // Constructor body
}
```

Example:

```
#include <iostream>
#include <string>

using namespace std;

class Student {
private:
    string name;
    int age;
    double gpa;

public:
    Student(string n, int a, double g)
        : name(n), age(a), gpa(g)
    {
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    Student s1("Alice", 20, 3.8);
    s1.display();

    return 0;
}
```

Example Requiring an Initialization List:

```
#include <iostream>

using namespace std;

class Example {
private:
    const int id;
    int& value;

public:
    Example(int i, int& v)
        : id(i), value(v)
    {
    }

    void display() {
        cout << "ID: " << id << endl;
        cout << "Value: " << value << endl;
    }
};

int main() {
    int x = 100;

    Example obj(1, x);
    obj.display();

    return 0;
}
```

In the second example, the initialization list is mandatory because both `id` (a `const` member) and `value` (a reference member) must be initialized when the object is created. They cannot be assigned values inside the constructor body.

9.3 Destructor

- A `destructor`, like a **constructor**, is a **special function**.
- A **constructor** is invoked at **object instantiation**, and a `destructor` is automatically invoked when an **object** is **destroyed**.

9.3.1 Declaring and Implementing a Destructor

- The **destructor** looks like a **function** that takes the name of the **class**, yet has a **tilde** (`~`).
- Example:

```
class Human
{
public:
    ~Human(); // declaration of a destructor
};
```

- The **destructor** can be implemented **inline** in the class or **externally** outside the class declaration.

`internally`:

```
class Human
{
public:
    ~Human()
    {
        // destructor code here
    }
};
```

`externally`:

```
class Human
{
public:
    ~Human(); // destructor declaration
};

// destructor definition (implementation)
Human::~~Human()
{
    // destructor code here
}
```


9.3.2 When and How to Use a Destructor

- A **destructor** is always invoked when an **object** of a class is **destroyed** when it **goes out of scope** or is **deleted** via `delete` or `delete[]` or `free`, etc.
- This property makes a **destructor** the ideal place to reset variables and **release dynamically allocated memory** and other resources.
 - `std::string` automatically manages dynamic memory for its character data, relieving the programmer from manually allocating and deallocating memory.
 - The `std::string` object itself is stored wherever it is created (e.g., on the stack for local variables or on the heap if dynamically allocated).
 - The character data managed by `std::string` may be stored within the object itself (using Small String Optimization) or in dynamically allocated heap memory, depending on the implementation and the length of the string.
 - But internally it would use constructors and destructors and operators.
- Listing 9.7 Destructor
- Smart pointers (Lesson 26) is where destructors play a critical role in working with pointers in a smarter way.
- **Destructors** cannot be overloaded.
- A class can only have **one destructor**, the compiler creates and invokes a dummy destructor, that is, an empty one (that does no cleanup of dynamically allocated memory)

9.4 Copy Constructors

- Argument passing to functions copy the argument into the function stack frame whenever the function is called and of course if it has arguments passed to it.
- This rule applies to **objects**, that is, instances of classes as well.

9.4.1 Shallow Copying and Associated Problems

- Classes such as `MyString` in **Listing 9.7** had a **pointer member** `buffer` that points to **dynamically allocated memory**, allocated in the constructor using `new` and then deallocated in the destructor using `delete[]`.
- When an **object** of this class is **copied**, the **pointer member** is copied, but not the **pointer memory**, resulting in 2 objects pointing to the **same dynamically allocated buffer in memory**.

So,

- Classes such as `MyString` in **Listing 9.7** contain a pointer member (`buffer`) that points to dynamically allocated memory. The memory is allocated in the constructor using `new` and released in the destructor using `delete[]`.
- When an object is copied, the compiler performs a **shallow copy** by default. This copies the **pointer value** (memory address), but **does not allocate or copy the dynamically allocated memory** it points to.
- As a result, both objects' pointer members refer to the **same block of dynamically allocated memory**. If one object modifies or deallocates the memory, it affects the other object, potentially leading to errors such as double deletion.
- Hence, when an **object** is **destructed**, `delete[]` deallocates the memory, thereby **invalidating** the **pointer copy** held by the other object.
- Such copies are *shallow* and are a **threat** to the stability of the program, as **Listing 9.8** Shallow copy Problem

Here is another small example:

```
#include <iostream>
#include <cstring>

using namespace std;

class MyString {
private:
    char* buffer;

public:
    MyString(const char* str) {
        buffer = new char[strlen(str) + 1];
        strcpy(buffer, str);
    }

    ~MyString() {
        delete[] buffer;
    }

    void display() {
        cout << buffer << endl;
    }
};

int main() {
    MyString s1("Hello");

    // Default copy constructor performs a shallow copy
    MyString s2 = s1;

    s1.display();
    s2.display();

    // When main() ends:
    // ~MyString() is called for s2 -> delete[] buffer;
    // ~MyString() is called for s1 -> delete[] buffer; (same memory!)
    // Result: Undefined behavior (double deletion)

    return 0;
}
```

This is a classic example demonstrating why the default copy constructor can be dangerous for classes that own dynamically allocated memory, and why a deep copy (custom copy constructor) is needed.

Output for listing 9.8:

```
String buffer in sayHello is 23 characters long
Buffer contains: Hello from String Class
Invoking destructor, clearing up for id: 21
Invoking destructor, clearing up for id: 0
free(): double free detected in tcache 2
Aborted (core dumped) ./a.out
```

where i changed the id in the callee to 21 and that shows the passed arg first destructs;

- So, the issue is both objects point to the same memory address, but when they both destructor we get the error of freeing an already deleted/freed data

9.4.2 Ensuring Deep Copy Using a Copy Constructor

- The **copy constructor** is an overloaded constructor that you supply.
- It is invoked everytime an **object** of the class is **copied**.
- So, we shall write a **copy constructor** to help solve the issues with **shallowing copying**, which we call **deep copying**.

- The **declaration syntax** of a **copy constructor** for class `MyString` is the following:

```
class MyString
{
    MyString(const MyString& copySource); // copy constructor
    // the const says "I promise not to modify the object I'm copying from."
};

another reason for the const:
MyString::MyString(const MyString& copySource) {
    // OK: read from copySource
    std::cout << copySource.buffer << std::endl;

    // Error: cannot modify the source object
    // copySource.buffer = nullptr;
}

...

MyString::MyString(const MyString& copySource)
{
    // copy constructor implementation code
}
```

- Thus, a **copy constructor** takes an **object** of the **same class** by **reference** as a parameter.
- This parameter is an **alias** of the **source object** and is the handle you have in writing your **custom copy code**.
- You would use the **copy constructor** to ensure a **deep copy** of **all buffers** in the source.
Like before you would only just copy the memory address, but here you would **allocate new memory on the heap** and then **copy the actual data contents** from the source object into this newly allocated memory, so that **both objects own independent copies** of the **data rather than** sharing the **same memory address**.

- **Listing 9.9** Define a copy constructor to ensure deep copy of dynamically allocated buffers.

- The **syntax difference** between a **regular constructor** and a **copy constructor** is in their **parameter lists**.

A regular constructor takes **raw values** (e.g., integers, strings, pointers), while a copy constructor takes a **reference to another object of the same class**.

```
// Regular constructor
MyString(const char* str);

// Copy constructor
MyString(const MyString& other);
```

- The copy constructor is used to create a new object from an existing object of the same type, typically performing a **deep copy** of dynamically allocated resources.
- Yes, the **copy constructor is invoked implicitly** in several situations where an object is copied. These include:

- When an object is initialized from another object:

```
MyString s2 = s1;
```

- When an object is passed to a function by value:

```
void func(MyString s);
func(s1);
```

- When an object is returned by value from a function:

```
MyString func();
```

- If no copy constructor is defined, C++ generates a default one that performs a **shallow copy**, meaning it copies pointer values rather than the data they point to.
- A **copy constructor can only take one argument**, which is a **reference to an object** of the **same class** (typically `const ClassName&`).
- This is because its purpose is to create a **new object as a copy** of an **existing object**.
- If more than one parameter is used, it is no longer considered a copy constructor but instead a regular overloaded constructor.
- The standard form is:

```
ClassName(const ClassName& source);
```

- The compiler does not "guess" or manually pass the object. Instead, when a copy constructor is invoked, the compiler automatically selects it based on the context where a copy is required (e.g., initialization from another object, passing by value, or returning by value).

Internally, the compiler rewrites the code so that the source object is passed as the single argument to the copy constructor. For example:

```
MyString s2 = s1;
```

is conceptually transformed by the compiler into:

```
MyString s2(s1);
```

Here, `s1` is explicitly passed as the argument to the copy constructor. The "implicit" part refers to the fact that the programmer does not explicitly call the constructor; the compiler inserts the call automatically based on the context.

- The **copy constructor** has ensured a **deep copy** in cases such as:

```
MyString sayHello("Hello");
UseMyString(sayHello);
```

- However, what if you tried copying via **assignment**:

```
MyString overwrite("who cares? ");
overwrite = sayHello;
```

This would **still** be a **shallow copy** because you still haven't yet supplied a **copy assignment operator**.

- In the absence of one, the compiler has supplied a default for you that does a **shallow copy**.
- The **copy assignment operator** is discussed in Lesson 12. Listing 12.8 is an improved `MyString` that implements the same:

```
MyString::operator= (const MyString& copySource)
{
    // ... copy assignment operator code
}
```

- Using **const** in the **copy constructor declaration** ensures that the **copy constructor** does **not modify** the **source object** being referred to.
- Additionally, the parameter in the **copy constructor** is **passed by reference** as a necessity.

If it weren't a reference, the **copy constructor** would itself invoke a copy, thus invoking itself again and so on till the system runs out of memory.

- Using **const** in the copy constructor ensures that the source object cannot be modified during copying. This guarantees that the original object remains unchanged and allows copying from both **const** and **non-const** objects.
- The copy constructor parameter must be passed by reference. If it were passed by value, a copy would need to be made to pass the argument into the constructor. This would recursively invoke the copy constructor again, leading to infinite recursion and eventually a stack overflow.
- Therefore, the correct and standard form of a copy constructor is:

```
ClassName(const ClassName& other);
```

- **Do** always implement a **copy constructor** and **copy assignment operator** when your class contains **raw pointer members** (e.g. `char*`). This prevents shallow copying and avoids issues such as double deletion and dangling pointers.
- **Do** declare the copy constructor using a **const reference parameter**:

```
ClassName(const ClassName& other);
```

This ensures the source object is not modified during copying and allows copying from both const and non-const objects.

- **Do** use the **explicit** keyword for constructors that take a single parameter to prevent unintended implicit conversions. For example:

```
class MyString {
public:
    explicit MyString(const char* str);
};

MyString s1 = "Hello";    // Not allowed (prevents implicit conversion)
MyString s2("Hello");    // Allowed (explicit construction)
```

- **Do** prefer using standard library types such as `std::string` and smart pointers (e.g. `std::unique_ptr`, `std::shared_ptr`) as class members instead of raw pointers, since they automatically manage memory and implement safe copy/move semantics.
- **Don't** use raw pointers as class members unless absolutely necessary, as they require manual memory management and increase the risk of memory leaks, double deletion, and undefined behavior.

9.4.3 Explicit keyword

- The `explicit` keyword in C++ is used to **prevent implicit conversions** that the compiler would otherwise perform automatically using constructors.
- Without `explicit`, a constructor that takes a single parameter can be used for implicit type conversion. For example:

```
class MyString {
public:
    MyString(const char* str) {
        // constructor
    }
};

MyString s = "Hello";    // implicit conversion allowed
```

- In this case, the compiler automatically converts the string literal into a `MyString` object.
- When the constructor is marked as `explicit`, implicit conversions are disallowed:

```
class MyString {
public:
    explicit MyString(const char* str) {
        // constructor
    }
};

MyString s = "Hello";    // Error: implicit conversion not allowed
MyString s2("Hello");    // OK: explicit construction
```

- The `explicit` keyword is used to prevent unintended or accidental conversions that may lead to bugs or unclear code.
- In summary, `explicit` forces the programmer to clearly state when a conversion or constructor call should occur.

- | | |
|------------------------|--|
| <i>explicit</i> | is a compile-time rule that prevents the compiler from performing automatic (implicit) type conversions using constructors. |
|------------------------|--|

9.4.4 Move Constructors Help Improve Performance

- There are cases where **objects** are subjected to copy steps automatically, due to the nature of the language and its needs.
- Consider:

```
class MyString
{
    // pick implementation from listing 9.9
};

MyString Copy(MyString& source) // function
{
    MyString copyForReturn(source.GetString()); // create copy
    return copyForReturn; // return by value invokes copy constructor
}

int main()
{
    MyString sayHello("Hi");
    MyString sayHelloAgain(Copy(sayHello)); // invokes 2x copy constructor
    return 0;
}
```

- As the comment indicates, in the **instantiation** of `sayHelloAgain`, the **copy constructor** is invoked twice, thus a **deep copy** was performed **twice** b/c our call to function `Copy(sayHello)` returns a `MyString` **by value**.
- However, this value returned is **very temporary** and is **not available** outside this expression. So, the **copy constructor** invoked in good faith by the C++ compiler is burden on **performance**. This impact becomes significant if our class were to contain objects of great size.

- To **avoid** this performance bottleneck, versions of C++ starting with C++11 feature a *move constructor* in addition to a **copy constructor**.
- The syntax of a *move constructor* is

```
// move constructor
MyString(MyString&& moveSource)
{
    if (moveSource != NULL)
    {
        buffer = moveSource.buffer; // take ownership i.e. 'move'
        moveSource.buffer = NULL; // set the move source to NULL
    }
}
```

– The constructor

```
MyString(MyString&& moveSource)
```

is called a **move constructor**.

- Here’s what each part means:
 - * `MyString` — the constructor’s name.
 - * `MyString&&` — an **rvalue reference** to another `MyString`.
 - * `moveSource` — the parameter name.
- In other words, this constructor says:

”This constructor takes ownership of another temporary (or movable) `MyString`.”
- Compare the different constructor signatures:


```
MyString(MyString other);      // pass by value
MyString(MyString& other);     // lvalue reference
MyString(MyString&& other);    // rvalue reference
```
- They each accept different kinds of arguments.
- Normally, variables are **lvalues**. For example:


```
MyString a("hello");
```

Since `a` has a name, it is an lvalue.
- A temporary object is an **rvalue**. For example:


```
MyString("hello")
```

or

```
return MyString("hello");
```

These objects are about to disappear, so C++ allows their resources to be transferred instead of copied.
- Example:


```
MyString a("hello");
MyString b(std::move(a));    // calls move constructor
```

`std::move(a)` converts `a` into an rvalue, so the move constructor is selected:

```
MyString(MyString&& moveSource)
```
- Inside the move constructor, the implementation may look like:


```
MyString::MyString(MyString&& moveSource)
{
    data = moveSource.data;      // steal pointer
    moveSource.data = nullptr;   // leave source safe
}
```

No new memory is allocated. The pointer ownership is simply transferred.
- Why not use `MyString&?`

If the constructor were

```
MyString(MyString& other)
```

then it could receive a normal object that the caller still intends to use. Stealing its resources would unexpectedly leave that object in an unusable state.
- The `&&` tells the compiler:

”This object is temporary (or has been explicitly marked movable), so it is safe to transfer its resources.”
- Summary:
 - * `&` = reference to a normal object (do not steal its resources).
 - * `&&` = reference to a temporary or movable object (safe to steal its resources).
 - * `MyString(MyString&& moveSource)` is the constructor that performs a **move** instead of a copy.

- When a **move constructor** is programmed, the compiler automatically opts for the same for "moving" the temporary resource and hence **avoiding** a **deep-copy** step.
- With this **move consutrctor** implemented, the comment should be appropriately changed to the following:

```
MyString sayHelloAgain(Copy(sayHello)); // invokes 1x copy, 1x move constructors
```

- The `move constructor` is usually implemented with the `move assignment operator`, which is dicussed in Lesson 12. Listing 12.11 is a better MyString that implements the **move constructor** and the **move assignment operator**.

9.5 Different Uses of Constructors and Destructors

- We've learned now concepts that enable you to create classes that can control how they're **created**, **copied**, **destroyed**, or **expose data** (through `access specifiers`).

9.5.1 Class That Does Not Permit Copying

- Sometimes we don't want duplicates at all in our program:

```
President ourPresident;  
DoSomething(ourPresident); // duplicate created in passing by value  
President clone;  
clone = curPresident;    // duplicate via assignment
```

- You need to ensure that certain resources **cannot** be **copied** or **duplicated**.
- If you **don't** declare a **copy constructor**, the C++ compiler inserts a **default public copy constructor** for you.
- This ruins your design and threatens your implementation.
- We have a solution.
- You would ensure that your class **cannot be copied** by declaring a **private copy constructor**.
- This ensures that the function call `DoSomething(ourPresident)` will cause a compiler failure.
- To avoid **assignment**, you declare a **private assignment operator**.

- Thus, the solution is the following:

```
class President  
{  
private:  
    President(const President&);    // private copy constructor  
    President& operator= (const President&); // private copy assignment operator  
  
    // ... other attributes  
};
```

- There is **no need for implementation** of the **private copy constructor** or **assignment operator**.
- Just **declaring them private** is adequate and **sufficient** toward fulfilling your goal of ensuring **non-copyable objects** of class `President`.

9.5.2 Singleton Class That Permits a Single Instance

- class `President` earlier is good, but has a **shortcoming**:
It cannot help creation of multiple presidents via instantiation of multiple objects:

```
President One, Two, Three;
```

- Individually, they are **non-copyable** thanks to the **private copy constructors**, but what youi ideally need is a class `President` that has **one**, and only one, real-world manifestation — that is, there is **only one object** and creation of additional ones is prohibited.
- Welcome to the concept of `singleton` that uses `private constructors`, a `private assignment operator`, and a `static instance member` to create this (controversally) powerful pattern.

- When the keyword `static` is used on a **class's data member**, it ensures that the member is shared across **all instances**
- When `static` is used on a **local variable** declared within the scope of a function, it ensures that the **variable retains its value** between **functions calls**.
- When `static` is used on a **member function** — a **method** — the method is **shared** across **all instances** of the class.

- `Listing 9.10 Singleton class President`

9.6 Class That Prohibits Instantiation on the Stack

- Space on the stack is often limited
- If you are writing a database that may contain **terabytes** of data in its internal structures, you might want to ensure that a client of this **class** cannot instantiate it on the **stack**; instead it is **forced** to create **instances** only on the free store.
- The key to ensuring this is **declaring** the **destructor** **private**:

```
class MonsterDB
{
private:
    ~MonsterDB(); // private destructor
    // ... members that consume a huge amount of data
};
```

- **Declaring** a **private destructor** ensures that one is **not allowed** to create an **instance** like this:

```
int main()
{
    MonsterDB myDatabase; // compile error
    // ... more code
    return 0;
}
```

- This **instance**, if successfully constructed, would be on the **stack**.
- **All objects** on the **stack** get popped when the **stack** is unwound and therefore the compiler would need to compile and invoke the **destructor** `~MonsterDB()` at the **end** of `main()`.
- However, this **destructor** is private and there inaccessible, resulting in a compile failure.

- A **private destructor** would **not stop** you from instantiating on the *heap*:

```
int main()
{
    MonsterDB* myDatabase = new MonsterDB(); // no error
    // ... more code
    return 0;
}
```

- We have a memory leak here: as the **destructor** is not accessible from **main**, you cannot do a **delete**, either.
- What class `MonsterDB` needs to support is a **public static member function** that would **destroy** the **instance** (a class member would have access to the **private destructor**.)
- Listing 9.11 Only allowed creation on heap

9.6.1 Using Constructors to Convert Types

- Earlier we've learned **constructors** can be **overloaded**, that is, they may take **one or more parameters**.
- This feature is often used to **convert** one type to another.
- Let's see a class `Human` that features an **overloaded constructor** that accepts an integer.

```
class Human
{
    int age;
public:
    Human(int humansAge) : age(humansAge) {}
};

// function that take a human as a parameter
void DoSomething(Human person)
{
    cout << "Human sent did something" << endl;
    return;
}
```

- The **constructor** allows a **conversion**:

```
Human kid(10); // convert integer into a Human
DoSomething(kid);
```

-
- I think the textbook messed here/is misleading so here is update:

```
No conversion (explicit construction)
Human kid(10);
DoSomething(kid);
```

- Yes. If you write:

```
Human kid(10);
DoSomething(10);
```

the `Human` object named `kid` is **not** used.

- Instead, the compiler creates a **completely separate temporary** `Human` object for the function call.
- Conceptually, it is as if you had written:

```
Human kid(10);      // Your object

Human temp(10);     // Compiler-created temporary
DoSomething(temp);
```

- Therefore, there are **two different** `Human` objects:
 - * `kid` — the object that you explicitly created.
 - * `temp` — a temporary object automatically created by the compiler.
- If, instead, you write:

```
Human kid(10);
DoSomething(kid);
```

then there is **no type conversion**. The function receives the object `kid` (or a copy of it, since the parameter is passed by value).

- Compare the two cases.

*** Case 1: No conversion**

```
Human kid(10);
DoSomething(kid);
```

The object flow is:

```
kid -----> DoSomething
```

*** Case 2: Implicit conversion**

```
DoSomething(10);
```

The object flow is:

```
10
```

```
|
```

```
v
```

```
Human(10)  <-- temporary object created by the compiler
```

```
|
```

```
v
```

```
DoSomething
```

- Therefore, the temporary object is **not** the same object that you previously created. It is a brand new object that exists only for the duration of the function call and is destroyed immediately afterward.
- This automatic creation of a temporary object is what allows a **single-argument constructor** to perform an **implicit type conversion**.

- We fix this via the keyword `explicit`
-

- Such **converting constructors** allow `implicit conversions`.

```
Human anotherKid = 11; // int converted to Human
DoSomething(10); // 10 converted to Human!
```

- We declared `DoSomething(Human person)` as a **function** that accepts a parameter of **type Human** and does not accept an int!
- So why did the line work?
- The compiler knows that **class Human supports a constructor** that accepts an integer and performed **implicit conversion!**

It created an **object** of type **Human** using the integer you supplied and sent it as an argument to the function.

- To **void implicit conversions**, use keyword `explicit` at the time of **declaring the constructor**.

```
class Human
{
    int age;
public:
    explicit Human(int humansAge) : age(humansAge) {}
};
```

- Using `explicit` is **not** a prerequisite but in many cases a **good programming practice**.
- **Listing 9.12** Using `explicit` to block implicit conversions

- The problem of **implicit conversion** and **avoiding** them using keyword `explicit` applies to operators too.

9.7 sizeof() a Class

- The `sizeof` operator returns the number of **bytes** occupied by an object or a type in memory.
- When applied to a class, `sizeof` returns the total amount of memory required to store all of its **data members**, including any **padding bytes** that the compiler inserts for memory alignment.
- The size of a class **does not** include the code for its member functions. Member functions are stored only once in the program's executable, regardless of how many objects of the class are created.
- A class may contain:
 - Fundamental data types (such as `int`, `char`, and `double`).
 - Objects of other classes.
 - Structures (`struct`).
 - Arrays and pointers.
- When a class contains another class or a structure as a data member, the size of the containing class includes the size of those objects as well.
- Consider the following example:

```
struct Address
{
    int houseNumber;
    char streetInitial;
};

class Birthday
{
public:
    int day;
    int month;
    int year;
};

class Student
{
private:
    int id;           // Fundamental type
    double gpa;       // Fundamental type
    char grade;       // Fundamental type

    Address address;   // Structure
    Birthday birth;    // Another class
};
```

- The size of a `Student` object is conceptually the sum of the sizes of all its data members:

```
sizeof(Student)
=
sizeof(int)
+ sizeof(double)
+ sizeof(char)
+ sizeof(Address)
+ sizeof(Birthday)
+ padding (if required)
```

- The compiler may insert **padding bytes** between members so that each data member begins at an address suitable for efficient access by the CPU.
- Therefore, the value returned by `sizeof(Student)` is often **larger** than simply adding the sizes of the individual data members.
- For example:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "sizeof(Address) = "
          << sizeof(Address) << endl;

    cout << "sizeof(Birthday) = "
          << sizeof(Birthday) << endl;

    cout << "sizeof(Student) = "
          << sizeof(Student) << endl;

    return 0;
}
```

- The exact output depends on:
 - The compiler being used.
 - The target architecture (32-bit or 64-bit).
 - The alignment rules of the system.
- In general,

```
sizeof(class) =
sum of the sizes of its data members
+ any compiler-added padding
```

- Remember that `sizeof` measures only the memory occupied by the object's data. It does **not** include memory used by member functions, nor does it include memory dynamically allocated through pointers.

So:

Typical output (64-bit system):

```
sizeof(Address)  = 8
sizeof(Birthday) = 12
sizeof(Student)  = 40
```

Explanation of the output

- Assume typical sizes:

- int = 4 bytes
- double = 8 bytes
- char = 1 byte

- **Address struct:**

```
struct Address
{
    int houseNumber;    // 4 bytes
    char streetInitial; // 1 byte
};
```

Memory layout:

houseNumber	4 bytes
streetInitial	1 byte
padding	3 bytes

Total	8 bytes

- **Birthday class:**

```
class Birthday
{
public:
    int day;
    int month;
    int year;
};
```

Memory layout:

day	4 bytes
month	4 bytes
year	4 bytes

Total	12 bytes

- **Student class:**

```
class Student
{
private:
    int id;           // 4 bytes
    double gpa;       // 8 bytes
    char grade;       // 1 byte

    Address address;   // 8 bytes
    Birthday birth;    // 12 bytes
};
```

Memory layout (typical alignment):

id	4 bytes	
padding	4 bytes	// align double
gpa	8 bytes	
grade	1 byte	
padding	7 bytes	// align Address
address	8 bytes	
birth	12 bytes	
padding	4 bytes	// final alignment

Total	40 bytes	

- Even though the raw sum of members is:

$4 + 8 + 1 + 8 + 12 = 33$ bytes

- The extra bytes come from **padding added by the compiler** to satisfy memory alignment rules, which makes access faster on most CPUs.

9.8 How struct differs from class

- struct are public by default.
- class is private by default.
- struct implementation:

```
#include <iostream>
#include <string>

using namespace std;

struct Student {
private:
    string name;
    int age;
    double gpa;

public:
    // default constructor
    Student() {}

    // Constructor to initialize data members
    Student(string n, int a, double g) {
        name = n;
        age = a;
        gpa = g;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    // Creating objects using constructor
    Student s1("Alice", 20, 3.8);
    Student s2("Bob", 22, 3.5);

    s1.display();
    cout << "-----" << endl;
    s2.display();

    return 0;
}
```

- class implementation

```
#include <iostream>
#include <string>

using namespace std;

class Student {
private:
    string name;
    int age;
    double gpa;

public:
    Student() {} // default constructor

    // Constructor to initialize data members
    Student(string n, int a, double g) {
        name = n;
        age = a;
        gpa = g;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
        cout << "GPA: " << gpa << endl;
    }
};

int main() {
    // Creating objects using constructor
    Student s1("Alice", 20, 3.8);
    Student s2("Bob", 22, 3.5);

    s1.display();
    cout << "-----" << endl;
    s2.display();

    return 0;
}
```

- Lookover

Class vs Struct

section above for more details.

9.9 Declaring a 'friend' of a class

- A class **does not permit** access to its **data members and methods** that are declared **private**.
- The rule is waived for **classes** and **functions** that are disclosed as `friend classes` or `friend functions` using the keyword `friend`
- `Listing 9.14 friend`

```
class B;

class A
{
private:
    int secret = 99;

    friend class B;
};

class B
{
public:
    void show(A obj)
    {
        std::cout << obj.secret << std::endl; // allowed
    }
};
```


9.10 union: A Special Data Storage Mechanism

- A `union` is a special class type where all non-static data members share the same memory location.
- This means only one member can hold a meaningful value at a time.
- A union **cannot participate in inheritance hierarchies**. It cannot be used as a base class or derived class.
- The `sizeof` of a union is always equal to the size of its **largest member**, even if that member is not currently active in a given instance.
- Example:

```
#include <iostream>
using namespace std;

union Data
{
    int i;
    float f;
    char c;
};

int main()
{
    Data d;

    d.i = 42;
    cout << "i = " << d.i << endl;

    d.f = 3.14f;
    cout << "f = " << d.f << endl;

    d.c = 'A';
    cout << "c = " << d.c << endl;

    cout << "sizeof(Data) = " << sizeof(Data) << endl;

    return 0;
}
```

- In this example, all members share the same memory, so writing to one member **overwrites** the previous value.
- Even though multiple members exist, only one is valid at any time.
- The output of `sizeof(Data)` depends on the largest member:
 - `int` might be 4 bytes
 - `float` might be 4 bytes
 - `char` might be 1 byte
- Therefore:
`sizeof(Data) = size of largest member (usually 4 bytes in this example)`
- The fact that a larger member exists does not change the size unless alignment/padding rules require it.
- C++17 introduces `std::variant` as an alternative typesafe structure to `union`

9.11 Using Aggregate Initialization on Classes and Structs

- **Aggregate initialization** is a way of initializing objects (typically **structs** and simple **classes**) using curly braces `{}` without explicitly calling a constructor.
- It allows you to initialize all public data members in order.
- This works best with **structs** or classes that behave like simple data containers (no private members or user-defined constructors in older rules).
- Example using a **struct**:

```
#include <iostream>
using namespace std;

struct Point
{
    int x;
    int y;
};

int main()
{
    Point p1 = {10, 20};    // aggregate initialization

    cout << "x = " << p1.x << endl;
    cout << "y = " << p1.y << endl;

    return 0;
}
```

- The values inside the braces are assigned to data members **in order**:
 - first value → first member (**x**)
 - second value → second member (**y**)
- Aggregate initialization also works with simple classes that have **public data members and no user-declared constructors**:

```
#include <iostream>
using namespace std;

class Color
{
public:
    int r;
    int g;
    int b;
};

int main()
{
    Color c1 = {255, 128, 64};

    cout << c1.r << " " << c1.g << " " << c1.b << endl;

    return 0;
}
```

- However, if a class has a **user-defined constructor**, then aggregate initialization is no longer used in the same way (it will prefer constructors instead).
- Example difference:

```
struct A
{
    int x;
    int y;
};

A a1 = {1, 2};    // OK: aggregate initialization

class B
{
public:
    B(int a, int b) {}
};

B b1 = {1, 2};    // calls constructor, NOT aggregate initialization
```

- Aggregate initialization is useful because it provides a concise way to initialize simple data structures without writing constructors.
- It is commonly used in:
 - simple structs
 - POD (plain old data) types
 - configuration-like data containers

9.12 constexpr with Classes and Objects

- We learned `constexpr` in Lesson 3 where it offers a powerful way to **improve performance** by computing the expression at **compile-time**.
- By marking your **functions** that operate on **constants** or **const-expression** as `constexpr`, we are instructing the compile to evaluate those functions and **insert their result** instead of inserting instructions that compute the result when the applicatin is executed.
- This keyword can also be used with **classes** and **objects** that evaluate as **constants** as demonstrated by **Listing 9.18** `constexpr` with class Human.
- Note the **compiler** would **ignore** the `constexpr` when the function or class is used with entities that are **not constant**.

9.13 QA

- **Q: What is the difference between the instance of a class and an object of that class?**

A: Essentially none. When you **instantiate** a class, you get an **instance** that can also be called an **object**.

- **Q: What is a better way to access members: using the dot operator (.) or using the pointer operator (— >)?**

A: If you have a **pointer** to an **object**, the **pointer operator** would be best suited. If you have instantiated an **object** as a **local variable** on the stack, then the **dot operator** is best suited.

- **Q: Why is the parameter of a copy constructor one that takes the copy source by reference?**

A: For one, the **copy constructor** is **expected** by the compiler that way. The reason behind it is that a **copy constructor** would invoke itself if it accepted the copy source **by value**, resulting in an endless **copy loop**.

9.14 Quiz

1. My class has a raw pointer `int*` that contains a dynamically allocated array of integers. Does `sizeof` report different sizes depending on the number of integers in the dynamic array?

A:

- No. The `sizeof` operator reports only the size of the pointer itself, not the size of the dynamically allocated memory that it points to.
- The dynamically allocated array resides on the **heap**, while the pointer is simply a variable stored as part of the class object.
- For example:

```
class Numbers
{
private:
    int* data;
};
```

- On a typical 64-bit system:

```
sizeof(Numbers) == 8
```

because the class contains only a single pointer, which is typically 8 bytes in size.

- Even if the pointer refers to arrays of different lengths:

```
Numbers a;    // data points to 5 integers
Numbers b;    // data points to 100 integers
Numbers c;    // data points to 1,000,000 integers
```

the result of `sizeof` is the same for every object:

```
sizeof(a) == sizeof(b) == sizeof(c)
```

- This is because `sizeof` measures only the memory occupied by the class object itself, not any memory that has been dynamically allocated on the heap.

10 Implementing Inheritance

- **Object-oriented programming** is based on 4 important aspects:
 1. encapsulation
 2. abstraction
 3. inheritance
 4. polymorphism

10.1 Basics of Inheritance

- **Inheritance** is an object-oriented programming (OOP) feature that allows one class to **acquire** the data members and member functions of another class.
- The class being **inherited from** is called the **base class** (also called the **parent** or **superclass**).
- The class that **inherits from** another class is called the **derived class** (also called the **child** or **subclass**).
- Inheritance promotes **code reuse** by allowing a derived class to reuse functionality already implemented in its base class.
- A derived class may:
 - Inherit existing data members.
 - Inherit existing member functions.
 - Add new data members.
 - Add new member functions.
 - Override inherited virtual functions.
- The general syntax for inheritance is:

```
class DerivedClass : access-specifier BaseClass
{
    // additional members
};
```

- The access specifier is commonly one of:
 - **public**
 - **protected**
 - **private**
- Public inheritance is the most common form and models an **"is-a"** relationship.
- Example:

```
#include <iostream>
#include <string>
using namespace std;

class Animal
{
public:
    void Eat()
```



```

        {
            cout << "Animal is eating." << endl;
        }
};

class Dog : public Animal
{
public:
    void Bark()
    {
        cout << "Dog is barking." << endl;
    }
};

int main()
{
    Dog pet;

    pet.Eat();    // inherited from Animal
    pet.Bark();   // defined in Dog

    return 0;
}

```

- Output:

```

Animal is eating.
Dog is barking.

```

- In the example above:
 - **Animal** is the base class.
 - **Dog** is the derived class.
 - The **Dog** object inherits the **Eat()** member function.
 - The **Dog** class also defines its own member function, **Bark()**.
- Conceptually, inheritance forms a hierarchy:

```

Animal
|
+----- Dog

```

- Since every **Dog** object is also an **Animal** object, a **Dog** can use all accessible members inherited from **Animal** in addition to its own members.
- Inheritance is one of the fundamental mechanisms that enables polymorphism and code reuse in object-oriented programming.

10.1.1 Access specific keyword `protected`

- Here is a mechanism that allows **derived class members** to modify chosen attributes of the **base class**, while **denying access** to the same from everyone else.
- `protected` keyword to the rescue.

- `protected`, like `public` and `private`, is also an `access specifier`.
- When you declare a **class attribute** or **function** as `protected`, you are effectively making it accessible to classes that **derive** (and **friends**), yet simultaneously making it **inaccessible** to everyone else **outside the class**, including `main()`

- `protected` is the **access specifier** you should use if you want a certain attribute in a **base class** to be **accessible** to classes that **derive** from this **base class**, as demonstrated in `Listing 10.2`.

10.1.2 Base Class Initiation — Passing Parameters to the Base Class

- What if a **base class** were to contain an **overloaded constructor** that requires arguments at the time of **instantiation**?
- How would such a **base class** be **instantiated** when the **derived class** is being **constructed**?
- The clue lies in using **initialization lists** and in invoking the appropriate **base class constructor** via the **constructor** of the **derived class** as shown in the following code:

```
class Base
{
public:
    Base(int someNumber) // overloaded constructor
    {
        // use someNumber
    }
};

class Derived: public Base
{
public:
    Derived() : Base(25) // instantiate Base with argument 25
    {
        // derived class constructor code
    }
};
```

- This is nice because the base class can ensure **every derived class** is forced to engage with the specific constructor as shown in Listing 10.3
- In this **Fish** now has a **constructor** that takes a **default parameter** initializing **Fish::isFreshWaterFish**. Thus, the **only possibility** to create an **object** of **Fish** is via providing it a **parameter** that initialized the **protected member**.
- This way **class Fish** ensures that the **protected member** doesn't contain a **random value**, especially if a **derive class** forgets to set it.
- The **derived classes** are now **forced** to define a **constructor** that instantiates the **base class instance** of **Fish** with the right parameter.

10.1.3 Derived Class Overriding Base Class's Methods

- if a class `Derived` implements the same functions with the same return values and signatures as in a class `Base` it inherits from, it effectively **overrides** that method in class `Base` as shown:

```
class Base
{
public:
    void DoSomething() {}
};

class Derived : public Base
{
public:
    void DoSomething() {}
};
```

- Thus, if **method** `DoSomething()` were to be invoked using an **instance** of `Derived`, the it would **not** invoke the functionality in class `Base`.
- | |
|--|
| Listing 10.4 Derived classes Overriding Base method |
|--|

10.1.4 Invoking Overridden Methods of a Base Class

- In **Listing 10.4** we saw a **derived class overriding** a function from the **base class** by implementing its version of the same signature.
- If you instead want to invoke the base's method, you have to use *scope resolution operator (::)*

```
myDinner.Fish::Swim(); // invokes Fish::Sim() using instance of tuna
// Fish being the base class and Tuna being the child class
```

- **Listing 10.5** Invoking Base class functions from derived class using ::

10.1.5 Derived Class Hiding Base Class's Methods

- **Overriding** can take an extreme form where `Tuna::Swim()` can potentially **hide** all **overloaded** versions of `Fish::Swim()` available, even causing compilation failure when the **overloaded** ones are used (hence, called **hidden**), as demonstrated by **Listing 10.6**.
- We hide the function in the base class from the compiler.
- So, if you want to invoke a **base function** like `Fish::Swim(bool)` via an **instance of Tuna (child class)**, you have the following solutions

1. Solution 1: Use the **scope resolution operator** in `main()`

```
myDinner.Fish::Swim();
```

2. Solution 2: Use keying **using** in class `Tuna` to **unhide** `Swim()` in class `Fish`:

```
class Tuna : public Fish
{
public:
    using Fish::Swim; // unhide all Swim() methods in class Fish

    void Swim()
    {
        cout << "Tuna swims real fast" << endl;
    }
};
```

3. Solution 3: **Override** all **overloaded variants** of `Swim()` in class `Tuna` (invoke methods of `Fish::Swim(...)` via `Tuna::Fish(...)` if you want):

```
class Tuna : public Fish
{
public:
    void Swim(bool isFreshWaterFish)
    {
        Fish::Swim(isFreshWaterFish);
    }

    void Swim()
    {
        cout << "Tuna swims real fast" << endl;
    }
};
```

10.2 Order of Construction

- When an object of a derived class is created, C++ constructs the **base class portion first**, followed by the **derived class portion**.
- This ensures that the base class is fully initialized before the derived class begins constructing its own data members.
- The order of construction is:
 1. Base class constructor.
 2. Derived class constructor.
- Example:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    Animal()
    {
        cout << "Animal constructor" << endl;
    }
};

class Dog : public Animal
{
public:
    Dog()
    {
        cout << "Dog constructor" << endl;
    }
};

int main()
{
    Dog pet;

    return 0;
}
```

- Output:

```
Animal constructor
Dog constructor
```

- Although the object being created is a `Dog`, the `Animal` portion must be constructed first because every `Dog` object contains an `Animal` subobject.
- A useful way to visualize the process is:

```
Create Dog object
      |
      v
Construct Animal
      |
      v
Construct Dog
```

10.3 Order of Destruction

- Object destruction occurs in the **reverse order** of construction.
- When a derived object is destroyed, the derived class destructor executes first, followed by the base class destructor.
- The order of destruction is:
 1. Derived class destructor.
 2. Base class destructor.
- Example:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    Animal()
    {
        cout << "Animal constructor" << endl;
    }

    ~Animal()
    {
        cout << "Animal destructor" << endl;
    }
};

class Dog : public Animal
{
public:
    Dog()
    {
        cout << "Dog constructor" << endl;
    }

    ~Dog()
    {
        cout << "Dog destructor" << endl;
    }
};

int main()
{
    Dog pet;

    return 0;
}
```

- Output:

```
Animal constructor
Dog constructor
Dog destructor
Animal destructor
```


- The derived class is destroyed first because it may depend on resources owned by the base class. The base class is destroyed only after the derived class has finished cleaning up.
- A useful way to visualize destruction is:

```
Destroy Dog object
    |
    v
Destroy Dog
    |
    v
Destroy Animal
```

- In summary:
 - Construction proceeds from the base class to the derived class.
 - Destruction proceeds from the derived class back to the base class.

10.4 Private Inheritance

- `Private inheritance` differs from `public inheritance` in that the inherited members become **private** members of the derived class (unless they were already private in the base class).
- The general syntax is:

```
class Derived : private Base
{
    // ...
};
```

- With private inheritance:
 - Public members of the base class become private in the derived class.
 - Protected members of the base class also become private in the derived class.
 - Private members of the base class remain inaccessible to the derived class.
- Example:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void Eat()
    {
        cout << "Animal is eating." << endl;
    }
};

class Dog : private Animal
{
public:
    void MakeDogEat()
    {
        Eat();    // OK: inherited as private
    }
};

int main()
{
    Dog pet;

    pet.MakeDogEat();

    // Compile error:
    // pet.Eat();

    return 0;
}
```

- Output:

`Animal is eating.`
- Although `Eat()` was originally a public member of `Animal`, it becomes a private member of `Dog` because of private inheritance.

- Therefore:
 - The `Dog` class itself can call `Eat()`.
 - Code outside of `Dog` cannot call `Eat()` through a `Dog` object.
- Unlike public inheritance, private inheritance does **not** model an **”is-a”** relationship.
- Instead, private inheritance is sometimes used to express an implementation relationship, where the derived class reuses the base class’s implementation without exposing its interface publicly.
- In modern C++, composition is generally preferred over private inheritance unless there is a specific reason to inherit privately.

10.5 Composition (side note)

- **Composition** is an object-oriented programming technique in which one class contains an object of another class as one of its data members.
- Composition models a **"has-a"** relationship.
- Inheritance models a **"is-a"** relationship.
- For example, a **Car has an Engine**, so composition is an appropriate design choice.

```
class Engine
{
public:
    void Start()
    {
        cout << "Engine started." << endl;
    }
};

class Car
{
private:
    Engine engine;    // Car has an Engine

public:
    void StartCar()
    {
        engine.Start();
    }
};
```

- By contrast, inheritance models an **"is-a"** relationship.
- For example, a **Dog is an Animal**, so inheritance is appropriate.

```
class Animal
{
public:
    void Eat() {}
};

class Dog : public Animal
{
};
```

- As a rule of thumb:
 - Use **inheritance** when there is an **"is-a"** relationship.
 - Use **composition** when there is a **"has-a"** relationship.
- This is why modern C++ encourages the design principle:

"Favor composition over inheritance."

- In many situations, composition leads to code that is easier to understand, more flexible, and simpler to maintain.

10.6 Protected Inheritance

- **Protected inheritance** is another form of inheritance in which the inherited `public` and `protected` members of the base class become `protected` members of the derived class.
- The general syntax is:

```
class Derived : protected Base
{
    // ...
};
```

- With protected inheritance:
 - Public members of the base class become protected.
 - Protected members of the base class remain protected.
 - Private members of the base class are still inaccessible.
- Example:

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void Eat()
    {
        cout << "Animal is eating." << endl;
    }
};

class Dog : protected Animal
{
public:
    void MakeDogEat()
    {
        Eat();    // OK
    }
};

class Puppy : public Dog
{
public:
    void Feed()
    {
        Eat();    // OK because Eat() is protected in Dog
    }
};

int main()
{
    Dog pet;
    pet.MakeDogEat();

    // Compile error:
    // pet.Eat();

    Puppy puppy;
```

```

        puppy.Feed();

    return 0;
}

```

- Output:

```

Animal is eating.
Animal is eating.

```

- In this example:
 - The `Dog` class can call `Eat()`.
 - The `Puppy` class can also call `Eat()` because it inherits from `Dog`.
 - Code outside these classes cannot call `Eat()` through a `Dog` or `Puppy` object.
- Therefore, protected inheritance allows derived classes to continue using inherited functionality while preventing direct access from outside the class hierarchy.
- The access transformations are summarized below:

Base member	After protected inheritance

<code>public</code>	<code>protected</code>
<code>protected</code>	<code>protected</code>
<code>private</code>	<code>inaccessible</code>

- Protected inheritance is used less frequently than public inheritance. It is most useful when a class wishes to reuse a base class's implementation while allowing only further derived classes to access the inherited members.
- Private and protected signify a "has-a" relationship

10.7 Multiple Inheritance

- **Multiple inheritance** allows a class to inherit from **more than one base class**.
- This enables a derived class to reuse the functionality of multiple independent classes.
- The general syntax is:

```
class Derived : public Base1, public Base2
{
    // ...
};
```

- Example:

```
#include <iostream>
using namespace std;

class Printer
{
public:
    void Print()
    {
        cout << "Printing..." << endl;
    }
};

class Scanner
{
public:
    void Scan()
    {
        cout << "Scanning..." << endl;
    }
};

class AllInOne : public Printer, public Scanner
{
};

int main()
{
    AllInOne device;

    device.Print();
    device.Scan();

    return 0;
}
```

- Output:

```
Printing...
Scanning...
```

- In this example:
 - **Printer** is a base class.
 - **Scanner** is another base class.

- AllInOne inherits from both classes.
- As a result, an AllInOne object has access to the member functions of both base classes.
- One issue that can arise with multiple inheritance is `name ambiguity`. For example:

```
class A
{
public:
    void Hello()
    {
        cout << "Hello from A" << endl;
    }
};

class B
{
public:
    void Hello()
    {
        cout << "Hello from B" << endl;
    }
};

class C : public A, public B
{
};

int main()
{
    C obj;

    // obj.Hello();    // Compile error: ambiguous

    obj.A::Hello();
    obj.B::Hello();

    return 0;
}
```

- Since both base classes define a member function named `Hello()`, the compiler cannot determine which one should be called.
- The ambiguity can be resolved using the `scope resolution operator` (`::`), as shown above.
- Multiple inheritance is a powerful language feature, but it should be used carefully because it can make class hierarchies more complex.
- One well-known issue associated with multiple inheritance is the **Diamond Problem**, which is discussed separately using `virtual inheritance`.

10.8 Avoid Inheritance via final

- The **final** keyword prevents a class from being used as a **base class**.
- Any attempt to derive from a class marked **final** results in a compile-time error.
- The general syntax is:

```
class ClassName final
{
    // ...
};
```

- Example:

```
#include <iostream>
using namespace std;

class Animal final
{
public:
    void Speak()
    {
        cout << "Animal speaks." << endl;
    }
};

// Compile error:
// class Dog : public Animal
// {
// };

int main()
{
    Animal pet;
    pet.Speak();

    return 0;
}
```

- Output:

```
Animal speaks.
```

- In the example above, **Animal** is declared as **final**, so no other class may inherit from it.
- Attempting to write:

```
class Dog : public Animal
{
};
```

results in a compile-time error because **Animal** is a **final** class.

- The **final** keyword is commonly used to:
 - Prevent further inheritance.
 - Protect a class from unintended extension.
 - Indicate that the class represents a complete implementation.

- The **final** keyword can also be applied to **virtual member functions**.
- When a virtual function is declared **final**, derived classes may inherit the function, but they cannot override it.

```
class Base
{
public:
    virtual void Print() final
    {
        cout << "Base::Print()" << endl;
    }
};

class Derived : public Base
{
    // Compile error:
    // void Print() override { }
};
```

- In summary:
 - Applying **final** to a class prevents inheritance.
 - Applying **final** to a virtual function prevents overriding.

11 Polymorphism

11.1 Basics of Polymorphism

- "Poly" is Greek for *many* and "morph" means *form*.
- **Polymorphism** is the feature of **object-oriented** programming that allows **objects** of different types to be accessed through a common interface, with each object providing its own implementation of the same operation.
- We go over **polymorphic** behavior that can be implemented in C++ via the **inheritance hierarchy** known as **subtype polymorphism**.

11.1.1 Need for Polymorphic Behavior

- Last section we saw how Tuna and Carp inherit public methods Swim() from Fish (Listing 10.1)
- It is, however, possible that both Tuna and Carp provide their own Tuna::Swim() and Carp::Swim() methods to make Tuna and Carp different swimmers.
- If a user with an **instance** of Tuna uses the **base class type** to invoke Fish::Swim(), he ends up executing only the **generic part** Fish::Swim() and **not** Tuna::Swim(), even though that **base class instance** Fish is a part of a Tuna.
- **Listing 11.1** Invoking Methods Using an Instance of the Base Class Fish That Belongs to a Tuna
- Code shows if we cast it back to a Fish, Tuna behaves like a generic Fish and not a Tuna.
- What the user would **ideally expect** is that an **object** of **type** Tuna behaves like a tuna **even if** the methods invoked is Fish::Swim().
- We use **virtual functions** to fix this.

11.1.2 Polymorphic Behavior Implemented Using Virtual Functions

- You have access to an **object** of **type** `Fish`, via **pointer** `Fish*` or **reference** `Fish&`.
This **object** could have been **instantiated** solely as `Fish` or be part of a `Tuna` or `Carp` that **inherits** from `Fish`.
- You don't know (or don't give a fuck)
- You invoke **method** `Swim()` using the **pointer** or **reference** like this:

```
pFish->Swim();  
myFish.Swim();
```

- What you expect is that the **object** `Fish` swims
 - as a `Tuna` if it is part of a `Tuna`,
 - as a `Carp` if it is part of a `Carp`,
 - or an anonymous `Fish` if it wasn't instantiated as part of a specialized class such as `Tuna` or `Carp`.
- You can ensure this by **declaring function** `Swim()` in the **base class** as a *virtual function*

```
class Base  
{  
    virtual ReturnType FunctionName (Parameter List);  
};  
  
class Derived  
{  
    ReturnType FunctionName (Parameter List);  
};
```

- Use of the keyword **virtual** means that the **compiler** ensures that any **overriding variant** of the requested **base class method** is invoked.
- Thus, if `Swim()` is declared **virtual**, invoking `myFish.Swim()` (`myFish` being of **type** `Fish&`) results in `Tuna::Swim()` being executed (Listing 11.2)
- This is *polymorphism*: treating different fishes as a common type `Fish`, yet **ensuring** that the right implementation of `Swim()` supplied by the **derived types** is executed

11.1.3 Need For Virtual Destructors

- What happens when a function calls operator `delete` using a **pointer** of type `Base*` that actually points to the **instance of type Derived**?
- See Listing 11.3 A function that invokes operator 'delete' on `Base*`
- Note in the listing: while Tuna and Fish were both **constructed** on the **free store (heap)** due to `new`, the **destructor** of Tuna **was not invoked** during `delete`, a rather only that of the Fish.
- This is in **stark contrast** to the **construction** and **destruction** of **local member** `myDinner` where **all** the **constructors** and **destructors** are invoked.
- Clearly, for the **free store** we have amiss. `~Tuna()` needs to be invoked.

```
Allocating a Tuna on the free store:
Constructed Fish
Constructed Tuna
Deleteing the Tuna:
Destructed Fish
Instantiating a Tuna on the stack:
Constructed Fish
Constructed Tuna
Automatic destruction as it goes out of scope:
Destructed Tuna
Destructed Fish
```

- The flaw means that the **destructor** of a **deriving class** that has been instantiated on the **free store** using `new` would **not** be invoked if `delete` is called using a **pointer of type `Base*`**.
- This can result in resources not being released, memory leaks, and so on and is a problem that is not to be taken lightly.

- Listing 11.4 Using virtual destructors to ensure that destructors in derived classes are invoked when deleting a pointer of Type `Base*`

```
Allocating a Tuna on the free store:
Constructed Fish
Constructed Tuna
Deleteing the Tuna:
Destructed Tuna
Destructed Fish
Instantiating a Tuna on the stack:
Constructed Fish
Constructed Tuna
Automatic destruction as it goes out of scope:
Destructed Tuna
Destructed Fish
```

- Always declare the base class destructor as 'virtual'

```
class Base
{
public:
    virtual ~Base() {}; // virtual destructor
};
```

- This ensures that one with a **pointer** `Base*` cannot invoke `delete` in a way that instances of the **deriving classes** are not correctly destroyed.

11.1.4 How do Virtual Functions Work? — The Virtual Function Table (vtable)

- A **virtual function** allows **runtime polymorphism**. When a member function is declared with the **virtual** keyword, the function that is executed is determined by the **actual type of the object** rather than the type of the pointer or reference.
- To support this behavior, most C++ compilers implement a hidden data structure called the **Virtual Function Table (vtable)**. Although the C++ standard does **not** require this implementation, it is the method used by virtually all modern compilers.
- The **vtable** is a table of function pointers. Each class that contains one or more virtual functions has its own vtable.
- Every object of a class containing virtual functions secretly stores a hidden pointer called the **vptr** (virtual pointer). This pointer points to the class's vtable.
- During object construction, the compiler automatically initializes the object's **vptr** so that it points to the correct vtable for that object's type.
- When a virtual function is called through a pointer or reference:
 1. The program follows the object's hidden **vptr**.
 2. The **vptr** points to the appropriate vtable.
 3. The corresponding function pointer is retrieved from the table.
 4. The function pointed to is invoked.
- This process is known as **dynamic dispatch** or **late binding**, since the decision of which function to execute is made at **runtime** instead of compile time.
- Without virtual functions, C++ performs **static binding** (also called **early binding**), meaning the compiler decides which function to call solely from the pointer or reference type.
- Consider the following example:

```
class Fish
{
public:
    virtual void Swim()
    {
        cout << "Fish swims\n";
    }
};

class Tuna : public Fish
{
public:
    void Swim() override
    {
        cout << "Tuna swims fast\n";
    }
};

Fish* pFish = new Tuna;
pFish->Swim();
```

- Even though **pFish** is a **Fish***, the object it points to is actually a **Tuna**. Since **Swim()** is virtual, the compiler uses the object's **vptr** to locate **Tuna**'s implementation in the vtable, producing:

```
Tuna swims fast
```


- If `virtual` were omitted, the compiler would instead call:

```
Fish swims
```

because it would only consider the pointer's declared type (`Fish*`).

- Conceptually, the compiler generates structures similar to the following (greatly simplified):

```
Fish vtable
```

```
+-----+
| &Fish::Swim      |
| &Fish::~~Fish    |
+-----+
```

```
Tuna vtable
```

```
+-----+
| &Tuna::Swim      |
| &Tuna::~~Tuna    |
+-----+
```

- Each object contains a hidden pointer:

```
Fish object
```

```
+-----+
| vptr -----+ |
+-----+---+
| data members | |
+-----+   |
              |
              v
          Fish vtable
          +-----+
          | Fish::Swim      |
          | Fish::~~Fish    |
          +-----+
```

- If the object is actually a `Tuna`, its hidden `vptr` instead points to `Tuna`'s vtable:

```
Tuna object
```

```
+-----+
| vptr -----+ |
+-----+---+
| Fish members  | |
| Tuna members  | |
+-----+   |
              |
              v
          Tuna vtable
          +-----+
          | Tuna::Swim      |
          | Tuna::~~Tuna    |
          +-----+
```

- Since the `vptr` belongs to the object itself, the correct virtual functions are called regardless of whether the object is accessed through a base-class pointer, base-class reference, or directly.

- Virtual destructors also use the vtable. When deleting an object through a base-class pointer:

```
Fish* pFish = new Tuna;  
delete pFish;
```

the virtual destructor ensures the call sequence is:

```
Tuna::~~Tuna()  
Fish::~~Fish()
```

If the destructor were not virtual, only `Fish:: Fish()` would be called, resulting in **undefined behavior** and potentially leaking resources owned by the derived class.

- Although virtual functions introduce a very small runtime overhead (one extra pointer lookup and one hidden pointer per object), this cost is generally negligible compared to the flexibility provided by runtime polymorphism.
- **Summary:**
 - Each polymorphic class has one vtable.
 - Each object stores one hidden `vptr`.
 - Virtual function calls use the `vptr` to locate the correct function in the vtable.
 - Function selection occurs at runtime (dynamic dispatch).
 - Virtual destructors ensure complete destruction of derived objects when deleting through base-class pointers.

11.2 Abstract Base Classes and Pure Virtual Functions

- A base class that cannot be instantiated is called an `abstract base class`.
- Such base class fulfills only one purpose, that of being derived from.
- C++ allows you to create an `abstract base class` using `pure virtual functions`
- A `virtual method` is said to be `pure virtual` when it has a **declaration** as shown in the following:

```
class AbstractBase
{
public:
    virtual void DoSomething() = 0; // pure virtual method
};
```

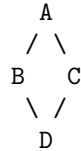
- This **declaration** essentially tells the **compiler** that `DoSomething()` needs to be implemented and by **class that derives** from `AbstractBase`:

```
class Derived : public AbstractBase
{
public:
    void DoSomething()
    {
        cout << "Implemented virtual function" << endl;
    }
};
```

- Thus, what the class `AbstractBase` has done is that it has enforced class `Derived` to **supply an implementation** for **virtual method** `DoSomething()`.
- This functionality where a **base class** can **enforce** support of methods with a **specified name and signature** in **classes** that derive from it is that of an `interface`.
- So, making class `Fish` an `abstract base class` with `Swim()` as a `pure virtual function` ensures that `Tuna` that **derives** from `Fish` implements `Tuna::Swim()` and swims like a `Tuna` and not like just any `Fish`.
- `Listing 11.6` class `Fish` as an Abstract Base Class for `Tuna` and `Carp`

11.3 Using 'virtual' Inheritance to Solve the Diamond Problem

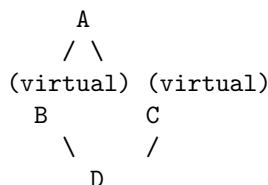
- The **diamond problem** occurs in **multiple inheritance** when a class inherits from two classes that both inherit from the same base class.
- This creates a diamond-shaped inheritance structure:



- In this structure:
 - Class A is the base class.
 - Classes B and C both inherit from A.
 - Class D inherits from both B and C.
- The problem arises because D ends up containing **two copies of A**:
 - One inherited through B
 - One inherited through C
- This leads to ambiguity. For example, if A has a member variable or function, calling it from D becomes unclear:

```
D obj;  
obj.A::function(); // ambiguous: which A?
```

- It also wastes memory since two separate copies of A's data exist inside D.
- C++ solves this problem using **virtual inheritance**.
- When B and C virtually inherit from A, they share a single common instance of A inside D.



- This is written in code as:

```
class A  
{  
public:  
    void foo() {}  
};  
  
class B : virtual public A {};  
class C : virtual public A {};  
  
class D : public B, public C {};
```

- Now, D contains only **one shared instance of A**, eliminating duplication and ambiguity.
- Without virtual inheritance, the compiler cannot decide which A subobject to use when accessing members through D, resulting in a compilation error.

- With **virtual inheritance**, the **most derived class** (D) is **responsible** for constructing the shared base class A.

- Example:

```
class A
{
public:
    int value;
};

class B : virtual public A {};
class C : virtual public A {};

class D : public B, public C {};

int main()
{
    D obj;
    obj.value = 10;  // OK: only one A exists
}
```

- **Key idea:**

- Virtual inheritance ensures that only one copy of a shared base class exists in the final derived object.
- It resolves ambiguity and prevents duplicate state.

- **Tradeoff:**

- Virtual inheritance introduces a small runtime and memory overhead because the shared base class is accessed indirectly.
- However, it is necessary when designing complex inheritance hierarchies.

- **Listing 11.8** Demonstrating How 'virtual' keyword in Inheritance Heirarchy helps restrict the number of instances of base class Animal to One

- In C++, `public virtual Animal` and `virtual public Animal` are **equivalent**. The order of the specifiers does not affect the meaning.
- Both forms indicate:
 - **public inheritance** (controls access level)
 - **virtual inheritance** (ensures a single shared base class instance in diamond inheritance)
- The C++ grammar allows **multiple valid orders of inheritance specifiers** for flexibility, so both are accepted by the compiler.
- There is no difference in behavior or memory layout between the two forms.
- **Style convention:** many modern C++ guidelines prefer `public virtual Base` because it reads as “publicly inherits from Base” with virtual inheritance as an additional detail.

```
class B : public virtual A {};    // ok
class B : virtual public A {};    // ok
class B : protected virtual A {}; // ok
class B : virtual protected A {}; // ok
class B : private virtual A {};   // ok
class B : virtual private A {};   // ok
```

11.4 Specifier 'override' to Indicate Intention to Override

- The `override` specifier in C++ is used to explicitly indicate that a member function is intended to **override a virtual function** from a base class.
- It improves code safety by letting the compiler verify that:
 - The base class actually has a matching virtual function.
 - The function signature in the derived class correctly matches the base class version.
- If there is no matching virtual function in the base class, the compiler will produce an error.
- This helps prevent subtle bugs caused by accidental mismatches in function signatures.
- Example without `override` (bug-prone):

```
class Animal
{
public:
    virtual void speak()
    {
        cout << "Animal sound\n";
    }
};

class Dog : public Animal
{
public:
    void speak(int x)    // does NOT override Animal::speak()
    {
        cout << "Bark\n";
    }
};
```

- In this case, `Dog::speak(int)` is a completely new function, not an override, because the signature does not match.
- Using `override` catches this mistake:

```
class Dog : public Animal
{
public:
    void speak(int x) override    // compiler error
    {
        cout << "Bark\n";
    }
};
```

- The compiler will reject this because there is no matching virtual function in the base class with the same signature.
- Correct usage:

```
class Dog : public Animal
{
public:
    void speak() override
    {
        cout << "Bark\n";
    }
};
```

- **Benefits of override:**
 - Prevents accidental signature mismatches.
 - Improves code readability by making intent explicit.
 - Helps the compiler catch polymorphism errors early.
- **Key idea:** `override` does not change runtime behavior — it is purely a **compile-time safety check**.

11.5 Using 'final' to Prevent Function Overriding

- The `final` specifier in C++ is used to prevent further overriding of a **virtual function** or prevent inheritance from a **class**.
- When applied to a virtual function, `final` ensures that no derived class can override that function.
- Example:

```
class Animal
{
public:
    virtual void speak() final
    {
        cout << "Animal sound\n";
    }
};
```

- In this case, any attempt to override `speak()` in a derived class will result in a compiler error:

```
class Dog : public Animal
{
public:
    void speak() override    // error: cannot override final function
    {
        cout << "Bark\n";
    }
};
```

- **Key idea:** marking a function as `final` "locks" its implementation in the inheritance hierarchy.
- `final` can also be applied to an entire class to prevent inheritance altogether:

```
class Animal final
{
public:
    void speak()
    {
        cout << "Animal sound\n";
    }
};
```

- In this case, any attempt to derive from the class is illegal:

```
class Dog : public Animal    // error: cannot inherit from final class
{
};
```

- **Benefits of final:**
 - Prevents unintended modification of critical behavior.
 - Allows the compiler to optimize virtual dispatch in some cases.
 - Clearly communicates design intent to other developers.
- **Key idea:** `final` is a compile-time restriction that strengthens class and interface design by limiting extensibility.

11.6 Virtual Copy Constructors?

- The question mark here is because it is technically impossible in C++ to have **virtual copy constructors**.
 - **Virtual copy constructors are not possible** because the concept of **virtual functions** in C++ relies on **runtime polymorphism**, where a function call is resolved based on the **dynamic type of an already-constructed object**.
 - A constructor, however, is fundamentally different: it is responsible for **creating an object from scratch**. At the time a constructor runs, there is **no fully constructed object yet**, and therefore no vtable-based dispatch can occur.
 - In other words, **virtual functions** require an existing object with a **vptr** (virtual table pointer), but during construction the object is still being built, so polymorphic dispatch cannot apply.
 - Additionally, constructors are **not inherited in the same way as normal member functions** and are always called based on the **static type being constructed**, not the dynamic type.
 - **Copy constructors specifically** must create a new object of a known type, so their signature and target type are fixed at compile time. There is no meaningful way to "choose" a derived constructor at runtime.
 - For this reason, C++ does not allow constructors (including copy constructors) to be declared as **virtual**.
 - **Key idea:** virtual dispatch requires an existing object and runtime type information, while construction is a compile-time process that produces the object in the first place. These two mechanisms are fundamentally incompatible.
-
- Having said that, there is a nice workaround in the form of defining your own clone function that allows you to do just that:

```
class Fish
{
public:
    virtual Fish* Clone() const = 0; // pure virtual function
};

class Tuna : public Fish
{
// ... other members
public:
    Tuna * Clone() const // virtual clone function
    {
        return new Tuna(*this); // return new Tuna that is a copy of this
    }
};
```

- Thus, **virtual function Clone** is a *smulated virtual copy constructor* that needs to be explicitly invoked, as shown in **Listing 11.9**

11.7 QA

11.8 Quiz

12 Operator Types and Operator Overloading

- In addition to encapsulating data and methods, classes can also encapsulate **operators** that make it easy to operate on **instances** of this class.
- You can use these **operators** to perform operations such as **assignment** or **addition** on **class objects** similar to those on integers that you saw in Lesson 5.
- Just like **functions**, **operators** can also be overloaded.

12.1 What are Operators in C++?

- On a syntactical level, there is very little that differentiates an **operator** from a **function**, save for the use of the keyword **operator**.
- Here is an **operator declaration**:

```
return_type operator operator_symbol (...parameter list...);
```

- The **operator_symbol** in this case could be any of the several **operator types** that the programmer can define.
- It could be
 - + (addition)
 - && (logical AND)
 - and so on...
- The **operands** help the compiler distinguish **one operator** from **another**.
- So, why does C++ provide operators when functions are supported?

- Consider a utility class **Date** that encapsulates the day, month, and year:

```
Date holiday(12, 25, 2016); // Dec 26, 2016
```

- Assuming that you want to **add a day** and get the **instance** to contain the next day — Dec 26, 2016 — which of the following two options would be more intuitive?

1. Option 1 (using the **increment operator ++**):

```
++holiday;
```

2. Option 2 (using a **member function Increment()**):

```
holiday.Increment(); // Dec 26, 2016
```

- Clearly, **Option 1** scores over method **Increment()**
- The **operator-based mechanism** facilitates consumption by supplying ease of use and intuitiveness.

- Implementing **operator <** in class `Date` would help you compare **two instances** of class `Date` like this:

```
if (date1 < date2)
    do something;
else
    do something else;
```

- **Operators** can be used in more situations than just **classes** that manage dates.
- An **addition operator (+)** in a **string utility class** such as `MyString` introduced in Listing 9.9 would facilitate easy **concatentation**:

```
MyString sayHello("Hello ");
MyString sayWorld(" world");
MyString sumThem (sayHello + sayWorld);    // if operator+ were supported by MyString
```

- The effort in implementing relevant operators will be rewarded by the ease of consumption of the class.
- On a broad level, **operators** in C++ can be classified into two types:

1. *Unary operators*
2. *Binary Operators*

12.2 Unary Operators

- Operators that function on a **single operand** are called *unary operators*.
- A **unary operator** that is implemented in the **global namespace** or as a **static member function** of a class uses the following structure:

```
return_type operator operator_type (parameter type)
{
    // ... implementation
}
```

- A **unary operator** that is a **(non-static)** member of a class has a similar structure but is lacking in parameters, b/c the **single parameter** that it works upon is the **instance of the class itself (*this)**:

```
return_type operator operator_type ()
{
    // ... implementation
}
```


12.2.1 Types of Unary Operators

Operator	Name
++	Increment
--	Decrement
*	Pointer dereference
->	Member selection
!	Logical NOT
&	Address-of
-	One's complement
+	Unary plus
-	Unary negation
Conversion operators	Conversion into other types

12.2.2 Programming a Unary Increment/Decrement Operator

- A unary prefix increment operator (++) is the following syntax **within the class declaration**:

```
Date& operator ++ (int)
{
    // store a copy of the current state of the object before incrementing day
    Date copy (*this);

    // increment implementation code

    // return state before increment (because, postfix)
    return copy;
}
```

- The **prefix** and **postfix decrement operators** have a similar syntax as the **increment operators**, just that the **declaration** would contain a `--` where you see a `++`.
- Listing 12.1 Allowing Incrementing and Decrementing Days
- The **postfix** differs from **prefix** operator in **return-type** and **parameters**.

12.2.3 Programming Conversion Operators

- doing `cout << holiday;` will produce an error in absence of conversion operator

```
error: binary '<<' : no operator found which takes a right-hand operand of type 'Date'
(or there is no acceptable conversion)
```

- `cout` didn't accept the type of the instance of the class.
- We know that `cout` can work well with a `const char*`:

```
std::cout << "Hello world"; // const char* works!
```

- So, getting `cout` to work with an **instance** of type `Date` might be as simple as adding an **operator** that returns a `const char*` version:

```
operator const char*()
{
    // operator implementation that returns a char*
}
```

- **Listing 12.2** implementing conversion operator `const char*` for class `Date`
- The **operator** opens up new possibilities toward consuming class `Date`. It allows you to even assign an **instance** of a `Date` directly to a **string**.

- Note that such assignments cause **implicit conversions**, that is, the compiler has used the available **conversion operator** (in this case `const char*`) thereby permitting unintended assignments that get compiled without error.

- To **avoid implicit conversions**, use the keyword `explicit` at the beginning of an **operator declaration**, as follows:

```
explicit operator const char*)_
{
    // conversion code here
}
```

- Using `explicit` would force the programmer to assert his intention to convert using a **cast**:

```
// explicit
string strHoliday(static_cast<const char*>(Holiday));
strHoliday = static_cast<const char*>(Date(11, 11, 2016));

// implicit
string strHoliday (holiday);
strHoliday = Date(11, 11, 2016);
```

- Casting, including `static_cast` is discussed in Lesson 13

- Program as many **operators** as you think your class would be used with.

If your application needs an integer representation of a **Date**, then you may program it as follows:

```
explicit operator int()
{
    return day + month + year;
}
```

- This would allow an **instance** of **Date** to be used or transacted as an **integer**:

```
FuncTakesInt(static_cast<int>(Date(12, 25, 2016)));
```

- **Listing 12.8** later on also demonstrates *conversion operators* used with a **string class**.

12.2.4 Programming Dereference Operator (*) and Member Selection Operator (→)

- The `dereference operator (*)` and `member selection operator (→)` are most frequently used in the programming of `smart pointer classes`.
- `Smart pointers` are **utility classes** that wrap regular pointers and simplify memory management by resolving **ownership** and **copy issues** using **operators**. (Lesson 26)
- This lesson takes a brief look at how **overloading operators** helps in making **smart pointers** work.
- Analyze the use of the `std::unique_ptr` in `Listing 12.3` and understand how it uses **operator (*)** and **operator (→)** to help you use the **smart pointer class** like any normal pointer.
- This example demonstrates how a `smart pointer` allows you to use **normal pointer syntax** as shown.
- It's like the variables are just the regular `int*` and `Date*`.
- The `secret lies` in the **pointer class** `std::unique_ptr` that is smart b/c it implements **operator (*)** and **operator (→)**.
- Smart pointers can do much more like de-allocate memory as they go out of scope.

12.3 Binary Operators

- Operators that function on **two operands** are called *binary operators*.
- The **definition** of a **binary operator** implemented as a **global function** or a **static member function** is the following:

```
return_type operator_type (parameter1, parameter2);
```

- The **definition** of a **binary operator** implemented as a **class member** is

```
return_type operator_type (paramater);
```

- The reason the **class member version** of a **binary operator** accepts **only one parameter** is that the **second parameter** is usually derived from the attributes of the class itself.

12.3.1 Type of Binary Operators

- Here is a table of **binary operators** that can be **overloaded** or **redefined**:

Operator	Name
,	Comma
!=	Inequality
%	Modulus
%=	Modulus/Assignment
&	Bitwise AND
&&	Logical AND
&=	Bitwise AND/assignment
*	Multiplication
*=	Multiplication/assignment
+	Addition
+=	Addition/assignment
-	Subtraction
-=	Subtraction/assignment
->*	Pointer-to-member selection
/	Division
/=	Division/assignment
<	Less than
<<	Left shift
<<=	Left shift/assignment
<=	Less than or equal to
=	Assignment, Copy Assignment and Move Assignment
==	Equality
>	Greater than
>=	Greater than or equal to
>>	Right shift
>>=	Right shift/assignment
^	Exclusive OR
^=	Exclusive OR/assignment
	Bitwise inclusive OR
=	Bitwise inclusive OR/assignment
	Logical OR
[]	Subscript operator

12.3.2 Programming Binary Addition (a + b) and Subtraction (a - b) Operators

- Similar to the **increment/decrement operators**, the **binary plus** and **minus**, when defined, enable you to add or subtract the value of a supported data type from an **object** of the class that implements these operators.
- Say for our class `Date` object we want to increase the date by say 5 days instead of just 1 (increment previously).

Listing 12.4 Binary Addition Operator
--

- Note: now we have the **binary operator** mechanism available to us we can make use of adding strings/instances together that are of class `MyString` type:

```
MyString Hello("Hello ");
MyString World(" World");
MyString HelloWorld(Hello + World);
```

- We do so by defining operator (+)

```
MyString operator+ (const MyString& addThis)
{
    MyString newString;

    if (addThis.buffer != NULL)
    {
        newString.bugger = new char[GetLength() + strlen(addThis.buffer) + 1];
        strcpy(newString.buffer, buffer);
        strcat(newString.buffer, addThis.buffer);
    }

    return newString;
}
```


12.3.3 Implementing Addition Assignment (+=) and Subtraction Assignment (-=) Operators

- The **addition assignment operators** allows syntax such as "a += b;" that allows the programmer to increment the value of an **object a** by an amount **b**.
- In doing this, the utility of the **addition assignment operator** is that it can be **overloaded** to accept different types of parameter **b**.
- **Listing 12.5** that follows allows you to add an **integer value** to a **Date object**.

- Your **class Date** now allows users to **add** or **remove** days from it as if they are dealing with **integers** using **addition** or **subtraction assignment operators** that take an **int** as a parameter.
- You can even provide **overloaded versions** of the **addition assignment operator** (+=) that work with an **instance** of a **fictitious class Days**:

```
void operator+= (const Days& daysToAdd)
{
    day += daysToAdd.GetDays();
}
```

The

- multiplication assignment `*=`
- division assignment `/=`
- modulus assignment `%=`
- subtraction assignment `-=`
- left-shift assignment `<<=`
- right-shift assignment `>>=`
- XOR assignment `^=`
- bitwise inclusive OR assignment `|=`
- bitwise AND assignment `&=`

have a similar syntax to the **addition assignment operator** shown in **Listing 12.5**

12.3.4 Overloading Equality (==) and Inequality (!=) Operators

- What do you expect when the user of class `Date` compares one **instance** to another:

```
if (date1 == date2)
{
    // do something
}
else {}
```

- In the **absence of an equality operator** `==`, the compiler simply performs a **binary comparison** of the **two objects** and returns **true** when they are **exactly identical**.
- A binary comparison checks whether the **raw bytes** of the two objects are identical in memory.
- This is different from **logical equality**: two objects can represent the same value but still have different byte representations (e.g., if they contain pointers or padding bytes).
- For a simple class such as `Date` containing only `int` members, two objects with the same values will typically have the same byte representation and therefore compare as identical. However that isn't always the case for other objects.
- Now if you don't even supply the **equality operator** `==` and you compare two objects, then you will get a compiler error (for modern c++ compilers) just like all the other operators on objects.

- So, This **binary comparison** will work for instances of classes containing simple data types (like the `Date` class as of now), but it will **not work** if the class in question has a **non-static** string member (`char*`), such as `MyString` in **Listing 9.9**.
- When **two instances** of class `MyString` are compared, a **binary comparison** of the **member attributes** would actually compare the **member string pointer values** (`MyString::buffer`).
- These would **not be equal** even when the **strings are identical in content**.
- Comparisons involving **two instances** of `MyString` would return **false** consistently.
- We solve this by **defining comparison operators** like so:

```
bool operator== (const ClassType& compareTo)
{
    // comparison code here, return true if equal else false
}
```

- The **inequality operator** can reuse that structure:

```
bool operator!= (const ClassType& compareTo)
{
    // comparison code here, return true if not equal else false
}
```

- The **inequality operator** can be the **inverse (logical NOT)** of the result of the **equality operator** (obviously)
- **Listing 12.6 Operators == and !=**

12.3.5 Overloading <, >, <=, and >= Operators

- Similarly from **Listing 12.6**, we can change this to the <, >, <=, and >= operators for comparing objects.
- We can do those conditional checks respectively akin to:

```
if (date1 < date2) { // ... }  
    or  
if (date1 <= date2) { // ... }  
    or  
if (date1 > date2) { // ... }  
    or  
if (date1 >= date2) { // ... }
```

- **Listing 12.7** Overloading <, >, <=, and >= Operators

12.3.6 Overloading Copy Assignment Operator (=)

- There are times when you want to **assign the contents** of an **instance** of a class to another like this:

```
Date holiday(12, 25, 2016);
Date anotherHoliday(1, 1, 2017);
anotherHoliday = holiday; // uses copy assignment operator
```

- The **assignment** invokes the **default copy assignment operator** that the compiler has built in to your class when you have **not supplied one**.
- Depending on the nature of the class, the **default copy assignment operator** might be inadequate, especially if your class is managing a resource that will not be copied.
- Similar problem to the **default copy constructor** and not actually copying its own resource.
- So, to ensure **deep copies** as with the **copy constructor**, you need to specify an accompanying **copy assignment operator**:

```
ClassType& operator= (const ClassType& copySource)
{
    if (this != &copySource) // protection against copy into self
    {
        // copy assignment operator implementation
    }
    return *this;
}
```

- **Deep copies** are important if your class **encapsulates a raw pointer**, such as class `MyString` shown in **Listing 9.9**.
- To ensure deep copy during assignments, define a **copy assignment operator** as shown in **Listing 12.8**
- **Listing 12.8** A Better 'class `MyString`' from **Listing 9.9** with a Copy Assignment Operator =

- Notice we did a `type conversion` (next lesson), which means we can go from `MyString` → `const char*` → output like so:

```
class MyString
{
    char buffer[100];

public:
    operator const char*()
    {
        return buffer;
    }
};
```

// we can do

```
MyString str;
printf("%s", str);
```

// Even though printf expects a const char*, C++ automatically calls:

```
str.operator const char*()
```

- The above `operator const char*` replaced the `GetString()` from before and we used this operator when doing

```
cout << string1 << string2 << endl;
```

- The **copy assignment operator** is similar **in function** to a **copy constructor** and performs a starting check to ensure that the same object is not both the copy source and destination.
- Look at it for more important details.

- To create a class that **cannot be copied**, declare the **copy constructor** and **copy assignment operator** private.

Declaration as private without implementation is **sufficient** for the compiler to **throw error** on all attempts at **copying this class** via passing to a **function by value** or **assigning one instance** into another.

12.3.7 Subscript Operator ([])

- The **operator** that allows **array-style []** access to a **class** is called **subscript operator**.
- The typical syntax of a **subscript operator** is:

```
return_type& operator [] (subscript_type& subscript);
```

- So when creating a class such as `MyString` that encapsulates a **dynamic array** class of characters in a `char*` buffer, a **subscript operator** makes it really easy to randomly access individual characters in the buffer:

```
class MyString
{
    // ... other members
public:
    /* const */ char& operator [] (int index) /* const */
    {
        // return the char at position index in buffer
    }
};
```

- **Listing 12.9** shows how the **subscript operator ([])** helps the user in iterating through the chars contained in an **instance** of `MyString` using **normal array semantics**.
- **Listing 12.9**

- Friendly recall:

```
// recall: const here tells us
// "This member function promises not to modify the object it belongs to."
int GetLength() const
{

}
```

- Using the keyword **const** is important when programming **operators**.

In **Listing 12.9** restricted the **return value** of **subscript operator []** to **const char&**

```
const char& operator[] (int index) const
{
    if (index < GetLength())
        return buffer[index];

    return buffer[0]; // failed
}
```

- The program works and compiles even without the **const** keywords, yet the reason you have it there is to avoid this code:

```
MyString sayHello("Hello World");
sayHello[2] = 'k'; // error: operator[] is const
```

- By using **const** you are protecting internal member `MyString::buffer` (so the rightmost **const**) from **direct modifications** from the outside via **operator []**
 - The **leftmost const** applies to the **return type**, making the returned object read-only.
 - The **rightmost const** applies to the **member function**, indicating that it does not modify the object (`*this`).
 - The two are **independent**: a function may have either, both, or neither, depending on its purpose.

```
char& operator[] (int index);           // neither
const char& operator[] (int index);    // left only
char& operator[] (int index) const;    // right only
const char& operator[] (int index) const; // both
```

- In addition to classifying the **return value** as **const**, you can also declare the **member function** as **const** (rightmost **const**) to ensure that it **cannot modify** the class's member attributes.

- When implementing `subscript operators`, you can improve on the version shown in **Listing 12.9**.
- The one is an implementation of a **single subscript operator** that works for both **reading from** and **writing to** the slots in the **dynamic array**.

- You can, however, implement **two subscript operators** — one as a **const** function and the other as a **non-const** one:

```
char& operator[] (int index); // use to write / change buffer at index
char& operator[] (int index) const; // used only for accessing char at index
```

- The compiler will invoke the **const function** for read operations and the **non-const** version for operations that write into the `MyString` object.
- Thus, you can (if you want to) have separate functionalities in the two **subscript operations**.

- There are other **binary operators** (listed in Table 12.2) that can be **redefined** or **overloaded**, but this book does not discuss them further, but its definition/logic is similar.

- Other **operators**, such as the `logical operators` and the `bitwiseop operators`, need to be programmed if the purpose of the class would be **enhanced** by having them.

12.4 Function Operator ()

- The **operator ()** that make **objects** behave like a **function** is called a ***function operator***.
 - They find application in the **standard template library (STL)** and are typically used in **STL algorithms**.
 - Their usage can include **making decisions**; such **function objects** are typically called ***unary*** or ***binary predicates***, depending on the **number of operands** they work on.
 - **Listing 12.10** A Function Object Created Using Operator ()
-
- Note how the compiler allows the use of object `displayFuncObj` as a **function** in Line 19 ish by **implicitly converting** what looks like a **function call** to a **call to `operator()`**.

12.5 Move Constructor and Move Assignment Operator for High Performance Programming

- The `move constructor` and the `move assignment operators` are performance optimization features that have become a part of the **standard** in C++11, ensuring that **temporary values** (`rvalues` that don't exist beyond the statement) are not wastefully copied.
- This is particularly useful when handling a **class** that manages a **dynamically allocated** resource, such as a **dynamic array class** or a **string class**.

12.5.1 The Problem of Unwanted Copy Steps

- Take a look at the `addition operator+` as implemented in **Listing 12.4**.

Notice that it actually **creates a copy** and **returns it**, the following lines of code would be valid examples of easy string concatenation:

```
MyString Hello("Hello ");
MyString World("World");
MyString CPP(" of C++");
MyString sayHello(Hello + World + CPP); // operator+, copy constructor
MyString sayHelloAgain("overwrite this");
sayHelloAgain = Hello + World + CPP; // operator+, copy constructor, copy assignment operator=
```

- This simple construct that makes concatenating three strings easy uses **binary addition operator+**:

```
MyString operator+ (const MyString& addThis)
{
    MyString newStr;

    if (addThis.buffer != NULL)
    {
        // copy into newStr
    }

    return newStr; \\ return copy by value, invoke copy constructor
}
```

- While making it easy to concatenate the strings, the `addition operator+` can cause performance problems
- The creation of `sayHello` requires the execution of the **addition operator** **twice**.
- Each execution of `operator+` results in the creation of a **temporary copy** as a `MyString` is returned **by value**, thus causing the execution of the **copy constructor**.
- The **copy constructor** executes a **deep copy** — to a **temporary value** that does **not exist after the expression**.
- Thus, the expression results in **temporary copies** (`rvalues`, for the purists) that are **not ever required after** the statement and hence are performance bottleneck forced by C++. Well, until recently.

Another explanation of the problem:

- The expression `Hello + World + CPP` is evaluated as `(Hello + World) + CPP`, so the addition operator `operator+` is called **twice**.
- Each call to `operator+` creates a **local** `MyString` object (e.g., `newStr`) to hold the concatenated result. Since it is returned **by value**, a temporary result is produced for use in the surrounding expression.
- Since `operator+` returns its result **by value**, the temporary object is copied (via the **copy constructor**) to the caller.
- The **copy constructor** performs a **deep copy**, allocating new memory and copying the string data, which can be expensive.
- These temporary objects exist only for the duration of the expression and are immediately destroyed afterwards, making the deep copies unnecessary work.
- Consequently, expressions involving repeated `operator+` calls can create multiple temporary objects and deep copies, leading to reduced performance.

Note

- The object `newStr` is a **local variable** and is destroyed when `operator+` returns.
- Since the function returns **by value**, the caller cannot receive the local object directly. Instead, a **new object** is created from `newStr` using the **copy constructor** (or the move constructor/copy elision in modern C++).

- This problem has now been finally resolved in C++11 in which the compiler specifically recognizes **temporaries** and uses `move constructors` and `move assignment operators`, where supplied by the **programmer**.

12.5.2 Declaring a Move Constructor and Move Assignment Operator

- The syntax of the `move constructor` is as follows:

```
class Sample
{
private:
    Type* ptrResource;

public:
    Sample(Sample&& moveSource) // move constructor, note &&
    {
        ptrResource = moveSource.ptrResource; // take ownership, start move
        moveSource.ptrResource = NULL;
    }

    Sample& operator= (Sample&& moveSource) // move assignment operator, note &&
    {
        if (this != &moveSource)
        {
            delete[] ptrResource; // free own resource
            ptrResource = moveSource.ptrResource; // take ownership, start move
            moveSource.ptrResource = NULL; // copy assignment
        }
    }

    Sample(); // default constructor
    Sample(const Sample& copySource); // copy constructor
    Sample& operator= (const Sample& copySource); // free move source of ownership
};
```

Quick recall on virtual methods (not for this move constructor/operator section)

Virtual Methods

- A **virtual method** is a member function in a base class that can be overridden by derived classes.
- When called through a base class pointer or reference, the correct derived class implementation is selected at **runtime**.

```
class Animal
{
public:
    virtual void Speak()
    {
        cout << "Animal sound\n";
    }
};

class Dog : public Animal
{
public:
    void Speak() override
    {
        cout << "Woof\n";
    }
};

int main()
{
    Animal* a = new Dog();
    a->Speak();    // Calls Dog::Speak()
}
```

Virtual Destructor

- A **virtual destructor** ensures that when a derived object is deleted through a base class pointer, the derived class destructor is called first.
- A base class intended for inheritance should usually have a virtual destructor.

```
class Animal
{
public:
    virtual ~Animal()
    {
        cout << "Animal destroyed\n";
    }
};

class Dog : public Animal
{
public:
    ~Dog()
    {
        cout << "Dog destroyed\n";
    }
};

int main()
```

```
{
    Animal* a = new Dog();
    delete a;
}
```

Output:

```
Dog destroyed
Animal destroyed
```

Virtual Constructor

- C++ does **not support virtual constructors**.
- Constructors create objects, and virtual dispatch requires an object to already exist.
- Instead, the **virtual constructor pattern** is implemented using virtual methods such as `clone()`.

```
class Shape
{
public:
    virtual Shape* clone() = 0;
};

class Circle : public Shape
{
public:
    Shape* clone() override
    {
        return new Circle(*this);
    }
};
```

Pure Virtual Functions and Abstract Classes

- A **pure virtual function** has no implementation in the base class and is declared using `= 0`.
- A class containing at least one pure virtual function is called an **abstract class**.
- Abstract classes cannot be instantiated directly.

```
class Shape
{
public:
    virtual double Area() = 0;
};

class Circle : public Shape
{
private:
    double radius;

public:
    Circle(double r) : radius(r) {}

    double Area() override
    {
        return 3.14 * radius * radius;
    }
};
```

This is invalid:


```
Shape s;    // ERROR
```

This is valid:

```
Shape* s = new Circle(5);  
cout << s->Area();
```

Summary of Virtual Features

Feature	Syntax	Purpose
Virtual method	<code>virtual void func();</code>	Allows derived override at runtime
Override	<code>void func() override;</code>	Explicitly replaces base method
Virtual destructor	<code>virtual ~Class();</code>	Ensures proper cleanup through base pointer
Virtual constructor	Not supported	Use <code>clone()</code> pattern instead
Pure virtual method	<code>virtual void func() = 0;</code>	Forces derived classes to implement
Abstract class	Contains pure virtual methods	Cannot create objects directly

Common Polymorphic Base Class Pattern

```
class Base  
{  
public:  
    virtual ~Base() {}  
    virtual void Function() = 0;  
};  
  
class Derived : public Base  
{  
public:  
    ~Derived() {}  
  
    void Function() override {}  
};
```

(back to move section)

- Thus, the **declaration** of the **move constructor** and **assignment operator** are different from the **regular copy constructor** and **copy assignment operator** in that the **input parameter** is a **type** `Sample&&`.
 - Additionally, as the **input parameter** is the **move-source**, it **cannot** be a `const` parameter as it is **modified**.
 - **Return values** remain the same, as these are **overloaded** versions of the **constructor** and the **assignment operator**, respectively.
-
- C++11 compliant compilers ensure that for **rvalue temporaries** the **move constructor** is used instead of the **copy constructor** and the **move assignment operator** is invoked instead of the **copy assignment operator**.
 - In your implementation of these two, you ensure that instead of **copying**, you are simply **moving** the resource from the **source** to the **destination**.
- **Listing 12.11** 'class `MyString`' with Move Constructor and Move Assignment Operator in addition to Copy Constructor and Copy Assignment Operator

- Note from **Listing 12.11** C++17 made `copy elision mandatory` in many return-by-value cases.

So, we only see as output:

```
Constructor called for: Hello
Constructor called for: World
Constructor called for:  of C++
Constructor called for: overwrite this
operator+ called:
Default constructor called
operator+ called:
Default constructor called
Move assignment op. moves: Hello World of C++
```

- Notice there is no Move constructor cout here.
- The reason the **move constructor** is not called is because C++17 performs `copy elision` in this situation. The compiler constructs the returned `MyString` object directly in its final destination instead of creating a temporary object and moving it.
- Therefore, the statement `return newStr;` does not require a copy or move constructor call. The local object `newStr` is optimized away, and its storage is used directly for the returned object.
- The **move assignment operator** is still called because the final temporary result of `Hello + World + CPP` must be assigned to the already existing object `sayHelloAgain`.

12.6 User Defined Literals

- **Literal constants** were introduced in Lesson 3. Here are some examples of a few:

```
int bankBalance = 10000;
double pi = 3.14;
char firstAlphabet = 'a';
const char* sayHello = "Hello!";
```

- In the preceding code, 10000, 3.14, 'a', and "Hello!" are all **literal constants**.
- C++11 extended literals by allowed you to define your own literals.
- Example, you want all your temperatures to be stored and operated using a scale called Kevin. You may now declare all your temperatures using a syntax similar to the following:

```
Temperature k1 = 32.15_F;
Temperature k2 = 0.0_C;
```

- Using **literals** `_F` and `_C` that **you have defined**, you have made your application a lot simpler to read and therefore maintain.

- To **define your own literal**, you **define operator ""** like this:

```
ReturnType operator"" YourLiteral(ValueType value)
{
    // conversion code here
}
```

- Depending on the nature of the **user defined literal**, the `ValueType` parameter would be restricted to one of the following:

- unsigned long int for **integer literal**
- long double for **floating point literal**
- char, wchar_t, char16_t, char32_t for **character literal**
- const char* for **raw string literal**
- const char* together with size_t for **string literal**
- wchar_t* together with size_t for **string literal**
- char16_t* together with size_t for **string literal**
- char32_t* together with size_t for **string literal**

- **Listing 12.12** shows a user defined literal that converts types.

12.7 Operators That Cannot Be Overloaded

- **Table 12.3** Operators that CANNOT be Overloaded or Redefined

Operator	Name
.	Member selection
.*	Pointer-to-member selection
::	Scope resolution
? :	Conditional ternary operator
sizeof	Gets the size of an object/class type

- **DO** program as many operators as would help making using your class easy, but not more.
- **DO** mark **conversion operators** as **explicit** to avoid **implicit conversions**.
- **DO** always program a **copy assignment operator** (with a **copy constructor** and **destructor**) for a class that contains **raw pointer members**.
- **DO** always program a **move assignment operator** (and **move constructor**) for classes that manage **dynamically allocated resources**, such as an array of data, when using a C++11-compliant compiler.
- **DON'T** forget that the compiler provides a **default copy assignment operator** and **copy constructor** if you don't supply these, and they **won't ensure deep copies** of any **raw pointers** contained within the class.
- **DON'T** forget that if you don't supply a **move assignment operator** or **move constructor**, the compiler **does not create these for you**, but instead falls back on the **regular copy assignment operator** and **copy constructor**.

13 Casting Operators

- **Casting** is the process of converting an object or value from one data type to another, changing how the compiler interprets and uses that value.
- **Operators** that change the **interpretation** of an object are called *casting operators*.

13.1 The Need for Casting

- Old programs and lots of modules working together inevitably call for the need to cast values.
- A common example is when using C library functions such as `malloc()`, `calloc()`, and `realloc()`, which return a `void*` pointer. Since `void*` has no associated data type, it must be cast to the appropriate pointer type before use in C++.

```
int* arr = (int*)malloc(10 * sizeof(int));

double* values = (double*)calloc(20, sizeof(double));

char* buffer = (char*)realloc(arr, 20 * sizeof(char));
```

- Although explicit casts are required in these cases, C++ programs should generally prefer `new` and `delete` over the C memory allocation functions, as they correctly invoke constructors and destructors for class objects.
- C++ devs don't like

```
int* intArray = (int*)staticStr;
```

because this would typically give an error compiled with a C++ compiler:

```
char* staticStr = "Hello World!";
int* intArray = staticStr; // error: cannot convert char* to int*
```

13.2 Examples of C++ Casts

- C++ provides four explicit cast operators:
 - `static_cast`,
 - `dynamic_cast`,
 - `const_cast`, and
 - `reinterpret_cast`.

Each is intended for a specific purpose.

1. `static_cast` Used for compile-time conversions between related types.

```
class Animal
{
public:
    virtual ~Animal() {}
};

class Dog : public Animal
{
public:
    void Bark()
    {
        cout << "Woof!" << endl;
    }
};

Animal* animal = new Dog();

Dog* dog = static_cast<Dog*>(animal);
dog->Bark();
```


2. `dynamic_cast` Used for safe runtime downcasting. Returns `nullptr` if the cast fails.

```
Animal* animal = new Dog();  
  
Dog* dog = dynamic_cast<Dog*>(animal);  
  
if (dog != nullptr)  
    dog->Bark();
```

3. `const_cast` Adds or removes the `const` qualifier.

```
const int value = 10;
```

```
int* ptr = const_cast<int*>(&value);
```

4. `reinterpret_cast` Reinterprets the bits of one pointer type as another.

```
int number = 65;
```

```
char* ptr = reinterpret_cast<char*>(&number);
```

```
cout << *ptr << endl;
```

- `static_cast` performs compile-time conversions between compatible types.
- `dynamic_cast` safely converts pointers or references within an inheritance hierarchy by checking the object's type at runtime.
- `const_cast` adds or removes `const` or `volatile` qualifiers.
- `reinterpret_cast` performs low-level reinterpretation of memory and should be used only when absolutely necessary.

13.3 The C++ Casting Operators

- Casting issues are the consequence of long history of code being developed by many different people and is just a natural consequence of not knowing the cast of something by another developer (like in the malloc case).
- But what is changed is again the **interpretation** of the data, which can be dangerous.
- C++ supplies a **casting operator** specific to **inheritance-based** scenarios (meaning mainly useful for inheritance-based scenarios) that did not exist with C programming.
- As seen, the 4 C++ **casting operators** are:
 1. `static_cast`
 2. `dynamic_cast`
 3. `reinterpret_cast`
 4. `const_cast`
- The usage syntax of the **casting operators** is consistent:

```
destination_type result = cast_operator<destination_type> (object_to_cast);
```

13.3.1 Using static_cast

- `static_cast` is a mechanism that can be used to **convert pointers** between related types, and perform **explicit type conversions** for standard data types that would otherwise happen automatically or **implicitly**.
- `static_cast` implements a basic compile-time check to ensure that the **pointer** is being **cast** to a **related type**.
- C-style casting allows a **pointer** to **one object** to be casted to an absolutely **unrelated type** without complaint by the compiler.

- Using a `static_cast`, a **pointer** can be **upcasted** to the **base type** or can be **downcasted** to the **derived type**:

```
Base* objBase = new Derived();
Derived* objDer = static_cast<Derived*>(objBase); // ok!

// class unrelated is not related to Base
Unrelated* notRelated = static_cast<Unrelated*>(objBase); // error
// The cast is not permitted as types are unrelated
```

- The following example demonstrates both an **upcast** and a **downcast**:

```
Derived* objDer = new Derived();

Base* objBase = objDer;
// upcast: Derived* -> Base*

Derived* objDer2 = static_cast<Derived*>(objBase);
// downcast: Base* -> Derived*
```

- An **upcast** converts a derived class pointer into a base class pointer. This is safe because every derived object is also a base object.
- A **downcast** converts a base class pointer into a derived class pointer. This is only safe if the object being pointed to is actually an instance of the derived class.
- A `static_cast` does not perform a runtime check during a downcast. If the object is not actually of the derived type, using the resulting pointer can cause undefined behavior. For safer runtime-checked downcasts, use `dynamic_cast` with polymorphic base classes.

- Casting a `Derived*` to a `Base*` is called *upcasting* and can be done without any **explicit casting operator**:

```
Derived objDerived;
Base* objBase = &objDerived; // ok!
```

- Casting a `Base*` to a `Derived*` is called *downcasting* and cannot be done without usage of **explicit casting operator**:

```
Derived objDerived;
Base* objBase = &objDerived; // upcast -> ok!!!
Derived* objDer = objBase; // error: downcast needs explicit cast
```

- However, note that `static_cast` verifies only that the **pointer types** are related. It does not perform any runtime checks.

- So, with `static_cast`, a programmer could still get away with this bug:

```
Base* objBase = new Base();
Derived* objDer = static_cast<Derived*>(objBase); // still no errors!
```

- Here, `objDer` actually **points** to a partial Derived object because the **object** being **pointed to** is actually a `Base()` type.
- Because `static_cast` performs **only a compile-time check** of verifying that the types in question **are related** and **does not** perform a **runtime check**, a call to `objDer->DerivedFunction()` would get compiled, but probably result in unexpected behavior during **runtime**.

- Apart from helping in **upcasting** or **downcasting**, `static_cast` can, in many cases, help make **implicit casts explicit** and bring them to the attention of the programmer or reader:

```
double Pi = 3.14;  
int num = static_cast<int>(Pi); // making an otherwise implicit cast, explicit
```

- This makes things explicit to the reader, and lower the likelihood of reader error from implicit conversions.
- You would also need to use `static_cast` when using **conversion operators** or **constructors** that have been declared using keyword `explicit`.
- **Avoiding implicit conversions** via keyword `explicit` was discussed in **Lesson 9** and **Lesson 12**.

13.3.2 Using `dynamic_cast` and Runtime Type Identification

- **`Dynamic_casting`** is the **opposite** of **`static_casting`** and executes the **cast** at **runtime**—that is, at **application execution time**.
- The result of a **`dynamic_cast` operation** can be checked to see whether the attempt at **casting succeeded**.
- Here is typical usage:

```
destination_type* Dest = dynamic_cast<class_type*>(Source);

if (Dest)    // Check for success of the casting operation
    Dest->CallFunc();
```

- For example:

```
Base* objBase = new Derived();

// perform a downcast
Derived* objDer = dynamic_cast<Derived*>(objBase);

if (objDer) // check for success of the cast
    objDer->CallDerivedFunction();
```

- As shown, given a **pointer** to a **base-class object**, we can resort to a **`dynamic_cast`** to **verify the type** of the **destination object** being **pointed to** before proceeding to use the pointer as such.
- Note that in the code snippet it is apparent that the **destination object** is a **Derived type**. So, the sample if of demonstrative value only.
- Yet, this is not always the case—when a **pointer of type `Derived*`** is passed to a function that accepts **type `Base*`**.
- The function can use **`dynamic_cast`** given a **base-class pointer type** to detected type and then perform operations specific to the types detected.
- Thus, **`dynamic_cast`** helps determine the **type** at **runtime** and use a **casted pointer** when it is safe to do so.
- Therefore, this mechanism of identifying the **type** of the **object** at **runtime** is called ***runtime identification (RTTI)***.
- **Listing 13.1** Using Dynamic Casting to Tell Whether a Fish Object is a Tuna or a Carp
- Note: the **return value** of a **`dynamic_cast` operation** should always be checked for **validity**.
- It is **NULL** when the **cast fails**.

13.3.3 Using reinterpret_cast

- `reinterpret_cast` is the **closest** a C++ **casting operator** gets to the C-style cast.
- It forces a **reinterpretation of type** using syntax like below (similar to the **unrelated casting** option C):

```
Base *objBase = new Base();
Unrelated* notRelated = reinterpret_cast<Unrelated*>(objBase);
// The code above compiles, but is not good programming!
```

- The **cast** actually **forces** the compiler to accept situations that `static_cast` would normally not permit.
- Sometimes used in cases like in drivers where data needs to be converted to a simple type than an API can accept—like some OS-level APIs require data to be sent as a **BYTE array**, that is, `unsigned char*`):

```
SomeClass* object = new SomeClass();
// Need to send the object as a byte-stream...
unsigned char* bytesFoAPI = reinterpret_cast<unsigned char*>(object);
```

- The **cast** used in the preceding code has not changed the binary representation of the **source object** and has effectively cheated the compiler into allowing the programmer to peek into **individual bytes** contained by an **object** of type `SomeClass`.
- B/c this comprises **type safety**, `reinterpret_cast` is a **last resort** in performing an otherwise unsafe (and nonportable) conversion.

13.3.4 Using `const_cast`

- `const_cast` enables you to add or remove the `const` qualifier from an object or pointer.
- It is primarily used when interfacing with legacy code or APIs that do not correctly use `const` qualifiers.
- Removing `const` is only safe if the original object was **not** declared as `const`. Modifying an object that was originally declared `const` results in **undefined behavior**.
- Example:

```
void Print(char* str)
{
    cout << str << endl;
}
```

```
const char* message = "Hello";
```

```
// Remove constness to call a legacy function
Print(const_cast<char*>(message));
```

- Since `Print()` does not modify the string, this use of `const_cast` is safe. However, if `Print()` attempted to modify the string, the program would have undefined behavior.

13.4 Problems with the C++ Casting Operators

- Not all C++ loves the casting operators given by C++: they can be cumbersome like example:

```
double Pi = 3.14;

// C++ style cast: static_cast
int num = static_cast<int>(Pi); // result: num is 3

// C-style cast
int num2 = (int)Pi; // result: num is 3

// leave casting to the compiler
int num3 = Pi; // result: num is 3
```

- In all three cases, the programmer achieved the same result.
- In practice the second option is most prevalent, followed by the third.
- The first makes things difficult to read.

- Another case is where C can handle its casting:

```
// using static_cast
Derived* objDer = static_cast<Derived*>(objBase);

// But, this works just as well...
Derived* objDerSimple = (Derived*)objBase;
```

- Thus, the adv of using `static_cast` is overshadowed by its syntax handle.
- Etc for the others, except the `dynamic_cast` is beneficial for modern C++ applications.
- Best practice is to **avoid casting** as far as possible.

- **DO** remember that casting a `Dervied*` to a `Base*` is called `upcasting` and this is safe.
 - **DO** remember that casting a `Base*` to a `Dervied*` is called `downcasting` and this is unsafe unless you use `dynamic_cast` and **check for success**.
 - **DO** remember that the objective of creating an **inheritance hierarchy** is typically having **virtual functions** that when invoked using **base class pointers** ensure that the avilable **derived class** versions are invoked (like the **virtual destructor** most importantly).
-
- **DON'T** forget to **check the pointer for validity** after using `dynamic_cast`.
 - **DON'T** design your application around `RTTI` using `dynamic_cast`.

14 An Introduction to Macros and Templates

14.1 The Preprocessor and the Compiler

- **Lesson 2** had introduced the `preprocessor`, which is what that runs **before** the compiler starts.
- In other words, the **preprocessor** actually decides what is compiled on the basis of how you instruct it.
- **Preprocessor directives** all start with the `#` sign:

```
// instruct preprocessor to insert contents of iostream here
#include <iostream>
```

```
// define a macro constant
#define ARRAY_LENGTH 25
int numbers[ARRAY_LENGTH]; // array of 25 integers
```

```
// define a macro function
#define SQUARE(x) ((x) * (x))
int TwentyFive = SQUARE(5);
```

- The lesson focuses on **two types** of **preprocessor directives**:
 - one using `#define` to define a **constant**
 - another using `#define` a **macro function**
- Both these **directives** tell the **preprocessor** to replace every instance of the **macro** (`ARRAY_LENGTH` or `SQUARE`) with the value they define (aka text substitute)

14.2 Using Macro `#define` to Define Constants

- The syntax of using `#define` to compose a **constant** is simple:

```
#define identifier value
```

```
// Example:
```

```
#define ARRAY_LENGTH 25
```

- So after the **preprocessor** runs we can see text substitutes like

```
int numbers [ARRAY_LENGTH] = {0};
```

```
// gets converted to
```

```
int numbers [25] = {0};
```

- I am not going over code for this trivial thing, but note that the **preprocessor** does not always check for **correctness** of the substitution (but the compiler always does).

14.2.1 Using Macros to Protection against Multiple Inclusion

- C/C++ programmers typically declare their classes and functions in `.h` files called *header files*.
- The respective functions are defined in *.cpp files* called *source files*, and they typically include the **header files** using the `#include <header>` **preprocessor directive**.
- A tricky part for any **circular dependency** of **header files** would entail **forward declarations** (it's best to not have circular dependency):

- A.h

```
#ifndef A_H
#define A_H

class B; // Forward declaration

class A {
public:
    B* b; // OK: pointer or reference
};

#endif
```

- B.h

```
#ifndef B_H
#define B_H

class A; // Forward declaration

class B {
public:
    A* a;
};

#endif
```

- A.cpp

```
#include "A.h"
#include "B.h"
```

- B.cpp

```
#include "B.h"
#include "A.h"
```


- So with many header files potentially including many others and having **multiple inclusions** (and not necessarily as the circular dependency case, but more and very likely in general), we use **macros** in conjunction with **preprocessor directives** **#ifndef** and **#endif**
- header1.h that includes <header2.h> looks like the following:

```
#ifndef HEADER1_H_ // multiple inclusion guard
#define HEADER1_H_ // preprocessor will read this and following lines once
#include <header2.h>

class Class1
{
    // class members
};
#endif // end of header1.h
```

- header2.h looks similar, but is flipped:

```
#ifndef HEADER2_H_ // multiple inclusion guard
#define HEADER2_H_ // preprocessor will read this and following lines once
#include <header1.h>

class Class2
{
    // class members
};
#endif // end of header2.h
```

- the **#ifndef** is read as "if-not-defined" and is a **preprocessing conditional command**, instructing the **preprocessor** to continue only if the identifier **has not been defined**.
- **#endif** marks the **end** of this **conditional processing instruction** for the **preprocessor**.
- It prevents the header file from being processed more than once during a single compilation, avoiding duplicate declarations/definitions and helping prevent errors caused by multiple inclusion.
- This mechanism is simply the **one of the most frequently used macro-based functionalities** in the world of C and C++ programming.

14.3 Using #define to Write Macro Functions

- The capability of the **preprocessor** to simply replace text elements identified by a **macro** often results it in being used to write **simple functions**, for example:

```
#define SQUARE(x) ((x) * (x))
```

```
// or
```

```
#define PI 3.1416
```

```
#define AREA_CIRCLE(r) (PI*(r)*(r))
```

- **Macro functions** are expanded **inline** before compilations and hence can help improve code performance in certain cases.
- | |
|---|
| Listing 14.2 Using Macro Functions |
|---|

14.3.1 Why All the Parentheses?

- For example:

```
#define AREA_CIRCLE(r) (PI*(r)*(r))
```

- The reason for the many parentheses is that a **macro** is expanded by the **preprocessor** using simple **text substitution**; it does not understand operator precedence.
- For example, if we call

```
AREA_CIRCLE(a + b)
```

then the expansion becomes

```
(PI*(a + b)*(a + b))
```

which correctly computes $\pi(a + b)^2$.

- Without the parentheses,

```
#define AREA_CIRCLE(r) PI*r*r
```

the same call expands to

```
PI*a + b*a + b
```

which is interpreted according to operator precedence and produces a completely different result.

- These brackets automatically result in the calculation of an accurate area, making your **macro code** independent of **operator precedence** and the effects thereof.

14.3.2 Using Macro 'assert' to Validate Expressions

- The `assert` macro enables you to **check that a condition is true at runtime**; if the condition evaluates to **false**, the program is terminated and a diagnostic message indicating the failed assertion is displayed.
- In large applications this is a decent way to test functionality.
- To use `assert` you include `<assert.h>` and the syntax is as follows:

```
assert (expression that evaluations to true or false);
```

- A sample use of `assert()` that validates the contents of a **pointer** is:

```
#include <assert.h>
int main()
{
    char* sayHello = new char[25];
    assert(sayHello != NULL); // throws a message if pointer is NULL;

    // other code

    delete[] sayHello;
    return 0;
}
```

- `assert()` ensures that you are notified if the **pointer** is **invalid**.
- This makes it a handy debugging feature to say check for validity of the parameters as well.
- This is **highly recommended** and helps you improve the quality of your code over the long term.

- `assert()` is typically **disabled** in `release modes` and provides you with an **error message** or information only in the `debug build` of most development environments.
- Additionally, some environments have implemented this as a **function**, not as a **macro**.

- An **assert** does not make it to the **release build**, it is important to ensure that checks are critical to the functioning of your application (for ex, the **return value** of a `dynamic_cast` operation) are still performed using an **if-statement**.
- an **assert** assists you in problem detection; it's not something to **replace pointer checks** in the code.

14.3.3 Adv and Disadv of Using Macro Functions

- **Advantages**

- **Eliminates function call overhead.** A macro is expanded by the **preprocessor** before compilation, so the generated code is inserted directly at each point of use rather than invoking a function at runtime.
- **May improve performance for small operations.** Since there is no function call, stack frame creation, or parameter passing, macro expansion can be more efficient for very simple operations.
- **Can operate on different data types.** Because a macro performs only **text substitution**, it is not restricted to a particular type (although this also introduces disadvantages).
- **Useful for compile-time constants and conditional compilation.** Macros are commonly used to define symbolic constants, enable or disable debugging code, and control platform-specific compilation using preprocessing directives.
- **Reduces repetitive code.** Frequently used expressions can be written once and reused throughout a program, improving consistency and maintainability when used appropriately.

- **Disadvantages**

- **No type checking.** The **preprocessor** simply performs text substitution and has no knowledge of C/C++ types, allowing type-related errors to pass unnoticed until compilation or execution.
- **Arguments may be evaluated multiple times.** A macro parameter appearing more than once in the macro body causes the corresponding argument to be substituted multiple times, potentially producing unintended side effects.

```
#define SQUARE(x) ((x) * (x))
```

```
SQUARE(i++)    // expands to ((i++) * (i++))
```

- **Operator precedence can produce incorrect results.** Since macro expansion is purely textual, failing to surround parameters and the entire expression with parentheses may lead to unexpected evaluations.

```
#define SQUARE(x) x*x
```

```
SQUARE(a + b)    // expands to a + b * a + b
```

- **More difficult to debug.** The compiler only sees the expanded code rather than the original macro invocation, making compiler errors and debugger output harder to interpret.
- **Code size may increase.** Every macro invocation expands into a copy of its replacement text, potentially increasing the size of the generated executable.
- **Lack of scope.** Unlike variables and functions, macros are processed before compilation and are not governed by C++ namespaces or normal scope rules, increasing the risk of name collisions.
- **Modern alternatives are usually preferable.** In modern C++, **inline functions**, **constexpr**, templates, and ‘const’ variables provide similar performance while offering type safety, proper scoping, and better debugging support.

Do's and Don't's

- **DO** program your own **macro functions** as infrequently as possible.
- **DO** use **const** variables where you can instead of **macros**
- **DO** remember that **macros** are not type safe and the **preprocessor** performs **no type checking**.

- **DON'T** forget to envelop every variable in a **macro function definition** with brackets
- **DON'T** forget to insert multiple **inclusion guards** using **#ifndef**, **#define**, and **#endif** in your **header files**.
- **DON'T** forget to sprinkle your code with **generous** number of **assert()** statements—these don't make it to the release version are good at improving **quality** of your code.

14.4 An Introduction to Templates

- You begin the **declaration** of a `template` using the keyword `template` followed by a **type parameter list**:

```
template <parameter list>
template function / class declaration...
```

- The keyword `template` marks the **start** of a **template declaration** and is followed by the **template parameter list**.
- This **parameter list** contains the keyword `typename` that **defines** the **template parameter** `objType`, making it a **placeholder** for the **type** of the **object** that the **template** is being instantiated for.

```
// a template function
template <typename T1, typename T2 = T1>
bool TemplateFunction(const T1& param1, const T2& param2);

// a template class
template <typename T1, typename T2 = T1>
class MyTemplate
{
private:
    T1 member1;
    T2 member2;

public:
    T1 GetObj1() { return member1; }
    // ... other members
};
```

- What you see is a `template function` and a `template class`, each taking **two template parameters** `T1` and `T2`, where `T2` has been given the **default type** as that of `T1`.

14.4.1 The Different Types of Template Declarations

- A `template declaration` can be
 - a declaration or definition of a function
 - a declaration or definition of a class
 - a definition of a member function or a member class of a class template
 - a definition of a static data member of a class template
 - a definition of a static data member of a class nested within a class template
 - a definition of a member template of a class or class template

14.4.2 Template Functions

- Imagine a **function** that would adapt itself to suit parameters of **different types**.
- Such a function is possible using **templates**.
- Let's look at a sample **template declaration** that returns the greater of two supplied parameters:

```
template <typename objType>
const objType& GetMax(const objType& value1, const objType& value2)
{
    if (value1 > value2)
        return value1;
    else
        return value2;
}

// usage:

int num1 = 25;
int num2 = 40;

int maxVal = GetMax<int>(num1, num2);

double double1 = 1.1;
double double2 = 1.001;
double maxVal = GetMax<double>(double1, double2);
```

- Note the `<int>` and `<double>`.
- It effectively **defines** the **template parameter** `objType` as `int` or `double` respectively.
- The preceding code leads to the compiler generating two versions of the **template function** `GetMax`, which can be visualized as the following:

```
const int& GetMax(const int& value1, const int& value2)
{
    // ...
}

const double& GetMax(const double& value1, const double& value2)
{
    // ...
}
```

- In reality, however, **template functions** don't necessarily need an **accompanying type specifier**. So the following function call works perfectly well:

```
int maxVal = GetMax(num1, num2);
```

- Compilers are intelligent enough to understand that the **template function** is being invoked for the **integer type** in this case.
- With **template classes**, however, you need to explicitly specify type, as shown in Listing 14.3

14.5 Templates and Type Safety

- Template functions `DisplayComparison()` and `GetMax()` shown in **Listing 14.3** are type safe.
- This means that they would not allow a meaningless call like this one:

```
DisplayComparison(num1, name1);
```

- This would immediately result in a **compile failure** (int and a string)

14.5.1 Template Classes

- Recall: **classes** are design blueprints, and the real-world representation of a **class** is an **object**.
- **Template classes** are the **templated versions** of C++ classes. They are blueprints of blueprints.
- When using a **template class**, you are given the option to specify the "**type**" you are specializing the **class** for.
- This enables you to create some humans with **template parameters** Age as a long long, some with int, and some with Age as an integer of type short.

- A simple **template class** that uses a **single parameter** T to hold a **member variable** can be written as the following:

```
template <typename T>
class HoldVarTypeT
{
private:
    T value;

public:
    void SetValue (const T& newValue) { value = newValue; }
    T& GetValue() { return value; }
};
```

- The **type** of the **variable** is T, and that is assigned at the time the **template** is used, that is, **instantiated**.

- So, let's look at a sample usage of this **template class**:

```
HoldVarTypeT <char> holdInt; // template instantiation for int
holdInt.SetValue(5);
cout << "The value stored is: " << holdInt.GetValue() << endl;
```

- You have used this **template class** to hold and retrieve an **object** of **type** int; that is, the Template class is **instantiated** for a **template parameter** of **type** int.

- You can do this for other types like for **character strings**:

```
HoldVarTypeT <char*> holdStr;
holdStr.SetValue("Sample string");
cout << "The value stored is: " << holdStr.GetValue() << endl;
```

- Thus, the **template class** defines a pattern for classes and helps implement that pattern on **different data types** that the **template** may be **instantiated** with.

- **Template classes** can be **instantiated** with types other than simple ones like `int` or classes supplied by the standard library.
- You may **instantiate** a **template** using a **class** defined by you..
- For example:

```
HoldVarTypeT<Human> holdHuman;  
holdHuman.SetValue(firstMan);  
holdHuman.GetValue().IntroduceSelf();
```

- For **class templates**, the template arguments must generally be specified inside `<>` when creating an object, such as `HoldVarTypeT<Human> holdHuman;`, because the compiler needs to know what type should replace the template parameter.

However, since **C++17**, **Class Template Argument Deduction (CTAD)** allows the compiler to infer the template arguments from constructor arguments in some cases, allowing the `<>` to be omitted.

14.5.2 Declaring Templates with Multiple Parameters

- The `template parameter list` can be expanded to **declare multiple parameters** separated by a comma.
- So you can declare a generic class that holds a pair of **objects** that can be of **differing types**:

```
template <typename T1, typename T2>
class HoldsPair
{
private:
    T1 value1;
    T2 value2;

public:
    // constructor that initializes member variables
    HoldsPair(const T1& val1, const T2& val2)
    {
        value1 = val1;
        value2 = val2;
    }

    // ... other member functions
};
```

- In this example, class `HoldsPair` accepts **two template parameters** named `T1` and `T2`.
- We can use this class to hold **two objects** of the **same type** or of **different types** as you can see here:

```
// a template instantiation that pairs an int with a double
HoldsPair<int, double> pairIntDouble (6, 1.99);

// a template instantiation that pairs an int with an int
HoldsPair<int, int> pairIntInt (6, 500);
```

14.5.3 Declaring Templates with Default Parameters

- We can modify the previous version of `HoldsPair` <...> to declare `int` as the *default template parameter type*:

```
template <typename T1=int, typename T2=int>
class HoldsPair
{
    // ... method declarations
};
```

- This is similar in **construction** to **functions** that **define default input parameter values** except for the fact that, in this case, we define **default *types***.
- The second usage of `HoldsPair` can thus be compacted to:

```
// pair an int with an int (default type)
HoldsPair<> pairsInts(6, 500);
```

14.5.4 Sample Template 'class<>' HoldsPair

- **Listing 14.4** A Template Class with a Pair of Member Attributes

14.5.5 Template Instantiation and Specilization

- A **template class** is a **blueprint** of a **class** (a blueprint of a blueprint), and therefore doesn't truly exist for the compiler before it has been used in one form or another.
- That is, as far as the compiler is concerned, a **template class** you **define** but don't consume in code that is simply ignored.
- However, once you instantiate a **template class**, like `HoldsPair`, by **supplying template arguments** like this:

```
HoldsPair<int, double> pairIntDbl;
```

- You are instructing the compiler to create a **class** for you using the **template** and **instantiate** it for the **types specified** as **template arguments** (`int` and `double` in this case).
- Thus, in the case of **templates**, *instantiation* is the act of process of creating a **specific type** using one ore more **template arguments**.

- On the other hand, there may be situations that require you to **explicitly define** a (different) **behavior** of a **template** when **instantiated** with a **specific type**.
- This is where you *specialize a template* (or **behavior thereof**) for that type.
- A *specilization* of a **template class** `HoldsPair` when instantiated with **template parameters** both of type `int` would look like this:

```
template<> class HoldsPair<int, int>
{
    // implementation code here
}
```

- Needless to say, code that **specializes a template** must follow the **template definition**.
- **Listing 14.5** Demonstrates Template Specialization

- **Template specialization** allows you to provide a custom implementation of a class template for a specific combination of types. In this example, `HoldsPair<int, int>` has a specialized version that behaves differently from the general template.

- Specialization is **useful** when a **particular type combination** requires **different behavior, optimizations, or additional functionality** that does not apply to all possible template types.
- The general template `HoldsPair<T1, T2>` provides a reusable implementation for any pair of types, while the specialized version `HoldsPair<int, int>` allows the programmer to override that behavior for integers.
- Template specialization avoids duplicating an entire class for every type combination while still allowing precise control when a specific case needs custom handling.
- In this example, the specialized `GetFirstValue()` function prints a message before returning the integer value, demonstrating how specialization can modify the behavior of individual template instances.

14.5.6 Template Classes and 'static' Members

- We learned how code in **templates** *begins to exist* for the **compiler** when used and not otherwise.
- So, how would **static member attributes** functions within a **template class**?
- In **lesson 9**, we've learned that **declaring a class member static** results in the member being shared across **all instances** of a **class**.
- It's similar with a **template class** too, save for the fact that a **static member** is shared across all objects of a **template class** with the same template instantiation.
- So, a **static member X** within a **template class** is static within **all instances** of the **class instantiated for int**.
- Similarly, **X** is also static within all instances of the class **specialized for double**, independent of the other **template instantiation** for **int**.
- In other words, you can visualize it as the compiler creating **two versions** of the **static member variable** in a **template class**:
 - **X_int** for **template instantiation** as **int** and
 - for **template instantiations** as **double**
- A **template class** does not generate actual code for the compiler until a specific type is used. The compiler creates a separate class definition for each **template instantiation**.
- This means that a **static member** in a template class is not shared across all possible template types; instead, it is shared only among objects that use the **same template instantiation**.
- For example, **MyClass<int>** and **MyClass<double>** are treated as two separate classes by the compiler. Therefore, each one receives its own independent copy of the static member variable.
- If a template class contains a static member **X**, then all objects of **MyClass<int>** share the same **X_int**, while all objects of **MyClass<double>** share a separate **X_double**.
- Conceptually, the compiler creates separate versions of the static member for each type:
 - * **X_int** for the **int** template instantiation.
 - * **X_double** for the **double** template instantiation.
- Thus, static members in templates are **shared within each template instantiation**, but are **independent between different template instantiations**.
- **Listing 14.6** The Effect of Static Variables on Template Class and Instances Thereof

- **Static member instantiation** syntax for a **template class** is **not to be missed** here:

```
// static member initialization
template<typename T> int TestStatic<T>::staticVal;
```

- This follows the pattern:

```
template<template parameters> StaticType
ClassName<Template Arguments>::StaticVarName;
```

```

#include <iostream>
using namespace std;

template <typename T>
class TestStatic
{
public:
    static int staticVal;
};

// static member initialization
template<typename T> int TestStatic<T>::staticVal;

```

- The class `TestStatic` is a **class template**, meaning the compiler creates a separate version of the class for each type used as the template argument.
- The declaration:

```
static int staticVal;
```

declares a **static member variable** that is shared among all objects of the same template instantiation.

- For example, all objects of `TestStatic<int>` share one copy of `staticVal`, while all objects of `TestStatic<double>` share a separate copy. The static variables are independent because `TestStatic<int>` and `TestStatic<double>` are different classes created by the compiler.
- The definition:

```
template<typename T> int TestStatic<T>::staticVal;
```

is required because the declaration inside the class only tells the compiler that the static member exists. This line actually allocates storage for the static member for each template instantiation.

- The compiler conceptually creates separate static variables for each type:
 - `TestStatic<int>::staticVal` for the `int` instantiation.
 - `TestStatic<double>::staticVal` for the `double` instantiation.
- Therefore, a static member inside a template class is **shared among objects of the same instantiated type**, but it is **separate between different template instantiations**.

- It's very similar to normal static declaration:

```

class Test
{
public:
    static int staticVal;
};

// definition of static member
int Test::staticVal;

```

14.5.7 Variable Templates, Also Called Variadic Templates

- Let's assume you want to write a **generic function** that adds two values.

Template function `Sum()` achieves just that:

```
template <typename T1, typename T2, typename T3>
void Sum(T1& res, T2 num1, T3 num2)
{
    res = num1 + num2;
    return;
}
```

- This is simple, however, if you were required to write **one single function** that would be capable of **adding any number of values**, each passed an **argument**, you would need to make use of `variable templates` in defining such a function.
- **Variable templates** or **variadic templates** have been part of C++ since C++14.
- **Listing 14.7** Function Using Variadic Templates Demonstrates Variable Arguments

- Note the **ellipsis mark ...**
- **Ellipsis** in C++ used with **templates** tell the compiler that the **template class** or **function** may accept an arbitrary number of template arguments of any type.

- **Variable templates** are powerful addition to C++.
- Programmers using **variable templates** save themselves the repetitive effort of implementing functions that perform a task in various **overloaded versions**, creating code that is shorter and simpler to maintain.

- Note: C++14 supplies you with an **operator** that would tell the **number of template arguments** passed in a call to a **variable templated**.
- In **Listing 14.7**, you could use this **operator** inside a function like `Sum()`, like this:

```
int arrNums[sizeof...(Rest)];
// length of array evaluated using sizeof...() at compile time
```

- You must not confuse `sizeof...()` with `sizeof(Type)`.
- The **latter** returns the **size** of a **type**; the **former** returns the **number of template arguments** sent to a **variadic template**.

- The support of **variable templates** has ushered in standard support of `tuples`: `std::tuple`
- `std::tuple` is the **class template** that implements the tuple and it may be **instantiated** with a varying number of member elements and types thereof.
- These may be **individually accessed** using standard library function `std::get`
- **Listing 14.8** Instantiating and Using a 'std::tuple'

14.5.8 Using 'static_assert' to Perform Compile-Time Checks

- The `static_assert` keyword allows you to perform **compile-time checks** by verifying that a condition is true while the program is being compiled.
- Unlike the `assert` macro, which checks conditions at **runtime**, `static_assert` checks conditions before the program is executed. If the condition evaluates to **false**, compilation fails and an error message is displayed.
- `static_assert` is useful for catching programming errors early, enforcing requirements, and ensuring that assumptions about types, sizes, or values remain valid.
- The syntax is:

```
static_assert(condition, "error message");
```

- The condition must be a **constant expression** that the compiler can evaluate during compilation.
- For example:

```
#include <iostream>

template <typename T>
class Array
{
private:
    T data[10];

public:
    Array()
    {
        static_assert(sizeof(T) <= 4,
                        "Type is too large for this array");
    }
};

int main()
{
    Array<int> intArray;          // Compiles successfully

    // Array<double> doubleArray;
    // Compilation error if sizeof(double) > 4

    return 0;
}
```

- In this example, `static_assert` ensures that the template type satisfies a requirement before generating the class. If a type violates the condition, the compiler stops and reports the provided error message.
- `static_assert` is especially useful with **templates**, because templates can accept many different types, and compile-time checks can prevent invalid template instantiations from being created.

- Listing 14.9 static_assert

- **DO** use **templates** for implementation of **generic concepts**.
- **DO** choose **templates** over **macros**.

- **DON'T** forget to use the principles of **const** correctness when programming **template functions** and **classes**.
- **DON'T** forget that a **static member** contained within a **template class** is **static** for every type-specilization of the **class**.

15 Lambda Expressions

- **Lambda expressions** are a compact way to **define** and **construct** function objects without a name.
- Introduced in C++11.

15.1 What is a Lambda Expression?

- A `lambda expression` can be visualized as a **compact** version of an **unnamed struct** (or class) with a **public operator()**.
- In that sense, a **lambda expressions** is a **function object**.
- Let's look into what is a `function object`:

```
// struct that behaves as a unary function
template <typename elementType>
struct DisplayElement
{
    void operator() (const elementType& element) const
    {
        cout << element << ' ';
    }
};
```

- The **function object** displays an **object element** on the screen using `cout` and is typically used in algos such as `std::for_each()`:

```
// display every integer contained in a vector
for_each (numsInVec.cbegin(), // start of range
          numsInVec.cend(),   // end of range
          DisplayElement<int>()); // unary function object
```

- A **lambda expression** compacts the entire code including the **definition** of the function object into three lines:

```
for_each (numsInVec.cbegin(), // start of range
          numsInVec.cend(),   // end of range
          [](const int& element) { cout << element << ' '; });
```

- When the compiler sees the **lambda expression**, in this case

```
[](const int& element) { cout << element << ' '; }
```

it automatically **expands** this expression into a representation that is similar to `struct DisplayElement<int>`:


```
struct NoName
{
    void operator() (const int& element) const
    {
        cout << element << ' ';
    }
};
```

- Note: **lambda expressions** are also called *lambda functions*.

15.2 How to Define a Lambda Expression

- The **definition** of a **lambda expression** has to start with **square brackets** []
- These **brackets** tell the compiler that the **lambda expression** has started.
- They are followed by a **parameter list**, which is the same as the **parameter list** you would supply your implementation of `operator()` if you were not using a **lambda expression**.

15.3 Lambda Expressions for a Unary Function

- The **lambda** version of a **unary operator**(Type) that takes **one parameter** would be the following:

```
[](Type paramName) { // lambda expression code here; }
```

- Note that you can pass the **parameter by reference** if you so wish:

```
[](Type& paramName) { // lambda expression code here; }
```

- **Listing 22.1** `for_each()` and lambda expression instead of function object

15.4 Lambda Expression for a Unary Predicate

- A `predicate` helps make decisions.
- A `unary predicate` is a **unary expression** that returns a `bool`: true or false.
- **Lambda expressions** can return values too.
- Example: **lambda expression** that returns true for even nums:

```
[](int num) { return (n & 1) == 0; }
```

- The nature of the **return value** in this case tells the compiler that the **lambda expressions** returns a `bool`
- You can use a **lambda expression** that is a **unary predicate** in algorithms, such as `std::find_if()`, to find even numbers in a collection.
- **Listing 22.2** `std::find_if()`

15.5 Lambda Expression with State via Capture Lists [...]

- In **Listing 22.2** we created a **unary predicate** that returned true if an int was even.
- What if we want to make it more generic to see if the int is divisible by a divisor of the user's choice—you need to maintain that "state" (the divisor) in the expression?

```
int divisor = 3; // initi value
auto element = find_if(numsInVec.cbegin(),
    numsInVec.cend(),
    [divisor](int dividend) { return (dividend % divisor) == 0; });
```

or

```
int divisor = userChoice;

auto pred = [divisor](int x) {
    return x % divisor == 0;
};
```

- **State** in a lambda means the values it **captures** from its surrounding scope.
- In this example, the lambda captures **divisor**:

```
[divisor](int dividend) {
    return (dividend % divisor) == 0;
}
```

- The captured **divisor** is stored inside the lambda, allowing it to remember the user's chosen divisor each time it is called.

- **Listing 22.3** Lambda Expressions that hold state

15.6 The Generic Syntax of Lambda Expressions

- A **lambda expressions** always starts with **square brackets** `[]` and can be configured to take **multiple state variables** separated using commas in a **capture list** `[...]`:

```
[stateVar1, stateVar2](Type& param) { // lambda code here; }
```

- If you want to ensure that these **state variables** are modified within a lambda, you add key-word `mutable`:

```
[stateVar1, stateVar2](Type& param) mutable { // lambda code here; }
```

- Note here the variables supplied in the **capture list** `[]` are **modifiable** within the lambda, but changes do not take effect outside of it.
- If you want to ensure that modifications made to the **state variables** within the lambda are valid **outside it**, then you use references:

```
[&stateVar1, &stateVar2](Type& param) { // lambda code here; }
```

- **Lambdas** can take **multiple input parameters**, separated by commas:

```
[stateVar1, stateVar2](Type1& var1, Type2& var2) { // lambda code here; }
```

- If you want to mention the **return type** and not leave the disambiguation to the compiler use the $\leftarrow$:

```
[stateVar1, stateVar2](Type1 var1, Type2 var2) -> ReturnType { // lambda code here; }
```

- Finally, the **compound statement** can hold multiple statements sep by a `;` (obviously)

```
[stateVar1, stateVar2] (Type1 var1, Type2 var2) -> ReturnType  
{  
    Statement 1;  
    Statement 2;  
    return (value or expression);  
}
```

- So if you lambda spans multiple lines, you are **required** to supply an **explicit return type**.

- Thus, a **lambda function** is a compact, fully functional replacement of a **function object** such as the following:

```
template<typename Type1, typename Type2>
struct IsNowTooLong
{
    // state variables
    Type1 var1;
    Type2 var2;

    // constructor
    IsNowTooLong(const Type1& in1, Type2& in2) : var1(in1), var2(in2) {};

    // the actual purpose
    ReturnType operator()
    {
        Statement 1;
        Statement 2;
        return (value or expression);
    }
};
```

15.7 Lambda Expression for a Binary Function

- a `binary function` takes **two parameters** and optionally returns a value.
- a `lambda expression` equivalent of the same would be

```
[...](Type1& param1Name, Type& param2Name) { // lambda code here; }
```

- `Listing 22.4` Lambda Expression as a Binary Function

15.8 Lambda Expression for a Binary Predicate

- A **binary function** that returns `true` or `false` to help make a decision is called a **binary predicate**.
- These `predicates` find use in sort algos, such as `std::sort()`, that invoke the **binary predicate** for any two values in a container to know which one should be placed after the other.
- Generic syntax of a **binary predicate** is:

```
[...](Type1& param1Name, Type2& param2Name) { // return bool expression; }
```

- **Listing 22.5** Lambda Expression as a Binary Predicate

- **DO** remember that **lambda expressions** always start with `[]` or `[state1, state2, ...]`
- **DO** remember that unless specified, **state variables** supplied within a **capture list** `[]` are **not modifiable** unless you use the keyword `mutable`

- **DON'T** forget that **lambda expressions** are **unnamed representations** of a **class** or a **struct** with **operator()**.
- **DON'T** forget to use `const` corrent parameter types when writing your **lambda expressions**:

```
[](const T& value) { // code; }
```

- **DON'T** forget to explicitly mention return type when **lambda expressions** includes **multiple statements** within the statement block
- **DON'T** choose lambda expressions over a **function object** when the lambda gets extremely long and spans multiple statements, for these are redefined in evrey use and do not assist code resuability.

16 Understanding Smart Pointers

- C++ can use **smart pointers** when managing memory on the heap (or free store).

16.1 What are Smart Pointers?

- A `smart pointer` in C++ is a **class** with **overloaded operators**, which behaves like a conventional pointer.
- Yet, it supplies additional value by ensuring proper and timely **destruction** of **dynamically allocated data** and facilitates a **well-defined** object lifecycle.

16.1.1 The Problem with Using Conventional (Raw) Pointers

- C++ supplies full flexibility to the programmer in memory allocation, deallocation, and management.
- One side of this, it makes C++ a powerful language; the other side: it allows the programmer to create memory-related problems, such as memory leaks, when dynamically allocated objects are not correctly released.
- Example:

```
SomeClass* ptrData = anObject.GetData();  
    // so here should caller or callee invoke delete/free here?  
    // no idea.  
ptrData->DoSomething();
```

- This can easily be solved, but it can also easily cause errors.

16.1.2 How Do Smart Pointers Help?

- The programmer can choose a smarter way to allocate and manage dynamic data by adopting the use of **smart pointers**:

```
smart_pointer<SomeClass> spData = anObject.GetData();

// use a smart pointer like a conventional pointer!
spData->Display();
(*spData).Display();

// don't have to worry about deallocation
// (the smart pointer's destructor does it for you!)
```

- Thus, **smart pointers** behave like **conventional (or raw) pointers** but supply useful features via their *overloaded operations* and *destructors* to ensure that **dynamically allocated data** is destroyed in a timely manner.

16.2 How Are Smart Pointers Implemented?

- Smart pointers, like above, can mimic raw pointers b/c of their **overloaded dereferencing operator** (*) and **member selection operator** (->).
- **Operator overloading** was discussed in Lesson 12.
- Additionally, almost all good **smart pointer classes** are **template classes** that contain a generic implementation of their functionality (so that you can manage a type of your choice on the heap).
- Being **templates**, they are versatile and can be **specialized** to manage an **object** of a **type** of your choice.
- **Listing 26.1** Minimal Essential Components of a Smart Pointer Class

- The listing displays the implementation of the **two operators** * and -> and assists in having this template class behave like a conventional pointer.
- For instance, you would use the **smart pointer** on an **object** of type **class Tuna**, you would instantiate it like this:

```
smart_pointer<Tuna> smartTuna(new Tuna);
smart_pointer->Swim();
(*smart_pointer).Swim();
```

- This is a very minimal example and don't have great benefits over a conventional pointer, however, it's nice to have the **constructor** accept a pointer that is saved as the **internal pointer object** in the **smart pointer class**.

And the **destructor** automatically freeing this pointer for **automatic memory release**.

- The implementation that makes a **smart pointer** really "smart" is the implementation of the **copy constructor**, the **assignment operator**, and the **destructor**.
- They determine the behavior of the **smart pointer object** when it is **passed across functions**, when is **assigned**, or when it **goes out of scope** (that is, gets **destroyed**)
- So, before looking at a **complete smart pointer implementation**, you should understand some **smart pointer types**.

16.3 Types of Smart Pointers

- The **management** of the **memory resource** (that is, the ownership model implemented) is what sets **smart pointer classes** apart.
- **Smart pointers** decide what they do with the resource when they are **copied** and **assigned** to.
- Classification of **smart pointers** is actually a classification of their **memory resource management** strategies. These are
 - Deep copy
 - Copy on Write (COW)
 - Reference counted
 - Reference linked
 - Destructive copy
- Let's look into these before studying C++'s `std::unique_ptr` (just an example of one of C++'s smart pointers)

16.3.1 Deep Copy

- In a **smart pointer** that implements **deep copy**, every **smart pointer instance** holds a **complete copy** of the **object** that is begin managed.
- Whenever the **smart pointer** is **copied**, the **object pointed to** is also copied (thus, **deep copy**).
- When the **smart pointer** goes out of scope, it **releases the memory** it points to (via the **destructor**)
- Although the **deep-copy-based smart pointer** does not seem to render any value over **passing objects by values**, its advantage becomes apparent in the treatment of **polymorphic objects** as seen in the following, where it can **avoid** *slicing*.

```
// example of Slicing when passing polymorphic objects by value
// fish is a base class for Tuna and Carp, Fish::Swim() is virtual
void MakeFishSwim(Fish aFish) // note parameter type
{
    aFish.Swim(); // virtual function
}

// ... some function
Carp freshWaterFish;
MakeFishSwim(freshWaterFish); // carp will be 'sliced' to Fish
// slicing: only the Fish part of Carp is sent to MakeFishSwim()

Tuna marineFish;
MakeFishSwim(marineFish); // slicing again

// so its an issue cause passing a polymorphic object by value
// slices off the derived-class portion,
// so polymorphism is lost and virtual functions
// behave as if the object were only the base class.
```

- **Slicing** issues are resolved when the programmer chooses a **deep-copy smart pointer**, as shown in **Listing 26.2**
- `deepcopy_smart_ptr` implements a **copy constructor** that allows a **deep copy** of the **polymorphic object** via a `Clone()` function that the **class** needs to implement.
- Similarly, it implements a **copy assignment operator**.
- Again, for the sake of simplicity, it is taken for granted in this ex that the **virtual function** implemented by the **base class** `Fish` is called `Clone()`.
- Typically **smart pointers** that implement **deep-copy models** have this function supplied either as a **template parameter** or a **function object**.
- Thus, when the **smart pointer** itself is passed as a **pointer** to **base class** type `Fish`:

```
deepcopy_smart_ptr<Carp> freshWaterFish(new Carp);  
MakeFishSwim(freshWaterFish); // carp will not be 'sliced'
```

- The **deep copy** implemented in the **smart pointer's constructor** kicks in to ensure that the **object** being **passed** is not sliced, even though syntactically only the **base part** of it is required by the destination function `MakeFishSwim()`
- The disadvantage of the **deep-copy-based** mechanism is performance.
- Instead the programmer may want to pass a **base type pointer (conventional pointer, `Fish*`)** to functions such as `MakeFishSwim()`

16.3.2 Copy on Write Mechanism

- **Copy on Write (COW)** attempts to **optimize performance** of **deep-copy smart pointers** by **sharing pointers** until the first attempt at writing to the **object** is made.
- One the **first attempt** at invoking a **non-const function**, a **COW pointer** typically **creates a copy** of the **object** on which the **non-const function** is invoked, whereas other **instances** of the **pointer** continue sharing the source object.
- **COW** has its fair share of fans.
- For those that swear by **COW**, implementing **operators (*)** and **(->)** in their **const** and **non-const** versions is key to the functionality of the **COW pointer**.
- The latter **creates a copy**.
- Know your shit or you will copy too little or too much

16.3.3 Reference-Counted Smart Pointers

- **Reference counting** in general is a mechanism that keeps a count of the **number of users** of an **object**.
- When the count reduces to **zero**, the **object is released**.
- So, **reference counting** makes a very good mechanism for **sharing objects** without having to **copy** them.
- Such **smart pointers**, when **copied**, need to have the **reference count** of the **object** in question **incremented**.
- There are at least **two** popular ways to keep this **count**:
 1. Reference count maintained in the **object being pointed to**.
 2. Reference count maintained by the **pointer class** in a **shared object**.
- The **first variant** where the **reference count** is maintained in the **object** is called *intrusive reference counting* because the **object** needs to be modified.
- The **object** in this case maintains, increments, and supplies the **reference count** to any **smart pointer class** that manages it.
- Incidentally, this is the approach chosen by **COM**.
- The **second variant** where the **reference count** is maintained in a **shared object** is a mechanism where the **smart pointer class** can keep the reference count on the free store (a dynamically allocated integer, for example) and **when copied**, the **copy constructor** increments this value.
- Therefore, the **reference-counting** mechanism makes it pertinent that the programmer works with the **smart pointers** only when using the object.
- A **smart pointer** managing the **object** and a **raw pointer** pointing to it is a bad idea because the **smart pointer** (smartly) releases the object when the count maintained by it goes down to zero, but the **raw pointer** continues pointing to the part of the memory that no longer belongs to your application.
- Similarly, **reference counting** can cause issues peculiar to their situation: **2 objects** that **hold a pointer to each other** are never released b/c their **cyclic dependency** holds their reference counts at a minimum of 1.

16.3.4 Reference-Linked Smart Pointers

- *Reference-linked smart pointers* are ones that don't proactively count the number of **references** of using the **object**; rather, they just need to know when the **number comes down to zero** so that the **object** can be released.
- They are called *reference-linked* because their implementation is based on a **double-linked list**.
- When a **new smart pointer** is created by **copying** an existing one, it is **appended** to the list.
- When a **smart pointer** goes **out of scope** or is **destroyed**, the destructor **de-indexes** the **smart pointer** from the list.
- **Reference counting** also suffers from the problem caused by **cyclic dependency**, as applicable to **reference-counted pointers**.

16.3.5 Destructive Copy

- **Destructive copy** is a mechanism where a **smart pointer**, when **copied**, transfers **complete ownership** of the object being handled to the destination and resets itself:

```
destructive_copy_smartptr<SampleClass> smartPtr(new SampleClass());

SomeFunc(smartPtr); // ownership transferred to SomeFunc
// don't use smartPtr in the caller any more!
```

- Although this mechanism is obviously intuitive to use, the advantage supplied by **destructive copy smart pointers** is that they ensure that at any point in time, **only one active pointer points** to an **object**.
- So, they make good mechanisms for **returning pointers** from functions, and are of use in scenarios where you can use their "destructive" properties to your advantage.
- The implementation of **destructive copy pointers** deviate from standard, recommended C++ programming techniques, as shown in **Listing 26.3**.
- **Listing 26.3** A Sample Destructive Copy Smart Pointer

- The **copy constructor** and **copy assignment operators** that are critical to the implementation of **destructive copy smart pointers** as shown in **Listing 26.3** also attract maximum criticism.
- Unlike most C++ classes, this **smart pointer class** cannot have the **copy constructor** and **assignment operator** accept **const references**, as it needs to invalidate the **source** after copying it.
- This is not only a deviation from traditional copy-constructor and assignment-operator semantics, but also makes using the smart pointer class counter intuitive.
- Few expect the **copy source** or the **assignment source** to be damaged after a **copy** or **assignment** step. counter intuitive.
- The fact that such **smart pointers** **destroy** the **source** also makes them unsuitable for the use in the **STL containers**, such as `std::vector` or any other dynamic collection class that you might use.
- These **containers** need to **copy** your content internally and end up invalidating the pointers.
- So, for more than one reason, you are **advised to avoid using destructive copy smart pointers** in your programs.

16.3.6 Using `std::unique_ptr`

- `std::unique_ptr` does not allow **copy** or **assignment**
- Use `#include <memory>` to use `std::unique_ptr`
- The `std::unique_ptr` is a **simple smart pointer** similar to what's shown in **Listing 26.1**, but with a **private copy constructor** and **assignment operator** to disallow copy via passing as an **argument to a function by value** or **copy via assignment**
- Listing 26.4 Using `std::unique_ptr`

- **Listing 26.4** demonstrated that the `unique_ptr` doesn't support copy:

```
copySmartFish = smartFish; // error: operator= is private
```

- It however **does support** `move semantics`.

Therefore, an option that would work is

```
unique_ptr<Fish> sameFish(std::move(smartFish));  
// smartFish is empty henceforth
```

- If you were ever to write a **lambda expression** that would need to capture an `unique_ptr`, then you would use `std::move()` in your **lambda capture** as supported by C++14.

```
std::unique_ptr<char> alphabet(new char);  
*alphabet = 's';  
auto lambda = [capture = std::move(alphabet)]()  
{ std::cout << *capture << endl; }  
  
// alphabet is empty henceforth as contents have been 'moved'  
  
lambda();
```

16.4 Smart pointers `std::`

16.4.1 `std::unique_ptr`

- A smart pointer that owns a dynamically allocated object exclusively.
- Automatically deletes the object when the `std::unique_ptr` goes out of scope (RAII).
- Cannot be copied, only moved, ensuring a single owner at all times.
- Commonly created with `std::make_unique` (C++14+).
- Useful for managing resources without manual `delete` calls.

```
#include <iostream>
#include <memory>

int main() {
    auto ptr = std::make_unique<int>(42);

    std::cout << *ptr << std::endl;    // 42

    // Ownership can be transferred
    auto ptr2 = std::move(ptr);

    if (!ptr)
        std::cout << "ptr is now empty\n";

    std::cout << *ptr2 << std::endl;    // 42
}
```

- Managing a user-defined object:

```
#include <iostream>
#include <memory>

class Dog {
public:
    Dog() { std::cout << "Dog created\n"; }
    ~Dog() { std::cout << "Dog destroyed\n"; }

    void bark() { std::cout << "Woof!\n"; }
};

int main() {
    auto dog = std::make_unique<Dog>();
    dog->bark();
} // Dog is automatically destroyed here
```

- Transferring ownership with `std::move()`:

```
#include <iostream>
#include <memory>

void takeOwnership(std::unique_ptr<int> p) {
    std::cout << *p << std::endl;
} // p is destroyed here
```

```
int main() {  
    auto ptr = std::make_unique<int>(100);  
  
    takeOwnership(std::move(ptr));  
  
    if (!ptr)  
        std::cout << "ptr no longer owns the object\n";  
}
```


16.4.2 `std::shared_ptr`

- A smart pointer that allows multiple owners of the same dynamically allocated object.
- Uses reference counting to track how many `std::shared_ptr`s own the object.
- The object is automatically deleted when the last owning `std::shared_ptr` is destroyed or reset.
- Can be copied freely, increasing the reference count.
- Commonly created with `std::make_shared`, which is more efficient than using `new`.

```
#include <iostream>
#include <memory>

int main() {
    auto ptr1 = std::make_shared<int>(42);
    auto ptr2 = ptr1;    // Shared ownership

    std::cout << *ptr1 << std::endl;    // 42
    std::cout << ptr1.use_count() << std::endl; // 2
}
```

- Managing a user-defined object:

```
#include <iostream>
#include <memory>

class Dog {
public:
    Dog() { std::cout << "Dog created\n"; }
    ~Dog() { std::cout << "Dog destroyed\n"; }

    void bark() { std::cout << "Woof!\n"; }
};

int main() {
    auto dog1 = std::make_shared<Dog>();
    auto dog2 = dog1;    // Both point to the same Dog

    dog1->bark();

    std::cout << dog1.use_count() << std::endl; // 2
} // Dog destroyed after both pointers go out of scope
```

- Sharing ownership by passing to a function:

```
#include <iostream>
#include <memory>

void printValue(std::shared_ptr<int> p) {
    std::cout << *p << std::endl;
    std::cout << p.use_count() << std::endl;
}

int main() {
    auto ptr = std::make_shared<int>(100);
```

```

        std::cout << ptr.use_count() << std::endl; // 1

        printValue(ptr);    // Copies the shared_ptr

        std::cout << ptr.use_count() << std::endl; // 1
    }

```

- Releasing ownership:

```

#include <iostream>
#include <memory>

int main() {
    auto ptr1 = std::make_shared<int>(10);
    auto ptr2 = ptr1;

    std::cout << ptr1.use_count() << std::endl; // 2

    ptr2.reset();    // Releases one owner

    std::cout << ptr1.use_count() << std::endl; // 1
}

```

16.4.3 std::weak_ptr

- A smart pointer that observes an object managed by a `std::shared_ptr`.
- Does not own the object and does not increase the reference count.
- Used to prevent circular references between `std::shared_ptr`s.
- Must be converted to a `std::shared_ptr` using `lock()` before accessing the object.
- Can check whether the object still exists using `expired()`.

```
#include <iostream>
#include <memory>

int main() {
    auto shared = std::make_shared<int>(42);
    std::weak_ptr<int> weak = shared;

    std::cout << shared.use_count() << std::endl; // 1

    if (auto ptr = weak.lock()) {
        std::cout << *ptr << std::endl; // 42
    }
}
```

- Detecting when an object has been destroyed:

```
#include <iostream>
#include <memory>

int main() {
    std::weak_ptr<int> weak;

    {
        auto shared = std::make_shared<int>(100);
        weak = shared;
    } // shared is destroyed here

    if (weak.expired()) {
        std::cout << "Object no longer exists\n";
    }
}
```

- Safely accessing the object with `lock()`:

```
#include <iostream>
#include <memory>

void printValue(std::weak_ptr<int> weak) {
    if (auto ptr = weak.lock()) {
        std::cout << *ptr << std::endl;
    } else {
        std::cout << "Object has been destroyed\n";
    }
}

int main() {
```

```
    auto shared = std::make_shared<int>(50);  
  
    printValue(shared);  
  
    shared.reset();  
  
    printValue(shared);  
}
```